RUST

CHEAT SHEETS

Safe, fast systems programming, from ownership to async

let / mut · Integers · String vs &str · match · if let · Ownership · Borrowing · Lifetimes · Option · Result · ? · Closures · Vec · HashMap · Iterators · Structs · Enums · Traits · Generics · dyn Trait · Box · Rc / Arc · RefCell · Cargo · Modules · Tests · Threads · Channels · async / await · Tokio · Serde · unsafe · Macros

107 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

001 Introduction & History
002 Installing with rustup
003 Cargo Basics
004 Program Structure
005 rustfmt, Clippy & Editions

OUTPUT & COMMENTS

006 Printing Macros
007 Format Syntax
008 Comments & Doc Comments

VARIABLES & DATA TYPES

009 Variables, Mutability & Shadowing
010 Constants & Statics
011 Integers
012 Floating-Point Numbers
013 Booleans & char
014 Tuples & the Unit Type
015 Type Conversion
016 Type Inference & Aliases

STRINGS

017 String vs &str
018 Building Strings
019 String Methods — Search & Inspect

ERROR HANDLING

041 panic! & Unrecoverable Errors
042 Option<T>
043 Result<T, E> & the ? Operator
044 Custom Error Types
045 thiserror & anyhow

FUNCTIONS & CLOSURES

046 Defining Functions
047 Parameters & Arguments
048 Closures
049 Fn, FnMut & FnOnce
050 Higher-Order Functions
051 const fn, Recursion & Attributes

COLLECTIONS & ITERATORS

052 Arrays
053 Vec — Basics
054 Vec — Methods
055 HashMap — Basics
056 Entry API & BTreeMap
057 HashSet & BTreeSet
058 VecDeque, BinaryHeap & Choosing
059 Iterators
060 Iterator Adapters & Consumers

FILES & I/O

077 Reading Files
078 Writing Files
079 Paths & Directories
080 stdin & stdout
081 Serde & JSON
082 CLI Arguments & Environment

MODULES, CRATES & CARGO

083 Modules
084 Visibility
085 Cargo Dependencies
086 Workspaces, Features & Profiles
087 Publishing & Documentation
088 Standard Library Tour

TESTING & TOOLING

089 Unit Tests
090 Integration & Doc Tests
091 Benchmarks & Debugging

CONCURRENCY & ASYNC

092 Threads
093 Channels
094 Shared State: Arc, Mutex & Atomics
095 Send & Sync

RS

Table of Contents

107 sheets · syntax · examples · notes

INTRODUCTION & SETUP 001 – 005

001 Introduction & History
002 Installing with rustup
003 Cargo Basics
004 Program Structure
005 rustfmt, Clippy & Edit…

OUTPUT & COMMENTS 006 – 008

006 Printing Macros
007 Format Syntax
008 Comments & Doc Co…

VARIABLES & DATA TYPES 009 – 016

009 Variables, Mutability …
010 Constants & Statics
011 Integers
012 Floating-Point Numb…
013 Booleans & char
014 Tuples & the Unit Type
015 Type Conversion
016 Type Inference & Alia…

STRINGS 017 – 021

017 String vs &str
018 Building Strings
019 String Methods — Se…
020 String Methods — Tra…
021 UTF-8, chars & bytes

OPERATORS 022 – 026

022 Arithmetic & Overflow
023 Comparison & Orderi…
024 Logical & Bitwise
025 Assignment & Ranges
026 Operator Overloading

CONTROL FLOW 027 – 033

027 if / else as Expressions
028 loop & Break Values
029 while & while let
030 for Loops
031 match Basics
032 Patterns in Depth
033 if let, let else & let Ch…

OWNERSHIP & BORROWING 034 – 040

034 Ownership & Moves
035 Copy vs Clone
036 References & Borrowi…
037 Mutable References
038 Slices
039 Lifetimes — Basics
040 Lifetimes — Structs &…

ERROR HANDLING 041 – 045

041 panic! & Unrecoverab…
042 Option<T>
043 Result<T, E> & the ? O…
044 Custom Error Types
045 thiserror & anyhow

FUNCTIONS & CLOSURES 046 – 051

046 Defining Functions
047 Parameters & Argum…
048 Closures
049 Fn, FnMut & FnOnce
050 Higher-Order Functio…
051 const fn, Recursion & …

COLLECTIONS & ITERATORS 052 – 061

052 Arrays
053 Vec — Basics
054 Vec — Methods
055 HashMap — Basics
056 Entry API & BTreeMap
057 HashSet & BTreeSet
058 VecDeque, BinaryHea…
059 Iterators
060 Iterator Adapters & C…
061 Sorting & Searching

STRUCTS, ENUMS & TRAITS 062 – 071

062 Structs
063 Methods & impl Blocks
064 Enums with Data
065 Traits — Basics
066 Default Methods, deri…
067 Common Standard Tr…
068 Generics
069 Trait Bounds & where
070 Trait Objects & dyn
071 Associated Types & …

SMART POINTERS 072 – 076

072 Box<T>
073 Rc<T> & Weak
074 Cell, RefCell & Interior…
075 Deref & Drop
076 Cow & Choosing a Poi…

FILES & I/O 077 – 082

077 Reading Files
078 Writing Files
079 Paths & Directories
080 stdin & stdout
081 Serde & JSON
082 CLI Arguments & Envi…

MODULES, CRATES & CARGO 083 – 088

083 Modules
084 Visibility
085 Cargo Dependencies
086 Workspaces, Feature…
087 Publishing & Docume…
088 Standard Library Tour

TESTING & TOOLING 089 – 091

089 Unit Tests
090 Integration & Doc Tes…
091 Benchmarks & Debug…

CONCURRENCY & ASYNC 092 – 097

092 Threads
093 Channels
094 Shared State: Arc, Mu…
095 Send & Sync
096 async / await & Futur…
097 Tokio

COMMON CRATES 098 – 101

098 HTTP Clients with req…
099 Web Servers with Axu…
100 Logging with log & tra…
101 Databases with sqlx

ADVANCED TOPICS 102 – 106

102 Declarative Macros
103 Procedural Macros
104 Unsafe Rust
105 FFI & Calling C
106 Idiomatic Patterns

QUICK REFERENCE 107

107 Quick Reference — S…

Sheet 001

Introduction & History

Mozilla Research, 2010 · Graydon Hoare · 1.0 in May 2015 · Rust Foundation (2021) · memory safety without GC · zero-cost abstractions

FAST, SAFE SYSTEMS PROGRAMMING

Rust started as Graydon Hoare's project, was sponsored by **Mozilla Research** from 2009–2010, and reached **1.0 in May 2015**. It gives C/C++-level speed with **memory safety enforced at compile time** by the borrow checker, with no garbage collector. Today it runs in Linux, Windows, Android, browsers, cloud infrastructure and CLI tools.

KEY POINTS

- Source files: `.rs` ; packages are **crates** built with **Cargo**
- New stable release every 6 weeks; **editions** (2015, 2018, 2021, 2024) group breaking changes
- The Rust Foundation (2021) stewards the project
- Known for great compiler error messages

HELLO, WORLD

```
// src/main.rs
fn main() {
    println!("Hello, World");             // Hello, World

    let name = std::env::args().nth(1).unwrap_or("World".into());
    println!("Hello, {name}!");           // Hello, Ada!
}

// $ cargo new hello && cd hello
// $ cargo run -- Ada
```

CODE EXPLAINED

- → `fn main()` is the entry point of a binary crate
- → `println!` ends in `!` because it is a **macro**, not a function
- → `args().nth(1)` returns an `Option` ; `unwrap_or` gives a default
- → `{name}` inlines a variable in the format string

COMING FROM…

- ↔ **C++** similar speed and control, but memory safety is the programmer's job
- ↔ **Go** also compiles to one binary, but uses a garbage collector
- ↔ **Python / Java** run on an interpreter or VM; Rust is compiled ahead of time

THE RUST WAY

- ⇄ learning Rust by fighting the borrow checker → read the error: it usually names the fix
- ⇄ `unwrap()` everywhere in real code → `?` and proper error types (sheet 043)

TIMELINE

YEAR	MILESTONE
2006	Graydon Hoare starts Rust
2015	Rust 1.0
2018	2018 edition, NLL borrow checker
2019	async/await stable
2021	Rust Foundation; 2021 edition
2025	2024 edition (Rust 1.85)

WHERE RUST RUNS

AREA	EXAMPLES
OS & kernels	Linux drivers, Windows
Web & cloud	Axum, Cloudflare, AWS Firecracker
CLI tools	ripgrep, fd, bat
Browsers	Firefox components
Embedded	no_std microcontrollers
WebAssembly	wasm-bindgen, Leptos

⚠ COMMON MISTAKES

- Expecting Rust to feel like a GC language: ownership changes how you design data
- Copying C++ patterns (shared mutable pointers) and fighting the compiler
- Skipping the compiler message, which usually suggests the fix

✓ TIPS & BEST PRACTICES

- Read "The Rust Programming Language" (the Book) free online
- Run `cargo clippy` from day one
- Try snippets at play.rust-lang.org

QUICK SUMMARY

- `fn main()` — entry point
- `println!` — macro
- borrow checker — safety at compile time
- editions — opt-in language changes

Installing with rustup

MANAGING RUST TOOLCHAINS

rustup installs and updates Rust and lets you switch between **stable**, **beta** and **nightly** toolchains. It also installs components (clippy, rustfmt, rust-analyzer, rust-src) and cross-compilation targets. A `rust-toolchain.toml` file in a project pins the toolchain for everyone who builds it.

KEY POINTS

- Install from rustup.rs (or `brew install rustup` on macOS)
- Tools live in `~/.cargo/bin` (add it to PATH)
- Use stable unless you need a nightly-only feature
- `rustup doc` opens the Book and std docs offline

RUSTUP COMMANDS

```
# Install (macOS / Linux)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

rustc --version             # rustc 1.9x.x (…)
cargo --version
rustup update               # get the newest stable

rustup toolchain install nightly
rustup default stable
cargo +nightly build        # one-off nightly build

rustup component add clippy rustfmt rust-analyzer
rustup target add wasm32-unknown-unknown
rustup target add aarch64-unknown-linux-gnu

rustup show                 # active toolchain and targets
rustup doc --book           # offline Book

# rust-toolchain.toml (project root)
# [toolchain]
# channel = "1.89.0"
# components = ["clippy", "rustfmt"]
```

CODE EXPLAINED

→ The installer script sets up rustup, rustc and cargo

→ `cargo +nightly` uses nightly for one command only

→ Targets add standard libraries for cross-compiling

→ `rust-toolchain.toml` makes the project build with the pinned version

COMING FROM…

↔ **Python** pyenv / uv manage interpreter versions

↔ **Java** SDKMAN! switches JDKs

↔ **JavaScript** nvm / fnm switch Node versions

THE RUST WAY

⇄ installing Rust from a distro package → rustup (current and switchable)

⇄ nightly by default → stable, nightly only for specific crates

CHANNELS

CHANNEL	USE
`stable`	Everyday work
`beta`	Next release preview
`nightly`	Unstable features (#![feature])

COMMANDS

COMMAND	DOES
`rustup update`	Update toolchains
`rustup default X`	Set default
`rustup override set X`	Per-directory
`rustup component add X`	Add tool
`rustup target add X`	Add target
`rustup self uninstall`	Remove all

⚠ COMMON MISTAKES

- `~/.cargo/bin` not on PATH, so `cargo` is not found
- Mixing a distro-installed rustc with rustup
- Depending on nightly features in a library others use on stable

✓ TIPS & BEST PRACTICES

- Commit `rust-toolchain.toml` so CI and teammates match
- Run `rustup update` regularly: releases are backward compatible
- Install rust-analyzer for editor support

QUICK SUMMARY

- rustup — toolchain manager
- stable / beta / nightly
- `component add` — tools
- `rust-toolchain.toml` — pin version

Sheet 003

Cargo Basics

cargo new / cargo init · cargo build / --release · cargo run · cargo check · target/ · Cargo.toml & Cargo.lock · cargo add

RUST'S BUILD TOOL AND PACKAGE MANAGER

Cargo creates projects, downloads dependencies from crates.io, builds, runs, tests and documents your code. `Cargo.toml` describes the package and its dependencies; `Cargo.lock` records exact versions. `cargo check` type-checks without producing a binary, which is much faster while editing.

KEY POINTS

- `cargo new app` = binary; `cargo new --lib mylib` = library
- Debug builds go to `target/debug`, release to `target/release`
- Commit `Cargo.lock` for applications
- Pass program arguments after `--`

EVERYDAY CARGO

```
cargo new hello             # binary crate with src/main.rs
cargo new --lib geometry    # library crate with src/lib.rs
cd hello

cargo check                 # fast type check, no binary
cargo build                 # debug build → target/debug/hello
cargo build --release       # optimized → target/release/hello
cargo run -- --name Ada     # build + run, args after --
cargo test                  # run tests
cargo add serde --features derive
cargo add tokio@1 -F full
cargo doc --open            # build and open docs
cargo clean                 # delete target/

# Cargo.toml
# [package]
# name = "hello"
# version = "0.1.0"
# edition = "2024"
#
# [dependencies]
# serde = { version = "1", features = ["derive"] }
```

CODE EXPLAINED

→ `cargo check` is the fastest feedback loop

→ `--release` turns on optimizations (much faster binaries)

→ `cargo add` edits Cargo.toml for you

→ `edition = "2024"` picks the language edition

COMING FROM…

↔ **Python** uv / pip + pyproject.toml

↔ **Java** Maven / Gradle with pom.xml or build.gradle.kts

↔ **JavaScript** npm with package.json + package-lock.json

THE RUST WAY

⇄ benchmarking debug builds → always measure `--release`

⇄ waiting on `cargo build` while editing → `cargo check` (or rust-analyzer)

COMMANDS

COMMAND	DOES
`cargo new / init`	Create project
`cargo check`	Type check
`cargo build [--release]`	Compile
`cargo run`	Build + run
`cargo test`	Tests
`cargo add / remove`	Dependencies
`cargo update`	Refresh Cargo.lock
`cargo fmt / clippy`	Format / lint

LAYOUT

PATH	HOLDS
`Cargo.toml`	Manifest
`Cargo.lock`	Exact versions
`src/main.rs`	Binary root
`src/lib.rs`	Library root
`src/bin/*.rs`	Extra binaries
`tests/`	Integration tests
`target/`	Build output

⚠ COMMON MISTAKES

- Timing debug builds and concluding Rust is slow
- Forgetting `--` before program arguments
- Deleting Cargo.lock in an application repo

✓ TIPS & BEST PRACTICES

- Use `cargo watch -x check` (cargo-watch crate) for live checking
- Add `[profile.release] lto = true` for smaller, faster binaries
- Use `cargo tree` to inspect dependencies

QUICK SUMMARY

- `cargo new` — create
- `cargo check` — fast check
- `cargo run --release`
- Cargo.toml / Cargo.lock

Sheet 004

Program Structure

`fn main()` · statements vs expressions · trailing expression & `;` · blocks `{}` · `main` returning `Result` · `main.rs` / `lib.rs` crate roots

EXPRESSIONS EVERYWHERE

A Rust program is a set of items (functions, structs, enums, modules) in a crate rooted at `main.rs` or `lib.rs`. Most constructs are **expressions** that produce values, including blocks, `if` and `match`. A block's value is its last expression **without** a semicolon; adding `;` turns it into a statement that yields `()`.

KEY POINTS

- `let` is a statement; `if`, `match`, blocks are expressions
- The semicolon matters: `x + 1` returns, `x + 1;` does not
- `main` can return `Result<(), E>` to use `?`
- Items can be declared in any order

BLOCKS, EXPRESSIONS AND MAIN

```rust
use std::fs;

fn area(w: f64, h: f64) -> f64 {
    w * h                          // no semicolon: this is the return value
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let a = area(3.0, 4.0);
    let label = if a > 10.0 { "big" } else { "small" };   // if is an expression

    let total = {                  // a block is an expression
        let tax = a * 0.06;
        a + tax                    // value of the block
    };

    let config = fs::read_to_string("config.toml")?;   // ? needs Result return
    println!("{label} {total:.2} {}", config.len());
    Ok(())                         // success value for main
}
```

CODE EXPLAINED

→ The last line of `area` is returned because it has no `;`

→ `if … else` produces a value assigned to `label`

→ The block computes `total` with a temporary `tax`

→ `main` returning Result lets `?` stop with an error message

COMING FROM…

↔ **Java** `if` is a statement; you need the ternary `?:` for values

↔ **Python** has conditional expressions, but blocks are not values

↔ **C++** blocks are statements; GCC statement expressions are non-standard

THE RUST WAY

⇄ `return x;` at the end of every function → trailing expression `x`

⇄ `unwrap()` in main → `main() -> Result` + `?`

STATEMENT VS EXPRESSION

CODE	KIND
`let x = 5;`	Statement
`x + 1`	Expression
`x + 1;`	Statement (value dropped)
`{ … }`	Expression
`if / match / loop`	Expressions
`fn item`	Item (declaration)

CRATE ROOTS

FILE	CRATE TYPE
`src/main.rs`	Binary
`src/lib.rs`	Library
`src/bin/x.rs`	Extra binary
`examples/x.rs`	Example (cargo run --example)

⚠ COMMON MISTAKES

- A stray `;` on the last line: "expected f64, found ()"
- Using `?` in a `main` that returns `()`
- Expecting `if` without `else` to produce a value

✓ TIPS & BEST PRACTICES

- Let the compiler's type errors guide you to missing or extra semicolons
- Use blocks to limit the scope of temporaries
- Return `Result` from main in CLI tools

QUICK SUMMARY

- expressions produce values
- no `;` — returned value
- blocks are expressions
- `main` → `Result` for `?`

Sheet 005

rustfmt, Clippy & Editions

cargo fmt · rustfmt.toml · cargo clippy · #[allow] / #[warn] / #[deny] · cargo fix --edition · rust-analyzer · editions 2015 / 2018 / 2021 / 2024

FORMATTING, LINTING AND EDITIONS

`cargo fmt` formats code to the community style, and `cargo clippy` adds hundreds of lints that catch bugs and suggest idiomatic code. **Editions** let Rust make breaking syntax changes without splitting the ecosystem: each crate picks an edition, and crates of different editions work together.

KEY POINTS

- Editions are set in Cargo.toml: `edition = "2024"`
- The 2024 edition shipped with Rust 1.85 (Feb 2025)
- `cargo fix --edition` migrates code automatically
- Lint levels: allow, warn, deny, forbid

TOOLS AND LINT ATTRIBUTES

```
// Terminal:
// cargo fmt                    format everything
// cargo fmt --check            CI: fail if not formatted
// cargo clippy                 lint
// cargo clippy -- -D warnings  CI: warnings become errors
// cargo fix --edition          migrate to the next edition

#![warn(clippy::pedantic)]      // crate-wide, stricter lints
#![deny(unsafe_code)]           // forbid unsafe in this crate

#[allow(clippy::too_many_arguments)]
fn build(a: i32, b: i32, c: i32, d: i32, e: i32, f: i32, g: i32, h: i32) {}

fn main() {
    let v = vec![1, 2, 3];
    if v.len() == 0 {}          // clippy: use `v.is_empty()`
    let x = 5;
    let _unused = x;            // leading _ silences unused warning
}

// rustfmt.toml
// max_width = 100
// imports_granularity = "Crate"
```

CODE EXPLAINED

→ `#![…]` applies to the whole crate; `#[…]` to the next item

→ Clippy catches style issues like `len() == 0`

→ `-D warnings` makes CI strict

→ An underscore prefix marks a variable as intentionally unused

COMING FROM…

↔ **Python** ruff / black / mypy

↔ **Java** Checkstyle, Spotless, SpotBugs; language levels via `--release`

↔ **JavaScript** Prettier + ESLint; ES versions via tsconfig `target`

THE RUST WAY

⇄ `#[allow(warnings)]` to silence noise → fix or allow specific lints

⇄ manual formatting debates → `cargo fmt` in CI

EDITIONS

EDITION	NOTABLE CHANGES
2015	Rust 1.0
2018	Path changes, async keyword, NLL
2021	Disjoint closure captures, IntoIterator for arrays
2024	let chains, unsafe extern, RPIT capture rules, gen keyword reserved

LINT LEVELS

LEVEL	EFFECT
allow	Silent
warn	Warning
deny	Error (can be overridden)
forbid	Error (cannot be overridden)
expect	Warn if lint does NOT fire

⚠ COMMON MISTAKES

- Not running Clippy and missing easy wins
- Thinking editions split crates (they interoperate)
- Blanket `#![allow(clippy::all)]`

✓ TIPS & BEST PRACTICES

- Run `cargo fmt --check` and `cargo clippy -- -D warnings` in CI
- Use `#[expect(lint)]` to document intentional exceptions
- Let rust-analyzer run clippy on save

QUICK SUMMARY

- `cargo fmt` — format
- `cargo clippy` — lint
- `#[allow]` / `#[deny]` — levels
- editions — opt-in changes

Printing Macros

OUTPUT WITH MACROS

Rust prints with macros: `println!` writes a line to stdout, `eprintln!` to stderr, and `format!` returns a `String` instead of printing. `write!` targets any writer, such as a file or String buffer. `dbg!` prints an expression with its file, line and value, then returns it.

KEY POINTS

- All take a format string checked at **compile time**
- `print!` does not flush; call `io::stdout().flush()` before input
- `dbg!` writes to stderr and takes ownership unless you pass `&x`
- Printing locks stdout per call; lock once for heavy output

OUTPUT MACROS

```
use std::io::{self, Write};
use std::fmt::Write as FmtWrite;

let name = "Ada";
let score = 92.456;

println!("Hello, {name}!");                  // Hello, Ada!
println!("{} scored {:.1}", name, score);    // Ada scored 92.5
print!("Enter value: ");
io::stdout().flush()?;                       // show prompt now
eprintln!("warning: low disk");              // stderr

let msg = format!("{name}-{}", 42);          // "Ada-42"

let mut s = String::new();
write!(s, "{}+{}", 1, 2)?;                   // write into a String
writeln!(io::stderr(), "logged")?;

let x = dbg!(score * 2.0);      // [src/main.rs:16:9] score * 2.0 = 184.912
dbg!(&msg);                                  // borrow, keep using msg

let stdout = io::stdout();
let mut out = stdout.lock();                 // fast bulk output
for i in 0..3 { writeln!(out, "line {i}")?; }
```

CODE EXPLAINED

- → `{name}` captures a variable; `{}` takes the next argument
- → `flush()` makes the prompt appear before waiting for input
- → `format!` builds a String with the same syntax
- → `dbg!` returns its value, so it can wrap expressions inline

COMING FROM…

- ↔ **C++** `std::println("Hello, {}", name)` (C++23) / `std::format`
- ↔ **Python** `print(f"Hello, {name}")`, `print(..., file=sys.stderr)`
- ↔ **Java** `System.out.printf("%s scored %.1f%n", name, score)`

THE RUST WAY

- ⇄ `println!` for debugging → `dbg!` (shows file, line, expression)
- ⇄ thousands of `println!` calls in a loop → lock stdout once

MACROS

MACRO	TARGET
`print! / println!`	stdout
`eprint! / eprintln!`	stderr
`format!`	Returns String
`write! / writeln!`	Any writer
`dbg!`	stderr + returns value
`panic!`	Message, then panic

⚠ COMMON MISTAKES

- Prompt not showing because `print!` was not flushed
- Passing a value to `dbg!` and losing ownership (use `&x`)
- Leaving `dbg!` calls in committed code

✓ TIPS & BEST PRACTICES

- Use inline captures `{name}` for readability
- Use the `log` / `tracing` crates for real logging (sheet 100)
- Clippy has a lint (`dbg_macro`) to catch leftover `dbg!`

QUICK SUMMARY

- `println!` / `eprintln!`
- `format!` — String
- `write!` — any writer
- `dbg!` — debug + return

Sheet 007

Format Syntax

{} Display vs {:?} Debug · {:#?} pretty · inline args {name} · positional {0} · width / fill / align {:>8} {:08.2} · {:x} {:b} {:e} · {{ escaping

FORMATTING VALUES

Format strings use {} placeholders. {} uses the **Display** trait (user-facing text) and {:?} uses **Debug** (developer view, derivable). After the colon come options: fill, alignment, width, precision and type, so {:>8.2} right-aligns a number in 8 columns with 2 decimals.

KEY POINTS

- Order: `{name:fill align sign # 0 width .precision type}`
- Width and precision can come from arguments: `{:>w$}`
- `#[derive(Debug)]` gives any struct `{:?}`
- Implement `Display` yourself for `{}` (sheet 067)

FORMAT OPTIONS

```
#[derive(Debug)]
struct Point { x: i32, y: i32 }
let p = Point { x: 3, y: -4 };
let pi = 3.14159;
let n = 255;
let w = 10;

println!("{p:?}");              // Point { x: 3, y: -4 }
println!("{p:#?}");             // pretty, multi-line
println!("{0} {1} {0}", "a", "b");   // a b a
println!("[{:>8}]", "right");  // [   right]
println!("[{:<8}]", "left");   // [left    ]
println!("[{:^8}]", "mid");    // [  mid   ]
println!("[{:*^9}]", "x");     // [****x****]
println!("{:.2}", pi);         // 3.14
println!("{:08.3}", pi);       // 0003.142
println!("{:+}", 5);           // +5
println!("{n:x} {n:X} {n:#x}"); // ff FF 0xff
println!("{n:b} {n:o}");       // 11111111 377
println!("{:e}", 1234.5);      // 1.2345e3
println!("[{:>w$}]", "dyn");   // [       dyn]
println!("{{literal}}");       // {literal}
```

CODE EXPLAINED

- → `:?` needs `Debug`; `:#?` pretty-prints it
- → `>`, `<`, `^` align right, left and center
- → `08.3` pads with zeros to width 8, 3 decimals
- → `w$` reads the width from the variable `w`

COMING FROM…

- ↔ **C++** `std::format("{:>8.2f}", x)` uses almost the same syntax
- ↔ **Python** f-strings: `f"{x:>8.2f}"` (Rust's syntax is modeled on it)
- ↔ **Java** `String.format("%8.2f", x)`

THE RUST WAY

- ⇄ manual padding with loops → width/fill options
- ⇄ implementing Debug by hand → `#[derive(Debug)]`

SPEC PARTS

PART	EXAMPLE
Display	{}
Debug	{:?} / {:#?}
Align + width	{:>8} {:<8} {:^8}
Fill	{:*^9}
Precision	{:.2}
Zero pad	{:08.3}
Sign	{:+}
Radix	{:x} {:b} {:o} {:#x}

⚠ COMMON MISTAKES

- `{}` on a type without Display ("doesn't implement Display")
- Expecting `{:.2}` to round integers (it applies to floats and strings)
- Unescaped `{` in a format string

✓ TIPS & BEST PRACTICES

- Derive Debug on nearly every type
- Implement Display for types users see
- Use `{:#?}` for nested structs in logs

QUICK SUMMARY

- `{}` Display · `{:?}` Debug
- `{:>8.2}` — align, width, precision
- `{:x}` / `{:b}` — radix
- `{{` — literal brace

Comments & Doc Comments

COMMENTS AND DOCUMENTATION

Rust has // line comments and nestable /* */ block comments. **Doc comments** use Markdown: /// documents the item that follows and //! documents the enclosing crate or module. `cargo doc` turns them into HTML, and code blocks in doc comments run as **tests**.

KEY POINTS

- Doc comments support headings, lists, links and code blocks
- Standard sections: Examples, Panics, Errors, Safety
- Link to items with `[ Type ]` intra-doc links
- docs.rs builds docs for every published crate

DOC COMMENTS

```rust
//! # geometry
//! Shapes and area calculations.

/// A rectangle with a width and height.
///
/// See also [`Circle`].
pub struct Rect { pub w: f64, pub h: f64 }

/// Returns the area of `r`.
///
/// # Examples
///
/// ```
/// use geometry::{Rect, area};
/// let r = Rect { w: 3.0, h: 4.0 };
/// assert_eq!(area(&r), 12.0);
/// ```
///
/// # Panics
///
/// Panics if a side is negative.
pub fn area(r: &Rect) -> f64 {
    assert!(r.w >= 0.0 && r.h >= 0.0, "negative side");
    r.w * r.h  // regular comment
    /* block /* nested */ comment */
}
```

CODE EXPLAINED

→ `//!` at the top documents the crate itself

→ `[ Circle ]` becomes a link in generated docs

→ The example in `runs during` cargo test`

→ `# Panics` tells callers when the function can panic

COMING FROM…

↔ **C++** Doxygen `///` or `/** */` comments

↔ **Python** docstrings + doctest

↔ **Java** Javadoc `/** */` with HTML and `@param` tags

THE RUST WAY

⇄ docs that drift from code → doc tests that run in CI

⇄ no docs on `pub` items → `#![warn(missing_docs)]`

COMMENT KINDS

SYNTAX	DOCUMENTS
//	Nothing (comment)
/* */	Nothing (nestable)
///	Next item
//!	Enclosing crate/module
/** */	Next item (rare)

DOC SECTIONS

HEADING	USE
# Examples	Runnable usage
# Panics	When it panics
# Errors	When it returns Err
# Safety	unsafe fn requirements

⚠ COMMON MISTAKES

- Doc examples that no longer compile (they fail `cargo test`)
- Using `//` where `///` was intended (no docs generated)
- Missing `# Safety` on `unsafe fn`

✓ TIPS & BEST PRACTICES

- Write an Examples section for every public function
- Use `cargo doc --open --no-deps` to preview your crate only
- Hide setup lines in doc tests with a leading `#`

QUICK SUMMARY

- `//` `/* */` — comments
- `///` — item docs
- `//!` — crate/module docs
- doc tests run with `cargo test`

Sheet 009

Variables, Mutability & Shadowing

`let` · `let mut` · shadowing with `let` · block scope · unused `_x` · destructuring `let (a, b)` · must initialize before use

IMMUTABLE BY DEFAULT

Variables declared with `let` are **immutable**: assign once and they cannot change. Add `mut` to allow changes. Declaring a new variable with the same name **shadows** the old one, which lets you transform a value (even to a different type) without inventing new names.

KEY POINTS

- Types are inferred but fixed: a `mut` i32 stays i32
- Shadowing creates a new variable; the old one is hidden
- Variables must be initialized before use (checked at compile time)
- Names are snake_case

LET, MUT AND SHADOWING

```
let x = 5;
// x = 6;                      // error: cannot assign twice to immutable

let mut count = 0;
count += 1;                    // OK: mut

let input = "42";
let input: i32 = input.parse().unwrap();   // shadowing, new type
let input = input * 2;         // 84

let (a, b) = (1, "one");       // destructuring
let [first, .., last] = [10, 20, 30, 40];  // first = 10, last = 40

let y;                         // declare now
if count > 0 { y = "positive"; } else { y = "zero"; }
println!("{y}");               // assigned exactly once on every path

{
    let x = "inner";           // shadows only inside this block
    println!("{x}");           // inner
}
println!("{x}");               // 5

let _unused = 99;              // no warning
```

CODE EXPLAINED

→ Reassigning an immutable variable is a compile error

→ Shadowing lets `input` go from `&str` to `i32`

→ Arrays and tuples destructure in `let`

→ `y` is assigned once on every path, so it needs no `mut`

COMING FROM…

↔ **C++** variables are mutable unless `const`; Rust is the reverse

↔ **Java** `final` ≈ Rust `let`; shadowing in the same scope is not allowed

↔ **Python / JavaScript** reassignment is free; `const` in JS ≈ Rust `let`

THE RUST WAY

⇄ `mut` by habit → immutable `let`, add `mut` only when needed

⇄ `input_str`, `input_num` names → shadow `input`

DECLARATIONS

FORM	MEANING
`let x = 5;`	Immutable, inferred
`let mut x = 5;`	Mutable
`let x: u8 = 5;`	Explicit type
`let x;`	Declare, assign later
`let (a, b) = t;`	Destructure
`let _x = v;`	Unused OK
`let _ = v;`	Ignore (drops at once)

⚠ COMMON MISTAKES

- Adding `mut` everywhere and losing the safety benefit
- Confusing shadowing with mutation
- `let _ = guard;` drops the value immediately (use `_guard`)

✓ TIPS & BEST PRACTICES

- Let the compiler tell you when `mut` is needed
- Use shadowing for parse-and-convert steps
- Keep variables in the smallest scope

QUICK SUMMARY

- `let` — immutable
- `let mut` — mutable
- shadowing — new variable
- initialize before use

Sheet 010

Constants & Statics

const · static · static mut (unsafe) · SCREAMING_CASE · compile-time evaluation · LazyLock / OnceLock

GLOBAL VALUES

`const` defines a compile-time value that is copied into every place it is used. `static` defines a single value with a fixed memory address for the whole program. Mutable statics require `unsafe`; for global state computed at runtime, use `LazyLock` or `OnceLock` from the standard library.

KEY POINTS

- Both need explicit types
- `const` can be used in array lengths and generics
- `static` values must be `Sync` (safe to share across threads)
- `LazyLock` (Rust 1.80) computes on first access

CONST, STATIC, LAZYLOCK

```rust
use std::sync::{LazyLock, Mutex, OnceLock};
use std::collections::HashMap;

const MAX_USERS: u32 = 1_000;
const BUFFER: usize = 4 * 1024;
static APP_NAME: &str = "cheatsheets";

const fn square(x: u32) -> u32 { x * x }
const AREA: u32 = square(12);          // computed at compile time

static CODES: LazyLock<HashMap<&str, u16>> = LazyLock::new(|| {
    HashMap::from([("ok", 200), ("not_found", 404)])
});

static COUNTER: Mutex<u64> = Mutex::new(0);   // safe mutable global

static CONFIG: OnceLock<String> = OnceLock::new();

fn main() {
    let buf = [0u8; BUFFER];              // const as array length
    println!("{} {}", MAX_USERS, AREA);   // 1000 144
    println!("{}", CODES["not_found"]);   // 404
    *COUNTER.lock().unwrap() += 1;
    CONFIG.set("prod".into()).unwrap();   // set once
    println!("{}", CONFIG.get().unwrap());
}
```

CODE EXPLAINED

→ `const fn` can run at compile time

→ `LazyLock` builds the HashMap on first use, once

→ A `Mutex` makes a static safely mutable

→ `OnceLock` is set once at runtime, then read many times

COMING FROM…

↔ **C++** `constexpr` ≈ `const`; globals ≈ `static`

↔ **Java** `static final` fields; lazy holder idiom

↔ **Python** module-level constants (by convention only)

THE RUST WAY

⇄ `static mut` → `Mutex`, atomics, or `OnceLock`

⇄ `lazy_static!` / `once_cell` crates → std `LazyLock` / `OnceLock`

COMPARE

ITEM	CONST	STATIC
Evaluated	Compile time	Compile time (or lazy wrapper)
Address	None (inlined)	One fixed address
Mutable	Never	With Mutex/atomic/unsafe
Use in types	Array length, generics	No
Typical	Limits, sizes	Global state

⚠ COMMON MISTAKES

- Using `static mut` and needing unsafe everywhere
- Expecting `const` to have one address (each use is a copy)
- Heavy runtime work in a constant (not allowed: use LazyLock)

✓ TIPS & BEST PRACTICES

- Prefer `const` for plain values
- Use `LazyLock` for regexes and lookup tables
- Keep global mutable state rare; pass values instead

QUICK SUMMARY

- `const` — inlined value
- `static` — one address
- `const fn` — compile-time code
- `LazyLock` / `OnceLock` — runtime init

Integers

SIZED INTEGER TYPES

Rust integers state their size and sign: i8 to i128 are signed, u8 to u128 unsigned, and isize/usize match the pointer size. The default is i32. Overflow **panics** in debug builds and **wraps** in release builds, so use the explicit checked_, wrapping_ and saturating_ methods when overflow is possible.

KEY POINTS

- `usize` is used for indexes and lengths
- Literal suffixes: `10u8`, `5_i64`
- No implicit conversion between integer types
- Integer division truncates toward zero

UNDER THE HOOD

- Debug builds insert an overflow check after each arithmetic op
- Release builds (`overflow-checks = false` by default) emit plain wrapping instructions
- Integers are plain machine values on the stack; no boxing

INTEGER TYPES AND OVERFLOW

```
let a = 42;                    // i32 (default)
let b: u8 = 255;
let big: i64 = 9_000_000_000;
let hex = 0xFF;                // 255
let oct = 0o17;                // 15
let bin = 0b1010;              // 10
let byte = b'A';               // 65u8
let idx: usize = 3;

println!("{} {}", i32::MAX, u8::MIN);   // 2147483647 0
println!("{}", 7 / 2);                  // 3
println!("{}", -7 / 2);                 // -3 (toward zero)
println!("{}", (-7i32).rem_euclid(2));  // 1

// let c = b + 1;              // debug: panic "attempt to add with overflow"
println!("{:?}", b.checked_add(1));     // None
println!("{}", b.wrapping_add(1));      // 0
println!("{}", b.saturating_add(10));   // 255
println!("{:?}", b.overflowing_add(1)); // (0, true)

println!("{}", u64::from(b) + big as u64);   // explicit widening
println!("{}", i32::from_str_radix("ff", 16).unwrap());   // 255
println!("{}", 255u8.count_ones());     // 8
```

CODE EXPLAINED

- → A literal with no suffix is i32 unless the context says otherwise
- → Overflow panics in debug: catches bugs early
- → `checked_add` returns `Option`, `None` on overflow
- → `From` widens safely; `as` can truncate (sheet 015)

COMING FROM…

- ↔ **C++** `int32_t`, `uint8_t`; signed overflow is undefined behavior
- ↔ **Java** fixed sizes (int, long), no unsigned types, always wraps
- ↔ **Python** ints are arbitrary precision, never overflow

THE RUST WAY

- ⇄ `as` casts to widen → `u64::from(x)` / `.into()`
- ⇄ relying on wraparound → `wrapping_*` to make it explicit

TYPES

TYPE	RANGE
`i8 / u8`	-128..127 / 0..255
`i16 / u16`	±32K / 0..65K
`i32 / u32`	±2.1e9 / 0..4.3e9
`i64 / u64`	±9.2e18 / 0..1.8e19
`i128 / u128`	±1.7e38 / 0..3.4e38
`isize / usize`	Pointer-sized

OVERFLOW METHODS

METHOD	ON OVERFLOW
`checked_*`	None
`wrapping_*`	Wraps around
`saturating_*`	Clamps to min/max
`overflowing_*`	(value, true)
`plain + (debug)`	Panic
`plain + (release)`	Wraps

⚠ COMMON MISTAKES

- Testing only in release mode and missing overflow bugs
- Mixing `i32` and `usize` without conversion (type error)
- Using `as` and silently truncating

✓ TIPS & BEST PRACTICES

- Use `usize` for indexes, `u64` / `i64` for IDs and money in cents
- Turn on `overflow-checks = true` in release for safety-critical code
- Use `TryFrom` when narrowing (sheet 015)

QUICK SUMMARY

- `i32` default, sized types
- `usize` — indexes
- debug panic / release wrap
- `checked_` / `saturating_`

Sheet 012

Floating-Point Numbers

f32 / f64 · f64::NAN / INFINITY · is_nan() · total_cmp · round / floor / ceil / trunc · powi / powf / sqrt · f64::EPSILON

IEEE 754 FLOATS

Rust has `f32` and `f64` (the default). Math functions are **methods** on the value: `x.sqrt()`, `x.powi(2)`, `x.round()`. Floats implement only `PartialEq`/`PartialOrd` because `NaN` is not equal to itself, so they cannot be sorted with plain `sort()` or used as HashMap keys.

KEY POINTS

- `2.0` is f64; write `2.0_f32` for f32
- No implicit int ↔ float conversion
- `total_cmp` gives a full ordering for sorting
- Use integers (cents) or a decimal crate for money

FLOAT METHODS

```
let x = 2.0_f64;
let y: f32 = 1.5;

println!("{}", x.sqrt());              // 1.4142135623730951
println!("{}", x.powi(10));            // 1024
println!("{}", x.powf(0.5));           // 1.414…
println!("{}", 2.567_f64.round());     // 3
println!("{}", (-2.5_f64).round());    // -3 (half away from zero)
println!("{}", 2.7_f64.floor());       // 2
println!("{}", 2.1_f64.ceil());        // 3
println!("{}", (-2.7_f64).trunc());    // -2
println!("{:.2}", 3.14159);            // 3.14
println!("{}", 0.1 + 0.2 == 0.3);      // false
println!("{}", (0.1_f64 + 0.2 - 0.3).abs() < f64::EPSILON);   // true

let nan = f64::NAN;
println!("{}", nan == nan);            // false
println!("{}", nan.is_nan());          // true
println!("{}", 1.0 / 0.0);             // inf

let mut v = vec![2.5_f64, -1.0, 9.0];
// v.sort();                           // error: f64 is not Ord
v.sort_by(|a, b| a.total_cmp(b));      // [-1.0, 2.5, 9.0]
println!("{}", v.iter().cloned().fold(f64::MIN, f64::max));   // 9
```

CODE EXPLAINED

- → Math functions are methods on f64/f32
- → `round` rounds half away from zero
- → Compare floats with a tolerance
- → `total_cmp` sorts floats, placing NaN consistently

COMING FROM…

- ↔ **C++** `std::sqrt(x)`, `std::round(x)`; floats are not totally ordered either
- ↔ **Java** `Math.sqrt(x)`; `Double.compare` for sorting
- ↔ **Python** `math.sqrt(x)`; `round(2.5)` is 2 (banker's)

THE RUST WAY

- ⇄ `partial_cmp(...).unwrap()` in sorts → `total_cmp`
- ⇄ f64 for money → integer cents or `rust_decimal`

METHODS

METHOD	RESULT
`sqrt / cbrt`	Roots
`powi(n) / powf(x)`	Integer / float power
`abs / signum`	Absolute / sign
`round / floor / ceil / trunc`	Rounding
`min / max / clamp`	Bounds
`sin / cos / ln / log10`	Math
`is_nan / is_finite`	Checks

CONSTANTS

CONSTANT	VALUE
`f64::consts::PI`	π
`f64::EPSILON`	~2.2e-16
`f64::MAX / MIN`	Extremes
`f64::INFINITY`	∞
`f64::NAN`	Not a number

⚠ COMMON MISTAKES

- Calling `sqrt(x)` as a free function (it is `x.sqrt()`)
- Sorting or deduping floats with Ord-based methods
- `2.0 * 3` (int literal with float: write `3.0`)

✓ TIPS & BEST PRACTICES

- Prefer f64 unless memory or GPU work needs f32
- Use `ordered-float` crate for float keys
- Use `{:.N}` only for display, not rounding stored values

QUICK SUMMARY

- `f64` default
- methods: `x.sqrt()`, `x.round()`
- NaN ≠ NaN → no `Ord`
- `total_cmp` — sort floats

Booleans & char

TRUE/FALSE AND UNICODE CHARACTERS

`bool` is `true` or `false` and is the only type allowed in conditions. A `char` is a 4-byte **Unicode scalar value** (any code point except surrogates), written in single quotes. Strings are UTF-8, so a String is **not** an array of chars (sheet 021).

KEY POINTS

- No truthy values: `if 1 {}` does not compile
- `b'a'` is a byte (`u8`), not a char
- `char` methods cover Unicode and ASCII classes
- `as u32` gives the code point

BOOL AND CHAR

```
let ok = true;
let ready: bool = 3 > 2;
println!("{}", ok && !ready);          // false
println!("{}", ok as i32);             // 1

let c = 'R';
let heart = '♥';
let crab = '🦀';
println!("{}", std::mem::size_of::<char>());   // 4
println!("{}", c as u32);              // 82
println!("{:?}", char::from_u32(0x1F980));     // Some('🦀')
println!("{}", 'a'.is_alphabetic());   // true
println!("{}", '7'.is_ascii_digit());  // true
println!("{:?}", '7'.to_digit(10));    // Some(7)
println!("{:?}", 'f'.to_digit(16));    // Some(15)
println!("{}", 'ß'.to_uppercase());    // SS (can be several chars)
println!("{}", 'a'.to_ascii_uppercase());     // A
println!("{}", heart.len_utf8());      // 3 bytes in UTF-8
println!("{}", ('a'..='e').collect::<String>());   // abcde
println!("{}", "Rust".chars().count());// 4
```

CODE EXPLAINED

→ `bool as i32` gives 0 or 1

→ A char is always 4 bytes in memory

→ `to_digit(radix)` returns an Option

→ `to_uppercase` returns an iterator because some letters expand

COMING FROM…

↔ **C++** `char` is 1 byte; Unicode needs `char32_t`

↔ **Java** `char` is a UTF-16 unit (2 bytes); emoji need two

↔ **Python** no char type; a 1-length string

THE RUST WAY

⇄ `c as u8 - b'0'` digit math → `c.to_digit(10)`

⇄ indexing strings to get chars → `.chars()`

CHAR METHODS

METHOD	EXAMPLE
`is_alphabetic()`	'é' → true
`is_numeric()`	'৭' → true
`is_ascii_digit()`	'7' → true
`is_whitespace()`	' ' → true
`to_digit(r)`	'f'.to_digit(16) → 15
`to_ascii_uppercase()`	'a' → 'A'
`len_utf8()`	'♥' → 3

LITERALS

LITERAL	TYPE
`'a'`	char
`"a"`	&str
`b'a'`	u8
`b"ab"`	&[u8; 2]
`'\u{1F980}'`	char 🦀

⚠ COMMON MISTAKES

- Using `"a"` where a char `'a'` is expected
- Assuming `to_uppercase()` returns one char
- Treating String length as character count

✓ TIPS & BEST PRACTICES

- Use `is_ascii_*` methods for protocol and ID parsing
- Use `chars().count()` for code point counts
- Use the `unicode-segmentation` crate for user-visible characters

QUICK SUMMARY

- `bool` — only true/false
- `char` — 4-byte Unicode scalar
- `'a'` char vs `"a"` str
- `to_digit` / `from_u32`

Sheet 014

Tuples & the Unit Type

(i32, &str) · .0 / .1 · destructuring · unit () · returning multiple values · never type !

GROUPING VALUES WITHOUT A STRUCT

A **tuple** groups a fixed number of values of possibly different types, accessed by position (`.0`, `.1`) or destructured with `let`. The empty tuple `()` is the **unit** type: the value of expressions that return nothing. The never type `!` marks expressions that never finish, like `panic!` or `loop {}`.

KEY POINTS

- Tuples up to 12 elements implement Debug, PartialEq, etc.
- A one-element tuple needs a comma: `(5,)`
- Functions with no return type return `()`
- `!` coerces to any type

UNDER THE HOOD

- `()` is a zero-sized type: it takes no memory
- Tuple fields may be reordered in memory for alignment (no guaranteed layout)
- `Vec<()>` stores only a length; no allocation for elements

TUPLES, UNIT AND NEVER

```
let point = (3, -4);
let record: (&str, u32, bool) = ("Ada", 36, true);

println!("{}", point.0);            // 3
println!("{:?}", record);           // ("Ada", 36, true)

let (name, age, _) = record;        // destructure, ignore the bool
println!("{name} is {age}");

fn min_max(v: &[i32]) -> (i32, i32) {
    (*v.iter().min().unwrap(), *v.iter().max().unwrap())
}
let (lo, hi) = min_max(&[4, 9, 1]);    // (1, 9)

let single = (5,);                  // one-element tuple
let not_tuple = (5);                // just 5 in parentheses

fn log(msg: &str) { println!("{msg}"); }   // returns ()
let unit: () = log("hi");

fn fail(msg: &str) -> ! { panic!("{msg}") }
let n: i32 = if lo > 0 { lo } else { fail("bad") };   // ! fits as i32

let mut pairs = vec![(2, 'b'), (1, 'a')];
pairs.sort();                       // sorts by .0, then .1
```

CODE EXPLAINED

- → Tuples are accessed by index fields `.0`, `.1`
- → Returning a tuple is the easy way to return two values
- → `()` is what functions without `->` return
- → `fail` returns `!`, so it fits where an i32 is expected

COMING FROM…

- ↔ **C++** `std::tuple` / `std::pair` with structured bindings
- ↔ **Python** tuples and unpacking work the same way
- ↔ **Java** no tuples; use records or `Map.Entry`

THE RUST WAY

- ⇄ tuples with 4+ fields passed around → a named struct
- ⇄ `(T,)` confusion → remember the trailing comma

FORMS

SYNTAX	MEANING
`(a, b)`	Tuple
`t.0`	First field
`let (x, y) = t;`	Destructure
`(5,)`	One-element tuple
`()`	Unit value and type
`!`	Never type

NEVER (!) SOURCES

EXPRESSION	WHY
`panic!()`	Unwinds
`loop {}`	Never ends
`return / break`	Leaves
`std::process::exit()`	Ends process
`todo!() / unreachable!()`	Panics

⚠ COMMON MISTAKES

- `(5)` expecting a tuple
- Long tuples that hide meaning (`t.3`?)
- Comparing tuples of different lengths (type error)

✓ TIPS & BEST PRACTICES

- Use tuples for short-lived pairs; structs for anything named
- Destructure in `for (i, x) in …` loops
- Use `()` as the "nothing" type in generics like `Result<(), E>`

QUICK SUMMARY

- `(a, b)` — tuple
- `.0` / destructure
- `()` — unit
- `!` — never returns

Sheet 015

Type Conversion

as casts (truncation, sign) · From / Into · TryFrom / TryInto · parse::<T>() · to_string() · lossless vs lossy

EXPLICIT CONVERSIONS ONLY

Rust never converts types implicitly. `as` does primitive casts and can **truncate** or change sign silently. The `From`/`Into` traits do lossless conversions, `TryFrom`/`TryInto` do checked ones that return `Result`, `parse` turns strings into values, and `to_string` goes the other way.

KEY POINTS

- Implementing `From<A> for B` also gives `Into<B> for A`
- `as` on floats to ints saturates and maps NaN to 0
- `parse` needs the target type (annotation or turbofish)
- Prefer `From` / `TryFrom` over `as` for integers

CONVERSIONS

```
use std::convert::TryFrom;

let big: i64 = 300;
let a = big as u8;                    // 44 (truncated!)
let b = -1i32 as u32;                 // 4294967295
let c = 3.99_f64 as i32;              // 3
let d = 1e20_f64 as i32;              // 2147483647 (saturates)
let e = 65u8 as char;                 // 'A'

let wide: i64 = i64::from(42i32);     // lossless
let w2: f64 = 7u32.into();            // lossless via Into
let small = u8::try_from(big);        // Err(TryFromIntError(()))
let ok: Result<u8, _> = 200i64.try_into();   // Ok(200)

let n: i32 = "42".parse().unwrap();
let m = "3.5".parse::<f64>()?;        // turbofish
let bad = "x".parse::<i32>();         // Err(ParseIntError)

let s = 42.to_string();               // "42"
let s2 = String::from("hi");          // &str → String
let owned: String = "hi".into();

struct Celsius(f64);
struct Fahrenheit(f64);
impl From<Celsius> for Fahrenheit {
    fn from(c: Celsius) -> Self { Fahrenheit(c.0 * 9.0 / 5.0 + 32.0) }
}
let f: Fahrenheit = Celsius(100.0).into();   // 212.0
```

CODE EXPLAINED

- → `as` keeps the low bits: 300 becomes 44
- → `try_from` reports out-of-range values as errors
- → `parse` returns a Result; `?` passes errors up
- → Implementing `From` gives you `.into()` for free

COMING FROM…

- ↔ **C++** implicit narrowing compiles; `static_cast` is explicit
- ↔ **Java** widening is implicit; `(byte) 300` truncates like `as`
- ↔ **Python** `int("42")`, `str(42)`; ints never truncate

THE RUST WAY

- ⇄ `x as u8` on values that may not fit → `u8::try_from(x)?`
- ⇄ custom `to_fahrenheit()` methods → `impl From<…>`

TOOLS

TOOL	WHEN
`as`	Primitive casts (may lose data)
`From / Into`	Always succeeds
`TryFrom / TryInto`	May fail → Result
`str::parse::<T>()`	Text → value
`to_string()`	Display → String
`AsRef / Borrow`	Cheap reference views

AS RESULTS

CAST	RESULT
`300i64 as u8`	44
`-1i32 as u32`	4294967295
`3.99 as i32`	3
`f64::NAN as i32`	0
`true as u8`	1
`'A' as u32`	65

⚠ COMMON MISTAKES

- Silent truncation with `as` on lengths and IDs
- `"42".parse()` with no target type (cannot infer)
- Implementing `Into` instead of `From`

✓ TIPS & BEST PRACTICES

- Enable clippy's `cast_possible_truncation` lint (pedantic)
- Implement `From` for your error types to use `?` (sheet 044)
- Use `TryFrom` for validated domain types

QUICK SUMMARY

- no implicit conversion
- `as` — may truncate
- `From` / `TryFrom`
- `parse` / `to_string`

Type Inference & Aliases

inference from usage · annotations `let x: u64` · turbofish `::<T>` · `type Alias = …` · `_` placeholder `Vec<_>` · `std::mem::size_of`

LETTING THE COMPILER WORK OUT TYPES

Rust infers types from how values are used, even from later lines in the same function. When it cannot, add an annotation on the variable, a **turbofish** `::<T>` on the call, or a partial type with `_`. Function signatures are never inferred: parameters and return types are always written out.

KEY POINTS

- Inference is local to a function body
- `Vec<_>` lets the element type be inferred
- `type` aliases are just names, not new types
- `size_of::<T>()` shows a type's size in bytes

INFERENCE AND ALIASES

```
use std::collections::HashMap;
use std::mem::size_of;

let mut v = Vec::new();            // Vec<?> for now
v.push(3u16);                      // now Vec<u16>

let nums: Vec<i32> = "1 2 3".split(' ').map(|s| s.parse().unwrap()).collect();
let nums = "1 2 3".split(' ').map(|s| s.parse::<i32>().unwrap())
    .collect::<Vec<_>>();

let total: u64 = nums.iter().map(|&n| n as u64).sum();   // sum needs a type
let words: HashMap<_, _> = [("a", 1), ("b", 2)].into_iter().collect();

type Grid = Vec<Vec<u8>>;
type Result<T> = std::result::Result<T, String>;

fn make_grid(n: usize) -> Grid { vec![vec![0; n]; n] }
fn parse_age(s: &str) -> Result<u8> { s.parse().map_err(|_| s.to_string()) }

println!("{}", size_of::<u8>());          // 1
println!("{}", size_of::<String>());      // 24 (ptr, len, cap)
println!("{}", size_of::<Option<Box<i32>>>());   // 8 (niche)
println!("{}", size_of::<(u8, u32)>());   // 8 (padding)
```

CODE EXPLAINED

→ `v`'s element type is fixed by the later `push`

→ `collect` and `sum` often need the target type

→ Turbofish puts the type on the method call instead

→ The `Result<T>` alias fixes the error type for a module

COMING FROM…

↔ **C++** `auto` infers from the initializer only

↔ **Java** `var` infers local types from the initializer

↔ **Python / JavaScript** dynamic typing; TypeScript infers like Rust

THE RUST WAY

⇄ annotating every `let` → annotate only where inference needs help

⇄ `type UserId = u64` for safety → newtype `struct UserId(u64)` (sheet 106)

WAYS TO SPECIFY

FORM	EXAMPLE
Variable annotation	let x: u64 = 1;
Literal suffix	1u64
Turbofish	parse::<i32>()
Partial	collect::<Vec<_>>()
Function signature	fn f() -> u64
Alias	type Grid = Vec<Vec<u8>>;

SIZES (64-BIT)

TYPE	BYTES
u8 / bool	1
char / u32	4
usize / &T	8
&str / &[T]	16 (fat pointer)
String / Vec<T>	24
()	0

⚠ COMMON MISTAKES

- `collect()` without a target type ("type annotations needed")
- Expecting `type` aliases to prevent mixing values
- Turbofish on the wrong method in a chain

✓ TIPS & BEST PRACTICES

- Put the type on `let` when a long chain ends in `collect`
- Use module-level `type Result<T>` aliases
- Check sizes with `size_of` when optimizing memory

QUICK SUMMARY

- local inference, explicit signatures
- `let x: T` / turbofish `::<T>`
- `Vec<_>` — partial
- `type` — alias only

String vs &str

String (owned, heap) · &str (borrowed slice) · literals are &'static str · &String → &str deref · as_str() · take &str params

TWO STRING TYPES

`String` is an **owned**, growable, heap-allocated UTF-8 buffer. `&str` is a **borrowed view** (a string slice) into UTF-8 data owned by someone else: a String, a literal baked into the binary, or part of a file buffer. Functions that only read text should take `&str`; return or store `String` when you need ownership.

KEY POINTS

- String literals have type `&'static str`
- `&String` coerces to `&str` automatically (deref coercion)
- Convert `&str` → `String` with `to_string()`, `String::from`, `into()`
- Both are always valid UTF-8

UNDER THE HOOD

- `String` = `Vec<u8>`: pointer, capacity, length (24 bytes on the stack)
- `&str` is a **fat pointer**: data pointer + length (16 bytes)
- Literals live in the read-only data section; no allocation
- A slice like `&s[0..3]` points into the same heap buffer

OWNED AND BORROWED TEXT

```
let literal: &'static str = "hello";        // in the binary
let mut owned: String = String::from("hello");
owned.push_str(", world");                  // String can grow

let view: &str = &owned;                    // &String → &str
let part: &str = &owned[0..5];              // "hello" (a slice)
let again: String = part.to_string();       // copy into a new String

fn shout(s: &str) -> String {               // read-only param: &str
    s.to_uppercase()
}
println!("{}", shout(literal));             // HELLO
println!("{}", shout(&owned));              // HELLO, WORLD
println!("{}", shout(owned.as_str()));      // explicit

struct User { name: String }                // store owned data
let u = User { name: "Ada".into() };

fn first_word(s: &str) -> &str {            // return a borrow of the input
    s.split(' ').next().unwrap_or("")
}
println!("{}", first_word("big red dog")); // big
```

CODE EXPLAINED

- → A literal is a `&'static str` pointing into the binary
- → `&owned` gives a `&str` view without copying
- → `shout` accepts literals and Strings because it takes `&str`
- → Structs usually own their text as `String`

COMING FROM…

- ↔ **C++** `std::string` vs `std::string_view` (same split, but views can dangle)
- ↔ **Go** one immutable `string` type; `[]byte` for building
- ↔ **Java / Python** one immutable string type; builders for mutation

THE RUST WAY

- ⇄ `fn f(s: &String)` → `fn f(s: &str)` (accepts more)
- ⇄ `fn f(s: String)` that only reads → `&str` (no clone needed)

COMPARE

ITEM	STRING	&STR
Owns data	Yes	No (borrow)
Growable	Yes	No
Size on stack	24 bytes	16 bytes
Literal type	–	&'static str
Param for reading	Avoid	Prefer
Struct field	Usual	Needs lifetime

CONVERSIONS

FROM → TO	HOW
&str → String	to_string() / into() / to_owned()
String → &str	&s / s.as_str()
&String → &str	Automatic (deref)
char → String	c.to_string()

⚠ COMMON MISTAKES

- Taking `&String` parameters (clippy: `ptr_arg`)
- Returning `&str` to a local String (E0515: returns a reference to local data)
- Cloning Strings just to call a function

✓ TIPS & BEST PRACTICES

- Default to `&str` params and `String` fields
- Use `impl Into<String>` for constructors that store a name
- Use `Cow<str>` when you only sometimes allocate (sheet 076)

QUICK SUMMARY

- `String` — owned, growable
- `&str` — borrowed view
- literal — `&'static str`
- read-only params take `&str`

Building Strings

CREATING AND GROWING STRINGS

Create a `String` from a literal with `String::from`, `.to_string()` or `.into()`, then grow it with `push` (one char) and `push_str` (a `&str`). `+` takes ownership of the left String and appends a `&str`, while `format!` builds a new String from any values without moving them.

KEY POINTS

- `s1 + &s2` moves `s1`; `s2` is only borrowed
- `format!` is the clearest way to join several values
- `with_capacity` avoids reallocations in loops
- Raw strings skip escape processing: `r"C:\dir"`

BUILDING TEXT

```
let mut s = String::new();
s.push('R');                          // one char
s.push_str("ust");                    // a &str
s += " 2024";                         // += takes &str
println!("{s}");                      // Rust 2024

let a = String::from("Hello");
let b = "World".to_string();
let c = a + ", " + &b + "!";          // a is moved here
// println!("{a}");                   // error: borrow of moved value
println!("{c}");                      // Hello, World!

let msg = format!("{}-{}-{}", "x", 42, 3.5);   // nothing moved
let mut out = String::with_capacity(64);
for i in 0..5 { out.push_str(&i.to_string()); }
println!("{out} {}", out.capacity()); // 01234 64

let path = r"C:\Users\ada";         // raw: no escapes
let json = r#"{"name": "Ada"}"#;    // # lets you use quotes
let multi = "line one
line two";                          // newline kept
let joined = "one \
    two";                           // \ at end: "one two"
let bytes: &[u8; 3] = b"abc";       // byte string
let names = ["a", "b", "c"].join(", ");   // "a, b, c"
```

CODE EXPLAINED

→ `push` adds a char; `push_str` adds a slice

→ `+` reuses `a`'s buffer, so `a` cannot be used afterwards

→ `format!` borrows its arguments

→ `r#"…"#` allows double quotes inside without escaping

COMING FROM…

↔ **Java** `StringBuilder.append()`; `+` creates new strings

↔ **Python** `"".join(parts)` and f-strings; strings are immutable

↔ **Go** `strings.Builder` and `fmt.Sprintf`

THE RUST WAY

⇄ `s = s + &x` in a loop → `s.push_str(&x)` or `write!(s, …)`

⇄ chains of `+` with conversions → one `format!`

CREATE

CODE	RESULT
`String::new()`	Empty
`String::from("x")`	From literal
`"x".to_string()`	Via Display
`"x".to_owned()`	Clone the slice
`"x".into()`	Needs target type
`format!(…)`	From any values
`String::with_capacity(n)`	Pre-sized

LITERALS

LITERAL	MEANING
`"a\tb\n"`	Escapes
`r"a\b"`	Raw
`r#"say "hi""#`	Raw with quotes
`b"bytes"`	&[u8; N]
`c"text"`	&CStr (1.77+)
`"\u{1F980}"`	Unicode escape

⚠ COMMON MISTAKES

- Using `a` after `a + &b` (moved)
- `a + b` where b is a String (needs `&b`)
- Calling `to_string()` on every literal inside a loop

✓ TIPS & BEST PRACTICES

- Use `write!` on a String with `use std::fmt::Write`
- Use `concat()` / `join()` for slices of strings
- Pre-size with `with_capacity` when the size is known

QUICK SUMMARY

- `push` / `push_str`
- `a + &b` moves `a`
- `format!` — borrows all
- `r#"…"#` raw strings

Sheet 019

String Methods — Search & Inspect

`len()` (bytes) · `is_empty()` · `contains` · `starts_with` / `ends_with` · `find` / `rfind` · `matches` · `strip_prefix` / `strip_suffix`

INSPECTING TEXT

Search methods live on `str`, so they work on both `String` and `&str`. They accept a **pattern**: a `&str`, a `char`, a slice of chars or a closure. `find` returns `Option<usize>` with a **byte** index, and `len()` counts bytes, not characters.

KEY POINTS

- Patterns: `"ab"`, `'a'`, `['a', 'b']`, `char::is_numeric`
- `strip_prefix` returns `Option<&str>`
- Byte indexes from `find` are safe to slice with
- Use the `regex` crate for regular expressions

SEARCH METHODS

```
let s = "Hello, Rust world";

println!("{}", s.len());                    // 17 (bytes)
println!("{}", "".is_empty());              // true
println!("{}", s.contains("Rust"));         // true
println!("{}", s.contains(char::is_numeric));   // false
println!("{}", s.starts_with("Hell"));      // true
println!("{}", s.ends_with('d'));           // true
println!("{:?}", s.find('o'));              // Some(4)
println!("{:?}", s.rfind('o'));             // Some(13)
println!("{:?}", s.find("xyz"));            // None
println!("{:?}", s.find([',', ' ']));       // Some(5)
println!("{}", s.matches('o').count());     // 2
println!("{:?}", s.match_indices("l").map(|(i, _)| i).collect::<Vec<_>>());
                                            // [2, 3, 15]

if let Some(i) = s.find(',') {
    println!("{}", &s[..i]);                // Hello
}
println!("{:?}", "v1.2".strip_prefix('v'));  // Some("1.2")
println!("{:?}", "v1.2".strip_prefix('x'));  // None
println!("{:?}", "a.txt".strip_suffix(".txt"));   // Some("a")
println!("{}", "café".len());               // 5 (é is 2 bytes)
println!("{}", "Abc".eq_ignore_ascii_case("aBC"));   // true
```

CODE EXPLAINED

→ `len()` counts UTF-8 bytes

→ Patterns can be chars, strings, arrays of chars or functions

→ `find` gives a byte index you can slice with

→ `strip_prefix` both checks and removes

COMING FROM…

↔ **Python** `in`, `startswith`, `find` (returns -1, not None)

↔ **Java** `contains`, `indexOf` (returns -1)

↔ **Go** `strings.Contains`, `strings.Index`, `strings.TrimPrefix`

THE RUST WAY

⇄ `if s.starts_with(p) { &s[p.len()..] }` → `s.strip_prefix(p)`

⇄ `s.len() == 0` → `s.is_empty()`

METHODS

METHOD	RETURNS
`len()`	usize bytes
`is_empty()`	bool
`contains(p)`	bool
`starts_with / ends_with`	bool
`find / rfind`	Option<usize>
`matches(p)`	Iterator of &str
`match_indices(p)`	Iterator (usize, &str)
`strip_prefix / strip_suffix`	Option<&str>

⚠ COMMON MISTAKES

- Treating `len()` as a character count
- Expecting -1 from `find` (it returns `None`)
- Building a byte index by counting chars and slicing (can panic)

✓ TIPS & BEST PRACTICES

- Use `chars().count()` for a character count
- Use `eq_ignore_ascii_case` for ASCII comparisons
- Reach for `regex` only when simple patterns are not enough

QUICK SUMMARY

- `len()` — bytes
- `find` → `Option<usize>`
- patterns: &str, char, fn
- `strip_prefix` → Option

Sheet 020

String Methods — Transform & Split

to_uppercase / to_lowercase · trim family · replace / replacen · split / split_whitespace / lines · split_once · repeat · join / concat

CHANGING AND SPLITTING TEXT

Transforming methods like `to_uppercase` and `replace` return a **new String**. Trimming and splitting return **slices** (`&str`) that borrow from the original, so they cost nothing until you collect or convert them. `split_once` is the handy way to split `key=value` pairs.

KEY POINTS

- `trim` returns `&str` (no allocation)
- `split` returns a lazy iterator
- `split_whitespace` handles runs of spaces and tabs
- `lines` handles both `\n` and `\r\n`

TRANSFORM AND SPLIT

```
let s = "  Hello, Rust!  ";
println!("[{}]", s.trim());                  // [Hello, Rust!]
println!("[{}]", s.trim_start());            // [Hello, Rust!  ]
println!("{}", "xxabcxx".trim_matches('x'));  // abc
println!("{}", "Rust".to_uppercase());       // RUST
println!("{}", "a-b-c".replace('-', "+"));   // a+b+c
println!("{}", "a-b-c".replacen('-', "", 1)); // ab-c
println!("{}", "ab".repeat(3));              // ababab

let parts: Vec<&str> = "a,b,,c".split(',').collect();   // ["a", "b", "", "c"]
let words: Vec<&str> = " one  two\tthree ".split_whitespace().collect();
                                             // ["one", "two", "three"]
let n = "k=v=w".splitn(2, '=').collect::<Vec<_>>();     // ["k", "v=w"]
println!("{:?}", "user=ada".split_once('='));   // Some(("user", "ada"))
println!("{:?}", "a/b/c".rsplit_once('/'));     // Some(("a/b", "c"))

for line in "one\ntwo\r\nthree".lines() { print!("{line} "); } // one two three

let csv = ["x", "y", "z"].join(",");          // "x,y,z"
let glued = ["ab", "cd"].concat();            // "abcd"
let title: String = "rust".chars().enumerate()
    .map(|(i, c)| if i == 0 { c.to_ascii_uppercase() } else { c })
    .collect();                               // "Rust"
```

CODE EXPLAINED

- → `trim` variants return slices of the original
- → `replacen` limits the number of replacements
- → `split_once` returns both halves as an Option
- → Mapping over chars and collecting builds a new String

COMING FROM…

- ↔ **Python** `strip`, `split`, `partition`, `"sep".join(list)`
- ↔ **Java** `strip()`, `split(regex)`, `String.join`
- ↔ **Go** `strings.TrimSpace`, `strings.Fields`, `strings.Cut`

THE RUST WAY

- ⇄ `s.split("=").collect::<Vec<_>>()[1]` → `s.split_once('=')`
- ⇄ `s.split(" ")` on user input → `split_whitespace()`

TRANSFORM

METHOD	RETURNS
`to_uppercase / to_lowercase`	String
`trim / trim_start / trim_end`	&str
`trim_matches(p)`	&str
`replace / replacen`	String
`repeat(n)`	String

SPLIT

METHOD	RETURNS
`split(p)`	Iterator<&str>
`splitn(n, p)`	At most n parts
`split_whitespace()`	Iterator<&str>
`lines()`	Iterator<&str>
`split_once(p)`	Option<(&str, &str)>
`split_terminator(p)`	No trailing empty

⚠ COMMON MISTAKES

- Expecting `split(' ')` to collapse repeated spaces
- Forgetting that `to_uppercase` allocates a new String
- `[1]` on a split Vec that may have one element (panics)

✓ TIPS & BEST PRACTICES

- Keep data as `&str` slices until you need to own it
- Use `split_once` for headers, env vars and config lines
- Use `str::parse` right after splitting numbers

QUICK SUMMARY

- case / replace → new String
- trim / split → slices
- `split_once` — key=value
- `join` / `concat`

Sheet 021

UTF-8, chars & bytes

UTF-8 guarantee · no `s[0]` indexing · byte-range slices & panics · `chars()` / `char_indices()` · `bytes()` · `is_char_boundary` · `String::from_utf8_lossy`

WHY YOU CANNOT INDEX A STRING

Rust strings are always **valid UTF-8**, where a character takes 1 to 4 bytes. Indexing with `s[0]` is not allowed because a byte position may fall in the middle of a character. Instead, iterate with `chars()` (Unicode scalars) or `bytes()` (raw bytes), and slice only on character boundaries.

KEY POINTS

- `&s[0..1]` panics if byte 1 is not a char boundary
- `chars().nth(i)` is O(n), not O(1)
- Invalid UTF-8 from files → `from_utf8` returns an error
- User-visible "characters" (graphemes) may be several chars

UNDER THE HOOD

- A `String` is a `Vec<u8>` with a UTF-8 invariant checked at creation
- `chars()` decodes UTF-8 on the fly; it does not store chars
- Slicing checks `is_char_boundary` at runtime and panics on failure
- `from_utf8_unchecked` (unsafe) skips validation

CHARS, BYTES AND SLICING

```rust
let s = "héllo🦀";
println!("{}", s.len());                    // 10 bytes
println!("{}", s.chars().count());          // 6 chars
// let c = s[0];                    // error: cannot index str by integer

println!("{:?}", s.chars().next());         // Some('h')
println!("{:?}", s.chars().nth(1));         // Some('é')
println!("{:?}", s.chars().rev().next());   // Some('🦀')
println!("{}", &s[0..1]);                   // h
// println!("{}", &s[0..2]);                // panic: byte 2 is inside 'é'
println!("{}", s.is_char_boundary(2));      // false

for (i, c) in s.char_indices() {
    print!("{i}:{c} ");                     // 0:h 1:é 3:l 4:l 5:o 6:🦀
}
println!();
println!("{:?}", "hé".bytes().collect::<Vec<u8>>());   // [104, 195, 169]
println!("{:?}", "hé".as_bytes());          // same, as &[u8]

let raw = vec![72, 105, 0xFF];
println!("{:?}", String::from_utf8(raw.clone()));   // Err(FromUtf8Error …)
println!("{}", String::from_utf8_lossy(&raw));      // Hi\u{FFFD}

let first3: String = s.chars().take(3).collect();   // "hél"
let ascii_upper = "hello".to_ascii_uppercase();     // fast for ASCII
```

CODE EXPLAINED

- → `len()` is 10 bytes for 6 characters
- → `char_indices` gives byte offsets you can slice at
- → Slicing in the middle of a character panics
- → `from_utf8_lossy` replaces bad bytes with \u{FFFD}

COMING FROM…

- ↔ **C++** `std::string` is bytes with no encoding guarantee
- ↔ **Python** `str` indexes by code point; `bytes` is separate
- ↔ **Java / JS** strings are UTF-16; `charAt` can split emoji

THE RUST WAY

- ⇄ `s.chars().nth(i)` in a loop (O(n²)) → iterate once with `chars()`
- ⇄ `&s[..n]` with a char count → `s.chars().take(n).collect()`

VIEWS OF A STRING

METHOD	YIELDS
`bytes()` / `as_bytes()`	u8
`chars()`	char (scalar)
`char_indices()`	(byte_idx, char)
`&s[a..b]`	&str (byte range)
`graphemes (crate)`	User characters

UTF-8 SIZES

CHARACTER	BYTES
ASCII (a)	1
Latin (é)	2
CJK (中)	3
Emoji (🦀)	4

⚠ COMMON MISTAKES

- Slicing with indices computed from char counts
- Assuming one char per visible character (é can be e + ◌́)
- Using `chars().nth` for random access

✓ TIPS & BEST PRACTICES

- Work in bytes for ASCII-only protocols
- Use `unicode-segmentation` for graphemes
- Validate external bytes with `from_utf8`

QUICK SUMMARY

- always UTF-8
- no `s[i]` indexing
- `chars()` / `bytes()`
- slice on char boundaries

Sheet 022

Arithmetic & Overflow

`+ - * / %` · integer division truncates · `rem_euclid` · `pow` · `checked_add` / `wrapping_add` / `saturating_add` / `overflowing_add` · no `++`

ARITHMETIC OPERATORS

Arithmetic needs both operands to be the **same type**: `i32 + i64` does not compile. Integer division truncates toward zero and `%` keeps the sign of the left operand. Overflow panics in debug builds, so code that might overflow should use the explicit checked, wrapping or saturating methods.

KEY POINTS

- There is no `++` or `--`: use `+= 1`
- Division by zero panics for integers; floats give inf/NaN
- `pow(u32)` for integers, `powi` / `powf` for floats
- `rem_euclid` always returns a non-negative result

OPERATORS AND OVERFLOW

```
let a = 17;
let b = 5;
println!("{} {} {}", a + b, a - b, a * b);    // 22 12 85
println!("{} {}", a / b, a % b);              // 3 2
println!("{} {}", -17 / 5, -17 % 5);          // -3 -2
println!("{}", (-17i32).rem_euclid(5));       // 3
println!("{}", (-17i32).div_euclid(5));       // -4
println!("{}", 17.0 / 5.0);                   // 3.4
println!("{}", 2i32.pow(10));                 // 1024
println!("{}", 10_i64.pow(12));               // 1000000000000

let x: i32 = 10;
let y: i64 = 20;
// let z = x + y;                             // error: mismatched types
let z = i64::from(x) + y;                     // 30

let mut n = 0;
n += 1;                                       // no n++

let big = u8::MAX;
println!("{:?}", big.checked_add(1));         // None
println!("{}", big.wrapping_add(1));          // 0
println!("{}", big.saturating_add(1));        // 255
println!("{:?}", big.overflowing_add(1));     // (0, true)
println!("{:?}", 10u32.checked_sub(20));      // None
println!("{:?}", 7i32.checked_div(0));        // None (no panic)
println!("{}", 7u32.abs_diff(10));            // 3
println!("{}", (-5i32).abs());                // 5
```

CODE EXPLAINED

- → `/` and `%` truncate toward zero
- → `rem_euclid` / `div_euclid` floor-style math for negatives
- → Mixed types need an explicit conversion
- → `checked_div` returns None instead of panicking on zero

COMING FROM…

- ↔ **C++** mixed types promote silently; signed overflow is UB
- ↔ **Java** silent wraparound; `Math.addExact` throws on overflow
- ↔ **Python** `-17 // 5` is -4 (floors); ints never overflow

THE RUST WAY

- ⇄ `i += 1` where `i` may hit MAX → `i = i.checked_add(1)?`
- ⇄ `(a % n + n) % n` → `a.rem_euclid(n)`

OPERATORS

OP	NOTES
`+ - *`	Same types only
`/`	Truncates (ints)
`%`	Sign of left operand
`-x`	Neg (signed only)
`pow(n)`	Integer power
`abs()`	Signed only

OVERFLOW CHOICE

METHOD	USE WHEN
`checked_*`	Must detect overflow
`wrapping_*`	Hashes, checksums
`saturating_*`	Counters, audio, UI
`overflowing_*`	Need the flag too
`strict_* (1.91)`	Always panic

⚠ COMMON MISTAKES

- Assuming release builds catch overflow
- `u32` subtraction going below zero (panic or wrap)
- Python-style floor division expectations for negatives

✓ TIPS & BEST PRACTICES

- Use `abs_diff` for unsigned distances
- Use `i64` for intermediate sums of `i32`
- Enable `overflow-checks` in release where correctness matters

QUICK SUMMARY

- same types only
- `/` `%` truncate
- no `++`
- checked / wrapping / saturating

Sheet 023

Comparison & Ordering

== != < > <= >= · PartialEq / Eq · PartialOrd / Ord · Ordering · cmp / partial_cmp · max / min / clamp

EQUALITY AND ORDERING TRAITS

Comparison operators are backed by traits: == uses **PartialEq**, < uses **PartialOrd**. `Eq` and `Ord` add the guarantee of a total order, which sorting, `BTreeMap` and `max()` on iterators require. Floats are only partial because of NaN. Derive these traits for your own types.

KEY POINTS

- Both sides must be the same type (or have an impl)
- `cmp` returns `Ordering::Less | Equal | Greater`
- Derived ordering compares fields top to bottom
- Strings compare by bytes (lexicographic)

COMPARISONS

```
use std::cmp::Ordering;

println!("{}", 3 == 3);                     // true
println!("{}", "apple" < "banana");         // true
println!("{}", "Z" < "a");                  // true (byte order)
println!("{:?}", 5_i32.cmp(&9));            // Less
println!("{:?}", 2.5.partial_cmp(&f64::NAN));   // None

match 7_i32.cmp(&5) {
    Ordering::Less => println!("smaller"),
    Ordering::Equal => println!("same"),
    Ordering::Greater => println!("bigger"),  // bigger
}

println!("{}", 3_i32.max(8));               // 8
println!("{}", 15_i32.clamp(0, 10));        // 10
println!("{}", 2.0_f64.max(f64::NAN));      // 2

#[derive(Debug, PartialEq, Eq, PartialOrd, Ord)]
struct Version { major: u32, minor: u32 }
let v1 = Version { major: 1, minor: 9 };
let v2 = Version { major: 2, minor: 0 };
println!("{}", v1 < v2);                    // true

let mut people = vec![("Ada", 36), ("Bob", 25)];
people.sort_by(|a, b| a.1.cmp(&b.1).then(a.0.cmp(&b.0)));
println!("{}", (1, "b") < (1, "c"));         // true (tuples compare in order)
```

CODE EXPLAINED

→ `cmp` gives an `Ordering` you can `match`

→ NaN makes `partial_cmp` return None

→ Derived `Ord` compares `major`, then `minor`

→ `then` chains a tie-breaker

COMING FROM…

↔ **C++** `operator<=>` (C++20) returns an ordering like `cmp`

↔ **Java** `Comparable.compareTo` returns an int

↔ **Python** `__eq__` / `__lt__`; tuples compare element by element

THE RUST WAY

⇄ `if a > b { a } else { b }` → `a.max(b)`

⇄ manual `PartialOrd` impl for field order → `#[derive]`

TRAITS

TRAIT	GIVES
`PartialEq`	`= !=`
`Eq`	Reflexive equality (HashMap key)
`PartialOrd`	< > <= >= partial_cmp
`Ord`	cmp, max, min, sort
`Hash`	HashMap/HashSet key

ORDERING HELPERS

METHOD	DOES
`a.cmp(&b)`	Ordering
`o.reverse()`	Flip
`o.then(o2)`	Tie-break
`o.then_with(|| …)`	Lazy tie-break
`cmp::Reverse(x)`	Reverse sort key
`x.clamp(lo, hi)`	Bound

⚠ COMMON MISTAKES

- Comparing `String` with `&str` the wrong way round (works both ways for `==`)
- Deriving `Ord` with fields in the wrong order
- Calling `.max()` on an iterator of floats

✓ TIPS & BEST PRACTICES

- Put fields in comparison order when deriving `Ord`
- Use `sort_by_key` with tuples for multi-key sorts
- Use `total_cmp` for floats

QUICK SUMMARY

- operators → traits
- `Ord` — total order
- `cmp` → `Ordering`
- `max` / `min` / `clamp`

Logical & Bitwise

BOOLEAN LOGIC AND BIT OPERATIONS

&& and || short-circuit and work only on `bool`. On integers, &, |, ^ and ! are bitwise (note ! is bitwise NOT, there is no ~). Shifts << and >> are arithmetic for signed types and logical for unsigned. Integer types also have methods for counting and rotating bits.

KEY POINTS

- `!x` on an integer flips all bits (C uses `~`)
- Shifting by ≥ bit width panics in debug
- `&` / `|` on bools evaluate both sides
- Use `u32` / `u64` for bit flags; the `bitflags` crate helps

LOGIC AND BITS

```rust
let a = true;
let b = false;
println!("{} {} {}", a && b, a || b, !a);   // false true false
println!("{}", a ^ b);                       // true (bool XOR)

fn check() -> bool { println!("called"); true }
let _ = false && check();                   // check() not called
let _ = false & check();                    // check() IS called

let x: u8 = 0b1100_1010;
let y: u8 = 0b1010_0110;
println!("{:08b}", x & y);                  // 10000010
println!("{:08b}", x | y);                  // 11101110
println!("{:08b}", x ^ y);                  // 01101100
println!("{:08b}", !x);                     // 00110101
println!("{:08b}", x << 2);                 // 00101000
println!("{:08b}", x >> 3);                 // 00011001
println!("{}", -16i32 >> 2);                // -4 (arithmetic shift)

println!("{}", x.count_ones());             // 4
println!("{}", 1u32.leading_zeros());       // 31
println!("{}", 8u32.trailing_zeros());      // 3
println!("{:08b}", x.rotate_left(4));       // 10101100
println!("{}", 64u32.is_power_of_two());    // true

const READ: u8 = 1 << 0;
const WRITE: u8 = 1 << 1;
let mut perms = READ;
perms |= WRITE;                             // set
perms &= !READ;                             // clear
println!("{}", perms & WRITE != 0);         // true
```

CODE EXPLAINED

→ `&&` skips the right side when the left is false

→ Bool `&` evaluates both sides

→ `!` on `u8` is bitwise NOT

→ `|=` sets and `&= !flag` clears a bit flag

COMING FROM…

↔ **C / C++** same operators, but `~` for NOT; `!` is logical only

↔ **Java** `>>>` for logical shift; Rust picks by signedness

↔ **Python** `~x` NOT; ints are unbounded

THE RUST WAY

⇄ `~x` → `!x`

⇄ hand-rolled flag constants everywhere → `bitflags!` crate

OPERATORS

OP	MEANING
`&& ||`	Logical, short-circuit
`!`	NOT (logical or bitwise)
`& | ^`	AND OR XOR
`<< >>`	Shift
`&= |= ^= <<= >>=`	Compound

BIT METHODS

METHOD	DOES
`count_ones / count_zeros`	Popcount
`leading_zeros / trailing_zeros`	Zero runs
`rotate_left / rotate_right`	Rotate
`swap_bytes / to_be`	Endianness
`is_power_of_two`	Check
`next_power_of_two`	Round up

⚠ COMMON MISTAKES

- Writing `~x` (not Rust syntax)
- Expecting `>>` on `i32` to shift in zeros (cast to `u32` first)
- Shifting by a value ≥ the bit width

✓ TIPS & BEST PRACTICES

- Parenthesize bit tests `(x & 1) == 0` for readers coming from C
- Use `{:08b}` / `{:#x}` to print bit patterns
- Use `checked_shl` when the shift amount is untrusted

QUICK SUMMARY

- `&& ||` — short-circuit
- `& | ^ !` — bits
- `!` not `~`
- `count_ones` / `rotate_left`

Sheet 025

Assignment & Ranges

= · compound += -= *= /= %= &= |= ^= <<= >>= · a..b / a..=b / .. / a.. · contains · step_by / rev

ASSIGNMENT AND RANGE EXPRESSIONS

Assignment = is an expression of type `()`, so `a = b = c` does not chain. Compound operators modify in place. **Ranges** are values: `a..b` excludes the end, `a..=b` includes it, and open forms `a..`, `..b` and `..` are used for slicing. Ranges are iterators, so they work in `for` loops and adapter chains.

KEY POINTS

- Assignment returns `()` (no `if (x = 5)` bugs)
- Destructuring assignment: `(a, b) = (b, a)`
- `step_by(n)` skips; `rev()` counts down
- Ranges in patterns: `1..=9`

ASSIGNMENT AND RANGES

```
let mut x = 10;
x += 5;   x -= 3;   x *= 2;   x /= 4;   x %= 4;
println!("{x}");                          // 2
let mut bits = 0b0001u8;
bits <<= 3;                               // 0b1000

let (mut a, mut b) = (1, 2);
(a, b) = (b, a);                          // swap via destructuring
println!("{a} {b}");                      // 2 1

for i in 0..3 { print!("{i} "); }         // 0 1 2
for i in 1..=3 { print!("{i} "); }        // 1 2 3
for i in (0..10).step_by(3) { print!("{i} "); }   // 0 3 6 9
for i in (1..=3).rev() { print!("{i} "); }  // 3 2 1
println!();

let r = 1..=10;
println!("{}", r.contains(&5));           // true
println!("{}", (1..5).len());             // 4
println!("{}", (1..=100).sum::<i32>());   // 5050
let evens: Vec<_> = (0..10).filter(|n| n % 2 == 0).collect();

let v = [10, 20, 30, 40, 50];
println!("{:?}", &v[1..3]);               // [20, 30]
println!("{:?}", &v[..2]);                // [10, 20]
println!("{:?}", &v[3..]);                // [40, 50]
println!("{:?}", &v[..]);                 // all
let letters: String = ('a'..='e').collect(); // "abcde"
```

CODE EXPLAINED

→ Compound operators update `x` in place

→ Tuples allow swapping without a temp variable

→ `step_by` and `rev` adapt the range iterator

→ Open ranges slice from the start or to the end

COMING FROM…

↔ **Python** `range(0, 10, 3)`; `v[1:3]`, `v[3:]`

↔ **Go** `v[1:3]` slices; `for i := range 10` (Go 1.22)

↔ **Kotlin / Swift** `1..3` is inclusive; Rust uses `..=`

THE RUST WAY

⇄ `for i in 0..v.len() { v[i] }` → `for x in &v`

⇄ `(0..n).rev()` instead of manual countdown with `while`

RANGE FORMS

SYNTAX	TYPE	INCLUDES
`a..b`	`Range`	a up to b-1
`a..=b`	`RangeInclusive`	a through b
`a..`	`RangeFrom`	a onward
`..b`	`RangeTo`	up to b-1
`..=b`	`RangeToInclusive`	up to b
`..`	`RangeFull`	everything

⚠ COMMON MISTAKES

- `1..10` expecting 10 to be included
- `a = b = 0` (assigns `()` to a: type error)
- `(0..10).step_by(0)` panics

✓ TIPS & BEST PRACTICES

- Use `..=` for "through" ranges like dice rolls
- Pass ranges to `drain`, `replace_range` and slices
- Ranges of chars work: `'a'..='z'`

QUICK SUMMARY

- `=` returns `()`
- compound ops in place
- `..` exclusive · `..=` inclusive
- `step_by` / `rev`

Sheet 026

Operator Overloading

std::ops::Add / Sub / Mul · Neg / Not · AddAssign · Index / IndexMut · type Output · derived comparisons

OPERATORS ARE TRAITS

Every operator maps to a trait in `std::ops`. Implement `Add` and `a + b` works for your type; `Output` names the result type. Comparison operators come from `PartialEq`/`PartialOrd`, usually derived. You cannot create new operators or overload &&, || or =.

KEY POINTS

- `Add<Rhs = Self>` can take a different right-hand type
- Implement for references too to avoid moving operands
- `Index` must return a reference
- Keep the meaning unsurprising (+ should add)

IMPLEMENTING OPERATORS

```
use std::ops::{Add, AddAssign, Index, Mul, Neg};

#[derive(Debug, Clone, Copy, PartialEq)]
struct V2 { x: f64, y: f64 }

impl Add for V2 {
    type Output = V2;
    fn add(self, o: V2) -> V2 { V2 { x: self.x + o.x, y: self.y + o.y } }
}
impl Mul<f64> for V2 {                      // V2 * f64
    type Output = V2;
    fn mul(self, k: f64) -> V2 { V2 { x: self.x * k, y: self.y * k } }
}
impl Neg for V2 {
    type Output = V2;
    fn neg(self) -> V2 { V2 { x: -self.x, y: -self.y } }
}
impl AddAssign for V2 {
    fn add_assign(&mut self, o: V2) { self.x += o.x; self.y += o.y; }
}

struct Poly(Vec<i32>);
impl Index<usize> for Poly {
    type Output = i32;
    fn index(&self, i: usize) -> &i32 { &self.0[i] }
}

let a = V2 { x: 1.0, y: 2.0 };
let mut b = a + a * 3.0;                    // V2 { x: 4.0, y: 8.0 }
b += -a;                                    // V2 { x: 3.0, y: 6.0 }
println!("{:?} {}", b, a == a);
println!("{}", Poly(vec![5, 7])[1]);        // 7
```

CODE EXPLAINED

- → `type Output` sets the result type of `+`
- → `Mul<f64>` lets a vector be multiplied by a scalar
- → Deriving `Copy` means operands are not moved
- → `Index` returns `&i32`; `p[1]` dereferences it

COMING FROM…

- ↔ **C++** `operator+` member or free functions
- ↔ **Python** `__add__`, `__mul__`, `__getitem__`
- ↔ **Java** no operator overloading

THE RUST WAY

- ⇄ `a.add(b)` methods for math types → `impl Add`
- ⇄ overloading `+` for non-additive meaning → a named method

OPERATOR TRAITS

OPERATOR	TRAIT
`a + b`	Add
`a - b`	Sub
`a * b`	Mul
`a / b / a % b`	Div / Rem
`-a / !a`	Neg / Not
`a += b`	AddAssign
`a[i]`	Index / IndexMut
`a & b | ^ << >>`	BitAnd … Shr
`*a`	Deref

⚠ COMMON MISTAKES

- Implementing only for owned values, so `a + b` moves non-Copy operands
- Forgetting `type Output`
- Returning a value from `Index` (must be a reference)

✓ TIPS & BEST PRACTICES

- Derive `Copy` for small math types
- Also implement `impl Add<&V2> for &V2` for heavy types
- Use the `derive_more` crate to cut boilerplate

QUICK SUMMARY

- operators = `std::ops` traits
- `type Output`
- `AddAssign` for `+=`
- `Index` returns `&T`

if / else as Expressions

CONDITIONS THAT PRODUCE VALUES

`if` needs a real `bool` and no parentheses. Because `if` is an expression, it replaces the ternary operator: `let x = if a { 1 } else { 2 };`. When used for a value, every branch must produce the same type and an `else` is required.

KEY POINTS

- Braces are always required; parentheses are not used
- No truthiness: `if n {}` fails for integers
- An `if` without `else` has type `()`
- Long `else if` chains often read better as `match`

IF EXPRESSIONS

```
let temp = 72;

if temp > 85 {
    println!("hot");
} else if temp > 60 {
    println!("nice");                       // nice
} else {
    println!("cold");
}

let label = if temp >= 70 { "warm" } else { "cool" };   // no ternary
let fee = if temp > 100 { 25 } else { 0 };

// let bad = if temp > 70 { "warm" } else { 5 };   // error: incompatible types
// if temp { }                          // error: expected bool, found integer

let items: Vec<i32> = Vec::new();
if !items.is_empty() {
    println!("{} items", items.len());
}

let grade = {
    let score = 88;
    if score >= 90 { 'A' }
    else if score >= 80 { 'B' }
    else { 'C' }
};
println!("{label} {fee} {grade}");          // warm 0 B

let name: Option<&str> = Some("Ada");
if let Some(n) = name { println!("hi {n}"); }   // see sheet 033
```

CODE EXPLAINED

→ `if` arms are blocks; the last expression is the value

→ `label` gets the value of whichever branch runs

→ Mismatched branch types are a compile error

→ Integers are not truthy; compare explicitly

COMING FROM…

↔ **C++ / Java** `cond ? a : b` ternary

↔ **Python** `a if cond else b`

↔ **Go** no ternary at all; Rust's `if` expression covers it

THE RUST WAY

⇄ `let x; if c { x = 1 } else { x = 2 }` → `let x = if c { 1 } else { 2 };`

⇄ deep `else if` on one value → `match`

FORMS

FORM	TYPE
`if c { … }`	()
`if c { a } else { b }`	Type of a and b
`else if`	Chained
`if let P = e`	Pattern test
`if let … && cond (2024)`	let chain

⚠ COMMON MISTAKES

- Adding `;` after the branch values (branches become `()`)
- Missing `else` when the if is used as a value
- Writing `if (x > 5)` (works, but rustc warns about unnecessary parens)

✓ TIPS & BEST PRACTICES

- Use `if` expressions to keep variables immutable
- Use `bool::then` / `then_some` for `Option`: `cond.then_some(x)`
- Switch to `match` once there are three or more cases on one value

QUICK SUMMARY

- condition is `bool`
- `if` is an expression
- branches same type
- no ternary needed

Sheet 028

loop & Break Values

loop · break / continue · break value · labels 'outer: · break 'outer · infinite loops & !

INFINITE LOOPS AND LABELED BREAKS

`loop` repeats forever until `break`. Unlike other loops, `loop` is an expression that can **return a value** with `break value`. Labels like `'outer:` let `break` and `continue` target an enclosing loop. A `loop` with no `break` has type `!`.

KEY POINTS

- Use `loop` instead of `while true` (clearer and helps the compiler)
- Labels start with an apostrophe: `'outer`
- Labeled blocks can also `break 'label value`
- Server main loops are often `loop {}`

LOOP, BREAK AND LABELS

```
let mut count = 0;
let result = loop {
    count += 1;
    if count == 10 {
        break count * 2;                    // value of the loop
    }
};
println!("{result}");                       // 20

let mut attempts = 0;
let data = loop {
    attempts += 1;
    match fetch(attempts) {
        Some(d) => break d,
        None if attempts < 3 => continue,
        None => break String::from("default"),
    }
};

'outer: for x in 0..5 {
    for y in 0..5 {
        if y > x { continue 'outer; }       // next x
        if x + y == 6 { break 'outer; }     // leave both loops
        print!("({x},{y}) ");
    }
}
println!();

let found = 'search: {                      // labeled block
    for n in [3, 8, 12] {
        if n % 4 == 0 { break 'search Some(n); }
    }
    None
};
println!("{found:?}");                      // Some(8)

fn serve() -> ! { loop { /* accept connections */ } }
fn fetch(n: u32) -> Option<String> { (n == 2).then(|| "ok".into()) }
```

CODE EXPLAINED

→ `break count * 2` gives the loop its value

→ Retry logic reads cleanly with `loop` and `match`

→ `continue 'outer` jumps to the next outer iteration

→ A labeled block can exit early with a value

COMING FROM…

↔ **Java** labeled `break outer;` but loops have no value

↔ **Python** no labels; use flags, functions or `for … else`

↔ **Go** labeled break/continue; `for {}` is the infinite loop

THE RUST WAY

⇄ `while true {}` → `loop {}`

⇄ `let mut found = false;` flags → `break` with a value

KEYWORDS

FORM	DOES
`loop { }`	Repeat forever
`break`	Exit loop
`break v`	Exit with value (loop only)
`continue`	Next iteration
`'a: loop`	Label
`break 'a / continue 'a`	Target label
`'a: { … }`	Labeled block

⚠ COMMON MISTAKES

- `break value` inside `for` or `while` (not allowed)
- Forgetting the apostrophe in labels
- Busy loops without sleep or blocking calls

✓ TIPS & BEST PRACTICES

- Use `loop` + `break value` for retry and search logic
- Extract nested loops into a function and `return` instead of labels
- Functions that never return should say `-> !`

QUICK SUMMARY

- `loop` — forever
- `break value` returns
- labels `'outer`
- never-ending → `!`

Sheet 029

while & while let

while cond · while let Some(x) = stack.pop() · counters vs iterators · continue · avoiding index loops

CONDITIONAL LOOPS

`while` repeats while a `bool` is true. `while let` repeats while a **pattern** matches, which is the idiomatic way to drain a stack, queue or iterator until it returns `None`. For walking collections, prefer `for` with iterators over index-based `while` loops.

KEY POINTS

- `while` loops have type `()` (no break value)
- `while let` stops at the first non-matching value
- Index loops pay a bounds check on each `v[i]`
- Works with `Option`, `Result` and other enums

WHILE AND WHILE LET

```
let mut n = 5;
while n > 0 {
    print!("{n} ");                          // 5 4 3 2 1
    n -= 1;
}
println!();

let mut stack = vec![1, 2, 3];
while let Some(top) = stack.pop() {
    print!("{top} ");                        // 3 2 1
}
println!();

let mut it = "a,b,c".split(',');
while let Some(part) = it.next() {
    if part == "b" { continue; }
    print!("{part} ");                       // a c
}
println!();

use std::collections::VecDeque;
let mut queue = VecDeque::from([1]);
while let Some(x) = queue.pop_front() {     // breadth-first style
    if x < 8 { queue.push_back(x * 2); queue.push_back(x * 2 + 1); }
}

let mut lines = std::io::stdin().lines();
while let Some(Ok(line)) = lines.next() {   // stops at EOF or error
    if line.is_empty() { break; }
}

let mut x: u64 = 27;
let mut steps = 0;
while x != 1 { x = if x % 2 == 0 { x / 2 } else { 3 * x + 1 }; steps += 1; }
println!("{steps}");                         // 111
```

CODE EXPLAINED

- → `while` checks the condition before each pass
- → `while let Some(top)` ends when `pop` returns None
- → The queue example processes items until empty
- → Nested patterns like `Some(Ok(line))` work too

COMING FROM…

- ↔ **Python / Java** `while stack:` / `while (!stack.isEmpty())`
- ↔ **C++** `while (!s.empty()) { auto x = s.top(); s.pop(); }`
- ↔ **Go** `for len(stack) > 0 { … }` (no while keyword)

THE RUST WAY

- ⇄ `while i < v.len() { v[i]; i += 1 }` → `for x in &v`
- ⇄ `loop { match it.next() { Some(x) => …, None => break } }` → `while let`

LOOP CHOICE

SITUATION	USE
Walk a collection	for x in &v
Count a range	for i in 0..n
Until condition	while cond
Until None	while let Some(x) = …
Forever / retry	loop
Need a value out	loop + break v

⚠ COMMON MISTAKES

- Off-by-one errors in manual index loops
- Mutating a Vec while iterating it with `for` (borrow error)
- Expecting `while let` to skip non-matching items (it stops)

✓ TIPS & BEST PRACTICES

- Use `while let` for stacks, queues and channel receivers
- Use `retain` instead of removing inside a loop
- Prefer iterator adapters for filtering

QUICK SUMMARY

- `while cond`
- `while let Some(x) = …`
- stops at first mismatch
- prefer `for` over index loops

for Loops

ITERATING OVER ANYTHING

`for` loops over anything that implements `IntoIterator`. How you name the collection decides ownership: `for x in &v` borrows each item, `for x in &mut v` lets you modify items, and `for x in v` **consumes** the collection. Adapters like `enumerate`, `rev` and `zip` add indexes, reverse order and pairing.

KEY POINTS

- `for x in v` moves `v`; it cannot be used afterwards
- `for` loops have no bounds checks on each item
- Use `enumerate()` instead of a manual counter
- Arrays iterate by value in edition 2021+

FOR LOOPS

```
let mut v = vec![10, 20, 30];

for x in &v { print!("{x} "); }              // borrow: 10 20 30
for x in &mut v { *x += 1; }                 // modify in place
for (i, x) in v.iter().enumerate() {
    print!("{i}:{x} ");                      // 0:11 1:21 2:31
}
for x in v.iter().rev() { print!("{x} "); }  // 31 21 11
println!();

let names = ["Ada", "Bob"];
let ages = [36, 25];
for (n, a) in names.iter().zip(ages) {
    println!("{n} is {a}");
}

for x in v { drop(x); }                      // consumes v
// println!("{:?}", v);                    // error: borrow of moved value

for i in (0..10).step_by(5) { print!("{i} "); }   // 0 5
for c in "hey".chars() { print!("{c}."); }   // h.e.y.
println!();

use std::collections::HashMap;
let scores = HashMap::from([("a", 1), ("b", 2)]);
for (k, val) in &scores { println!("{k}={val}"); }   // order not fixed

let grid = vec![vec![1, 2], vec![3, 4]];
for row in &grid {
    for cell in row { print!("{cell} "); }
}
```

CODE EXPLAINED

- → `&v` yields `&i32`; `&mut v` yields `&mut i32`
- → `*x += 1` dereferences to change the item
- → `zip` pairs two iterators and stops at the shorter
- → Looping over `v` by value moves it

COMING FROM…

- ↔ **Python** `for i, x in enumerate(v)`, `zip(a, b)`
- ↔ **Java** enhanced `for (var x : list)`; no built-in enumerate
- ↔ **C++** range-based `for (auto& x : v)` ≈ `for x in &mut v`

THE RUST WAY

- ⇄ `for i in 0..v.len() { v[i] }` → `for x in &v` (clippy: `needless_range_loop`)
- ⇄ manual counter `let mut i = 0` → `.enumerate()`

OWNERSHIP IN FOR

FORM	ITEM TYPE	AFTER LOOP
`for x in &v`	`&T`	`v` usable
`for x in &mut v`	`&mut T`	`v` usable
`for x in v`	`T`	`v` moved
`for x in v.iter()`	`&T`	`v` usable
`for x in v.drain(..)`	`T`	`v` empty, usable

⚠ COMMON MISTAKES

- Consuming `v` with `for x in v` and needing it later
- Pushing to `v` inside `for x in &v` (borrow error)
- Forgetting `*` when modifying through `&mut`

✓ TIPS & BEST PRACTICES

- Use `iter().copied()` to get values from `&[i32]`
- Use `chunks(n)` / `windows(n)` for groups (sheet 038)
- Labels work on `for` too: `'row: for …`

QUICK SUMMARY

- `for x in iter`
- `&v` · `&mut v` · `v`
- `enumerate` / `rev` / `zip`
- no index loops

Sheet 031

match Basics

`match` · exhaustiveness · `_` wildcard · `|` multiple patterns · ranges `1..=5` · match as an expression · matching `Option` / `Result`

PATTERN MATCHING

`match` compares a value against patterns from top to bottom and runs the first arm that matches. The compiler checks that it is **exhaustive**: every possible value must be covered, so adding an enum variant shows you every `match` to update. `match` is an expression, and all arms must return the same type.

KEY POINTS

- `_` matches anything and binds nothing
- `a | b` matches either; `1..=5` matches a range
- Arms are `pattern => expression,`
- A name in a pattern binds the value

MATCH

```
let n = 7;
let kind = match n {
    0 => "zero",
    1 | 2 | 3 => "small",
    4..=9 => "medium",
    _ => "large",
};
println!("{kind}");                          // medium

let c = 'q';
match c {
    'a' | 'e' | 'i' | 'o' | 'u' => println!("vowel"),
    'a'..='z' => println!("consonant"),     // consonant
    _ => println!("other"),
}

let maybe: Option<i32> = Some(42);
match maybe {
    Some(0) => println!("zero"),
    Some(x) => println!("got {x}"),         // got 42
    None => println!("nothing"),
}

let r: Result<u8, String> = "300".parse::<u8>().map_err(|e| e.to_string());
let value = match r {
    Ok(v) => v,
    Err(e) => { eprintln!("bad: {e}"); 0 }  // bad: number too large…
};

let pair = (0, -3);
match pair {
    (0, y) => println!("on y axis at {y}"),
    (x, 0) => println!("on x axis at {x}"),
    _ => println!("elsewhere"),
}
```

CODE EXPLAINED

→ Arms are tried in order; `_` catches the rest

→ Char ranges and alternatives combine

→ `Some(x)` binds the inner value to `x`

→ A block arm can run statements then return a value

COMING FROM…

↔ **C++ / Java** `switch`; Java 21 pattern `switch` is the closest match

↔ **Python** `match` (3.10) with similar patterns, but not exhaustive

↔ **Go** `switch` with no exhaustiveness check

THE RUST WAY

⇄ `_ =>` on your own enums → list variants so new ones cause errors

⇄ chains of `if x == …` → `match`

PATTERN FORMS

PATTERN	MATCHES
`5`	Literal
`x`	Anything, binds x
`_`	Anything, no binding
`a | b`	Either
`1..=5`	Inclusive range
`Some(x)`	Enum variant
`(a, 0)`	Tuple shape
`n if n > 0`	With guard

⚠ COMMON MISTAKES

- Putting `_` first so later arms never run (warning: unreachable)
- Non-exhaustive match on integers without `_`
- Arms that return different types

✓ TIPS & BEST PRACTICES

- Let exhaustiveness errors guide refactors
- Use `matches!(x, Pat)` for a bool test
- Keep arms short; call functions for big branches

QUICK SUMMARY

- first matching arm wins
- exhaustive by design
- `_` · `|` · `..=`
- match is an expression

Sheet 032

Patterns in Depth

destructuring structs / tuples / enums · `..` rest · `@` bindings · match guards · `ref` / `ref mut` · slice patterns `[first, .., last]`

DESTRUCTURING WITH PATTERNS

Patterns can take apart structs, tuples, enums, references and slices in one step. `..` skips the remaining fields, `@` binds a value while also testing it, and a **guard** (`if cond`) adds a condition to an arm. Patterns appear in `match`, `let`, `if let`, function parameters and `for` loops.

KEY POINTS

- Struct patterns: `Point { x, y: 0 }`
- Slice patterns match length and position
- Default binding mode borrows when matching a reference
- Guards do not count toward exhaustiveness

ADVANCED PATTERNS

```
struct Point { x: i32, y: i32 }
enum Shape { Circle { r: f64 }, Rect { w: f64, h: f64 } }

let p = Point { x: 3, y: 0 };
match p {
    Point { x: 0, .. } => println!("on y axis"),
    Point { x, y: 0 } => println!("on x axis at {x}"),   // matches
    Point { x, y } => println!("({x}, {y})"),
}

let s = Shape::Rect { w: 2.0, h: 3.0 };
let area = match s {
    Shape::Circle { r } => std::f64::consts::PI * r * r,
    Shape::Rect { w, h } if w == h => w * w,    // guard
    Shape::Rect { w, h } => w * h,
};

let age = 15;
match age {
    n @ 13..=19 => println!("teen: {n}"),   // bind and test
    n => println!("age {n}"),
}

let v = [1, 2, 3, 4, 5];
match v.as_slice() {
    [] => println!("empty"),
    [one] => println!("one: {one}"),
    [first, .., last] => println!("{first}..{last}"),   // 1..5
}
if let [a, b, rest @ ..] = &v[..] { println!("{a} {b} {rest:?}"); }

let opt = Some(String::from("hi"));
if let Some(ref s) = opt { println!("{s}"); }   // borrow, don't move
match &opt { Some(s) => println!("{s}"), None => {} }   // same, modern style

let ((a2, b2), Point { x: px, .. }) = ((1, 2), Point { x: 9, y: 9 });
fn print_pair(&(a, b): &(i32, i32)) { println!("{a},{b}"); }
```

CODE EXPLAINED

- → `..` ignores fields you do not need
- → The guard `if w == h` only runs after the shape matches
- → `n @ 13..=19` tests the range and keeps the value
- → `rest @ ..` captures the remaining slice

COMING FROM…

- ↔ **Python** structural `match` with class patterns and guards
- ↔ **Java 21** record patterns and `when` guards in switch
- ↔ **C++** structured bindings, but no pattern matching in `switch`

THE RUST WAY

- ⇄ `ref` keyword in new code → match on `&value`
- ⇄ indexing `v[0]` after a length check → slice pattern

SYNTAX

PATTERN	MEANING
`S { a, .. }`	Struct, ignore rest
`S { a: 0, b }`	Field equals + bind
`(x, _)`	Tuple, ignore 2nd
`[x, .., y]`	First and last
`[x, rest @ ..]`	Head and tail
`n @ 1..=9`	Bind + range
`&(a, b)`	Through a reference
`p if cond`	Guard

⚠ COMMON MISTAKES

- Moving out of a value inside `match` and using it later (match on `&x`)
- Relying on a guard for exhaustiveness
- Forgetting `..` and listing every struct field

✓ TIPS & BEST PRACTICES

- Destructure in function parameters for clarity
- Use slice patterns for command parsing: `["add", name]`
- Match on tuples to handle combinations of two values

QUICK SUMMARY

- destructure anything
- `..` — rest
- `@` — bind + test
- guards `if cond`

Sheet 033

if let, let else & let Chains

`if let` · `if let … else` · `let … else { return }` · let chains (2024 edition) · `matches!` · `while let`

MATCHING ONE PATTERN

`if let` runs code only when one pattern matches, a shorter form of a two-arm `match`. `let … else` binds a value or **diverges** (return, break, panic), which keeps the happy path unindented. The 2024 edition (Rust 1.88+) adds **let chains**: `if let Some(x) = a && x > 0`.

KEY POINTS

- `let … else` requires the else block to diverge
- Let chains need edition 2024
- `matches!(v, Pat)` returns a bool
- Bindings from `if let` live only in its block

IF LET, LET ELSE, LET CHAINS

```rust
let config: Option<&str> = Some("8080");

if let Some(port) = config {
    println!("port {port}");
} else {
    println!("default port");
}

fn parse_port(s: Option<&str>) -> Result<u16, String> {
    let Some(text) = s else {
        return Err("missing".into());       // must diverge
    };
    let Ok(port) = text.parse::<u16>() else {
        return Err(format!("bad port: {text}"));
    };
    Ok(port)                                // happy path stays flat
}
println!("{:?}", parse_port(config));      // Ok(8080)

// Let chains (edition 2024)
let user: Option<(&str, u32)> = Some(("ada", 36));
if let Some((name, age)) = user
    && age >= 18
    && let Some(first) = name.chars().next()
{
    println!("{first} is an adult");        // a is an adult
}

#[derive(Debug)]
enum Cmd { Quit, Move(i32) }
let c = Cmd::Move(3);
println!("{}", matches!(c, Cmd::Move(n) if n > 0));   // true

let mut tokens = vec!["b", "a"];
while let Some(t) = tokens.pop() { print!("{t} "); }  // a b
```

CODE EXPLAINED

- → `if let` handles the one case you care about
- → `let … else` returns early when the pattern fails
- → Let chains combine patterns and conditions with `&&`
- → `matches!` accepts guards too

COMING FROM…

- ↔ **Swift** `if let` and `guard let … else` (same idea)
- ↔ **Kotlin** `?.let { }` and `?: return`
- ↔ **Python** walrus `if (m := re.match(...)):`

THE RUST WAY

- ⇄ nested `if let` pyramids → `let … else` or let chains
- ⇄ `match x { Some(v) => …, None => {} }` → `if let Some(v) = x`

FORMS

FORM	USE
`if let P = e { }`	One pattern
`if let … else { }`	Pattern or fallback
`let P = e else { return };`	Bind or exit
`if let … && cond`	Let chain (2024)
`while let P = e`	Loop while matching
`matches!(e, P)`	Bool test

⚠ COMMON MISTAKES

- `let … else` block that does not diverge (compile error)
- Let chains on an older edition (use nested ifs)
- Using `if let` where a missing case should be handled

✓ TIPS & BEST PRACTICES

- Use `let … else` for validation at the top of functions
- Combine with `?` for Option/Result chains
- Use `matches!` in `filter` closures

QUICK SUMMARY

- `if let` — one pattern
- `let … else` — bind or exit
- let chains (2024)
- `matches!` — bool

Sheet 034

Ownership & Moves

three ownership rules · move on assignment · move into functions · drop at scope end · E0382 use after move · stack vs heap

EVERY VALUE HAS ONE OWNER

Ownership is how Rust manages memory without a garbage collector. **Each value has one owner; when the owner goes out of scope, the value is dropped.** Assigning a heap-owning value like `String` or `Vec` to another variable, or passing it to a function, **moves** it: the old name can no longer be used. This prevents double frees at compile time.

KEY POINTS

- Rule 1: each value has an owner
- Rule 2: only one owner at a time
- Rule 3: owner out of scope → value dropped
- Copy types (ints, bool, char) are copied instead of moved

UNDER THE HOOD

- A move is a shallow `memcpy` of the stack part (ptr, len, cap)
- The source is marked uninitialized at compile time; no runtime flag in most cases
- Drop runs at the closing brace, in reverse declaration order
- When drops depend on runtime paths, the compiler adds a hidden drop flag

MOVES AND DROPS

```
let s1 = String::from("hello");             // s1 owns the heap buffer
let s2 = s1;                                // MOVE: s2 owns it now
// println!("{s1}");                 // E0382: borrow of moved value: `s1`
println!("{s2}");

let n1 = 5;
let n2 = n1;                                // i32 is Copy: both usable
println!("{n1} {n2}");

fn take(s: String) { println!("took {s}"); }   // s dropped at end
fn give() -> String { String::from("new") }    // ownership moves out

let a = String::from("data");
take(a);                                    // a moved into take
// take(a);                                 // E0382: use of moved value
let b = give();                             // b owns the returned String

fn take_and_return(s: String) -> String { s }
let c = take_and_return(b);                 // round-trip ownership

{
    let temp = vec![1, 2, 3];               // allocated
    println!("{}", temp.len());
}                                           // temp dropped here, memory freed

let v = vec![String::from("x"), String::from("y")];
let first = v[0].clone();                   // can't move out of a Vec index
for s in v { println!("{s}"); }              // v moved into the loop
```

CODE EXPLAINED

- → `let s2 = s1` transfers ownership; `s1` becomes unusable
- → Integers implement `Copy`, so assignment copies them
- → Passing `a` to `take` moves it into the function
- → Leaving a block drops everything it owns

COMING FROM…

- ↔ **C++** like `std::move`, but the source cannot be used afterwards (no moved-from state)
- ↔ **Java / Python / Go** assignment copies a reference; a GC frees memory later
- ↔ **C** manual `free`; Rust inserts the free at scope end

THE RUST WAY

- ⇄ `.clone()` to silence E0382 → pass `&s` if the callee only reads
- ⇄ returning a value just to give it back → borrow instead (sheet 036)

WHAT MOVES

TYPE	ON ASSIGN
`i32, f64, bool, char`	Copy
`&T`	Copy
`(i32, bool)`	Copy if all fields are
`String, Vec<T>`	Move
`Box<T>`	Move
`Your struct`	Move (unless derive Copy)

STACK VS HEAP

STACK	HEAP
`Fixed size, known at compile time`	Size can change
`Fast push/pop`	Allocator call
`i32, arrays, &T, String header`	String/Vec contents, Box
`Freed at scope end`	Freed when owner drops

⚠ COMMON MISTAKES

- Using a variable after passing it to a function by value
- Cloning everything instead of borrowing
- Trying to move a field out of a borrowed struct

✓ TIPS & BEST PRACTICES

- Read E0382 notes: they show where the move happened
- Take `&T` when a function only needs to read
- Use `std::mem::take` to move out and leave a default

QUICK SUMMARY

- one owner per value
- assignment moves heap types
- scope end → drop
- Copy types are copied

Sheet 035

Copy vs Clone

Copy types (ints, bool, char, &T) · Clone / .clone() · #[derive(Copy, Clone)] · cost of cloning · to_owned() · when cloning is fine

IMPLICIT COPIES AND EXPLICIT CLONES

Copy types are duplicated bit-for-bit on assignment; the original stays usable. **Clone** is an explicit, possibly expensive deep copy you ask for with `.clone()`. A type can be `Copy` only if all its fields are `Copy` and it has no `Drop`, so `String` and `Vec` are Clone but never Copy.

KEY POINTS

- Copy requires Clone (`#[derive(Clone, Copy)]`)
- `clone()` on `Rc` / `Arc` only bumps a counter
- `to_owned()` turns a borrow into an owned value (`&str` → `String`)
- Cloning small data in non-hot code is fine

COPY AND CLONE

```
#[derive(Debug, Clone, Copy, PartialEq)]
struct Point { x: i32, y: i32 }             // all fields Copy → can be Copy

let p1 = Point { x: 1, y: 2 };
let p2 = p1;                                // copied
println!("{p1:?} {p2:?}");                  // both usable

#[derive(Debug, Clone)]
struct User { name: String, tags: Vec<String> }   // String → not Copy

let u1 = User { name: "Ada".into(), tags: vec!["admin".into()] };
let u2 = u1.clone();                        // deep copy: new heap buffers
let u3 = u1;                                // move
// println!("{:?}", u1);                    // E0382
println!("{:?} {:?}", u2, u3);

let s: &str = "hi";
let owned: String = s.to_owned();           // &str → String
let slice: &[i32] = &[1, 2];
let v: Vec<i32> = slice.to_vec();           // &[T] → Vec<T>

use std::rc::Rc;
let shared = Rc::new(vec![0u8; 1_000_000]);
let alias = Rc::clone(&shared);             // cheap: count 1 → 2

let mut names = vec![String::from("b"), String::from("a")];
let mut sorted = names.clone();             // fine: keep the original
sorted.sort();
names.push("c".into());
```

CODE EXPLAINED

- → `Point` is Copy because both fields are
- → `u1.clone()` allocates new String and Vec buffers
- → `to_owned` / `to_vec` create owned copies of borrowed data
- → `Rc::clone` shares instead of copying the data

COMING FROM…

- ↔ **C++** copy constructors run implicitly; Rust makes deep copies explicit
- ↔ **Python** `copy.deepcopy()` ; assignment only copies the reference
- ↔ **Java** `clone()` exists but is rarely used; records are copied by reference

THE RUST WAY

- ⇄ `x.clone()` on a Copy type → just `x` (clippy: `clone_on_copy`)
- ⇄ `rc.clone()` → `Rc::clone(&rc)` to show it is cheap

COMPARE

ITEM	COPY	CLONE
Trigger	Implicit on assign	Explicit .clone()
Cost	Bitwise, cheap	Can allocate
Allowed for	No heap, no Drop	Any type
Examples	i32, char, &T, (u8, bool)	String, Vec, HashMap
Derive	Clone, Copy	Clone

⚠ COMMON MISTAKES

- Deriving `Copy` on a type with a `String` field (compile error)
- Cloning large Vecs inside loops
- Making big structs Copy (silent large memcpys)

✓ TIPS & BEST PRACTICES

- Derive Copy for small value types (points, IDs, colors)
- Clone freely at startup and in tests; avoid in hot loops
- Use `Rc` / `Arc` when many owners need the same large data

QUICK SUMMARY

- Copy — implicit, bitwise
- Clone — explicit `.clone()`
- no Copy with heap or Drop
- `to_owned()` — borrow → owned

Sheet 036

References & Borrowing

&T shared borrow · ∗ deref · auto-deref on method calls · the borrowing rules · references must stay valid · passing & to functions

USING VALUES WITHOUT TAKING THEM

A **reference** &T borrows a value without taking ownership, so the owner can keep using it afterwards. The borrow checker enforces two rules: you can have **many &T or one `&mut T`** at a time, and a reference can never outlive the value it points to. References are never null.

KEY POINTS

- `&x` creates a reference; `*r` reads through it
- Method calls and `.` field access auto-deref
- Comparison operators work through references
- Function signatures show borrows clearly: `fn len(s: &String)`

UNDER THE HOOD

- `&T` is a plain pointer at runtime (8 bytes); the rules cost nothing at runtime
- `&[T]`, `&str` and `&dyn Trait` are fat pointers (16 bytes)
- References to Copy data may be optimized into passing the value
- `Option<&T>` is the same size as `&T` (null is the None niche)

SHARED REFERENCES

```
fn length(s: &String) -> usize { s.len() }  // borrows, doesn't own

let name = String::from("Ferris");
let n = length(&name);                      // lend it
println!("{name} has {n} letters");         // name still usable

let x = 5;
let r = &x;
println!("{}", *r + 1);                     // 6, explicit deref
println!("{}", r + 1);                      // 6, arithmetic auto-derefs
println!("{}", r == &5);                    // true

let words = vec![String::from("a"), String::from("bb")];
let first: &String = &words[0];             // borrow an element
println!("{}", first.len());                // auto-deref for methods

let r1 = &words;
let r2 = &words;                            // many shared borrows OK
println!("{} {}", r1.len(), r2.len());

fn longest_len(items: &[String]) -> usize {
    items.iter().map(|s| s.len()).max().unwrap_or(0)
}
println!("{}", longest_len(&words));        // 2

// let dangling: &String;
// { let tmp = String::from("x"); dangling = &tmp; }
//                                          // E0597: does not live long enough
// println!("{dangling}");

let rr: &&i32 = &&7;
println!("{}", **rr);                       // 7
```

CODE EXPLAINED

- `length(&name)` lends `name` and gets it back automatically
- `*r` follows the reference to the value
- Many `&` borrows can coexist
- A reference to `tmp` cannot escape `tmp`'s block (E0597)

COMING FROM…

- **C++** references and pointers can dangle; Rust proves they cannot
- **Go** pointers keep values alive via the GC
- **Java / Python** every object variable is already a shared reference

THE RUST WAY

- `fn f(v: Vec<T>)` that only reads → `fn f(v: &[T])`
- explicit `(*r).len()` → `r.len()` (auto-deref)

BORROWING RULES

RULE	MEANING
Many &T	Any number of readers
One &mut T	Exactly one writer
Not both at once	No reading during a write
Must not dangle	Ref ≤ lifetime of value
Never null	Use Option<&T>

SYNTAX

CODE	MEANING
&x	Borrow x
&mut x	Mutable borrow
*r	Deref
r.method()	Auto-deref
&T in signature	Takes a borrow
&&T	Reference to reference

⚠ COMMON MISTAKES

- Returning a reference to a local variable (E0515/E0106)
- Storing references in structs without lifetimes (sheet 040)
- Cloning instead of borrowing to make errors go away

✓ TIPS & BEST PRACTICES

- Borrow by default; move only when the callee must own
- Prefer `&str` and `&[T]` parameters
- Let NLL shorten borrows: use the value, then move on

QUICK SUMMARY

- `&T` — shared borrow
- many `&` or one `&mut`
- refs never dangle or null
- `*` deref, auto-deref on `.`

Sheet 037

Mutable References

`&mut T` · one mutable OR many shared · E0499 / E0502 · non-lexical lifetimes (NLL) · reborrowing · `mem::swap` / `take` / `replace`

EXCLUSIVE ACCESS

A `&mut T` lets you change a value you do not own. While it exists, it is the **only** way to reach that value: no other `&mut` (E0499) and no `&` (E0502). This rule prevents data races and iterator invalidation at compile time. Borrows end at their **last use** (non-lexical lifetimes), not at the closing brace.

KEY POINTS

- The variable itself must be `let mut` to take `&mut`
- `&mut` is not Copy; it is moved or reborrowed
- Passing `&mut *r` or just `r` to a function reborrows
- `mem::take` moves a value out, leaving `Default`

&MUT AND THE RULES

```
fn add_suffix(s: &mut String) { s.push_str("!"); }

let mut msg = String::from("hi");
add_suffix(&mut msg);
println!("{msg}");                          // hi!

let r1 = &mut msg;
// let r2 = &mut msg;                       // E0499: second mutable borrow
r1.push('?');

let mut v = vec![1, 2, 3];
let first = &v[0];
// v.push(4);                               // E0502: v borrowed as immutable
println!("{first}");                        // last use of `first`
v.push(4);                                  // OK now (NLL)

for x in v.iter_mut() { *x *= 10; }         // [10, 20, 30, 40]

fn bump(n: &mut i32) { *n += 1; }
let mut count = 0;
let r = &mut count;
bump(r);                                    // implicit reborrow
bump(r);                                    // r still usable
println!("{count}");                        // 2

use std::mem;
let mut a = String::from("a");
let mut b = String::from("b");
mem::swap(&mut a, &mut b);                  // a = "b", b = "a"
let old = mem::take(&mut a);                // a = "", old = "b"
let prev = mem::replace(&mut b, "z".into()); // b = "z", prev = "a"

let (left, right) = v.split_at_mut(2);      // two &mut to disjoint parts
left[0] += right[0];
```

CODE EXPLAINED

→ `&mut msg` lets `add_suffix` modify the caller's String

→ A second `&mut` while `r1` is live is rejected

→ After `first`'s last use, `v` can be mutated again

→ `split_at_mut` gives two non-overlapping mutable slices

COMING FROM…

↔ **C++** any number of non-const references may alias

↔ **Go / Java** shared mutable objects; data races are possible

↔ **Python** everything is mutable through any reference

THE RUST WAY

⇄ `let tmp = x.clone(); x = y; y = tmp;` → `mem::swap(&mut x, &mut y)`

⇄ `Option` field move-out errors → `field.take()`

ERRORS

CODE	MEANING
E0499	Two &mut at once
E0502	& and &mut overlap
E0506	Assign while borrowed
E0596	&mut of non-mut binding

STD::MEM

FUNCTION	DOES
`swap(&mut a, &mut b)`	Exchange
`take(&mut a)`	Move out, leave Default
`replace(&mut a, v)`	Move out, put v
`drop(x)`	Drop now

⚠ COMMON MISTAKES

- Holding a reference into a Vec while pushing to it
- Forgetting `let mut` on the owner
- Calling `self.a()` (&mut self) while holding `&self.b` across it

✓ TIPS & BEST PRACTICES

- Shorten borrows: copy out the value you need, then mutate
- Borrow disjoint struct fields separately: `&mut s.a`, `&mut s.b`
- Use `split_at_mut` / `chunks_mut` for parallel slice edits

QUICK SUMMARY

- `&mut T` — exclusive
- E0499 / E0502
- borrows end at last use
- `mem::swap` / `take` / `replace`

Slices

VIEWS INTO CONTIGUOUS DATA

A **slice** `&[T]` is a borrowed view of a contiguous run of elements in an array, `Vec` or other buffer. Slicing with a range out of bounds **panics**. Functions that read sequences should take `&[T]`, which accepts arrays, Vecs and sub-slices alike, just as `&str` accepts every kind of string.

KEY POINTS

- `&Vec<T>` coerces to `&[T]` automatically
- `get(range)` returns `Option<&[T]>` instead of panicking
- `&mut [T]` allows in-place edits but not resizing
- Most Vec reading methods are actually slice methods

UNDER THE HOOD

- A slice reference is a fat pointer: start pointer + length (16 bytes)
- `&v[1..3]` is pointer arithmetic plus a bounds check
- `[T]` itself is unsized (`!Sized`); it lives behind `&`, `Box` or `Rc`
- Iterating a slice compiles to a pointer loop with no per-item bounds check

SLICE OPERATIONS

```
let v = vec![10, 20, 30, 40, 50];
let arr = [1, 2, 3];

let mid: &[i32] = &v[1..4];                 // [20, 30, 40]
println!("{:?} {:?}", &v[..2], &v[3..]);    // [10, 20] [40, 50]
// let bad = &v[3..9];                      // panic: range end 9 out of range
println!("{:?}", v.get(3..9));              // None

fn total(xs: &[i32]) -> i32 { xs.iter().sum() }
println!("{} {} {}", total(&v), total(&arr), total(mid));   // 150 6 90

println!("{:?} {:?}", v.first(), v.last()); // Some(10) Some(50)
let (left, right) = v.split_at(2);          // [10, 20] [30, 40, 50]
if let Some((head, tail)) = v.split_first() { println!("{head} {tail:?}"); }

for pair in v.chunks(2) { print!("{pair:?} "); }    // [10, 20] [30, 40] [50]
println!();
for w in v.windows(3) { print!("{} ", w.iter().sum::<i32>()); }   // 60 90 120
println!();

let mut nums = [5, 3, 8, 1];
let s: &mut [i32] = &mut nums[1..];
s.sort();                                   // sort part of the array
println!("{nums:?}");                       // [5, 1, 3, 8]
nums.reverse();
nums.fill(0);
println!("{}", v.contains(&30));            // true
println!("{:?}", v.iter().position(|&x| x == 40));   // Some(3)
println!("{:?}", [[1, 2], [3, 4]].concat()); // [1, 2, 3, 4]
```

CODE EXPLAINED

- → `&v[1..4]` borrows three elements without copying
- → `get(range)` avoids a panic on bad ranges
- → One function accepts Vec, array and sub-slice
- → `chunks` splits into groups; `windows` slides an overlapping view

COMING FROM…

- ↔ **C++20** `std::span<T>` ≈ `&[T]`
- ↔ **Go** slices, but Go slices can grow with `append` and share backing arrays
- ↔ **Python** `v[1:4]` copies into a new list

THE RUST WAY

- ⇄ `fn f(v: &Vec<i32>)` → `fn f(v: &[i32])`
- ⇄ `v[v.len() - 1]` → `v.last()` (no panic on empty)

SLICE METHODS

METHOD	RETURNS
`len / is_empty`	Size
`first / last`	Option<&T>
`get(i) / get(a..b)`	Option
`split_at(i)`	(&[T], &[T])
`split_first / split_last`	Option<(&T, &[T])>
`chunks(n) / windows(n)`	Iterators
`contains / starts_with`	bool
`iter / iter_mut`	Iterators

IN-PLACE (&MUT)

METHOD	DOES
`sort / sort_unstable`	Sort
`reverse`	Reverse
`fill(v)`	Set all
`swap(i, j)`	Swap two
`rotate_left(n)`	Rotate
`copy_from_slice`	Overwrite

⚠ COMMON MISTAKES

- Out-of-range slicing in production code (use `get`)
- Trying to `push` through a slice
- Returning a slice of a local Vec (dangles: E0515)

✓ TIPS & BEST PRACTICES

- Take `&[T]` / return `Vec<T>` in APIs
- Use `chunks_exact` when the remainder must be handled separately
- Slice patterns work on slices: `[first, rest @ ..]`

QUICK SUMMARY

- `&[T]` — borrowed view
- out-of-range slice panics
- `get(range)` → Option
- `chunks` / `windows` / `split_at`

Sheet 039

Lifetimes — Basics

'a syntax · why lifetimes exist · E0106 / E0597 · functions returning references · 'static · lifetimes describe, they don't extend

NAMING HOW LONG BORROWS LIVE

A **lifetime** is the region of code where a reference is valid. The compiler infers most of them, but when a function returns a reference derived from its inputs, you sometimes have to say **which input** it borrows from with a lifetime parameter like `'a`. Lifetimes are checked at compile time and never change how long a value lives.

KEY POINTS

- `'a` reads "lifetime a"
- `fn f<'a>(x: &'a str) -> &'a str`: result lives as long as x
- `'static` = valid for the whole program (literals, leaked data)
- Lifetime errors mean a reference might dangle

LIFETIME ANNOTATIONS

```
// fn longest(a: &str, b: &str) -> &str  // E0106: missing lifetime specifier
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}

let s1 = String::from("long string");
let result;
{
    let s2 = String::from("xyz");
    result = longest(&s1, &s2);
    println!("{result}");                    // OK: s2 still alive here
}
// println!("{result}");            // E0597: `s2` does not live long enough

fn first<'a>(a: &'a str, _b: &str) -> &'a str { a }   // only tied to a

let kept;
{
    let temp = String::from("temp");
    kept = first(&s1, &temp);
}
println!("{kept}");                          // fine: depends only on s1

let lit: &'static str = "lives forever";    // in the binary
fn get_name() -> &'static str { "Ferris" }  // returning a literal is OK

// fn make() -> &String {                    // E0106
//     let s = String::from("x"); &s         // would dangle
// }
fn make() -> String { String::from("x") }   // return owned data instead
```

CODE EXPLAINED

→ `'a` says the result borrows from both a and b

→ The result cannot be used after the shorter input dies

→ `first` ties the result only to `a`, so `temp` can die

→ A reference to a local cannot be returned; return the owned value

COMING FROM…

↔ **C++** returning a `string_view` to a temporary compiles and dangles

↔ **Go / Java / Python** the GC keeps referenced data alive

↔ **C** no checks; dangling pointers are undefined behavior

THE RUST WAY

⇄ adding `'static` to silence errors → return owned data

⇄ `<'a>` on every function → let elision handle simple cases

SYNTAX

FORM	MEANING
`'a`	Lifetime parameter
`&'a T`	Reference valid for a
`&'a mut T`	Mutable, valid for a
`fn f<'a>(…)`	Declare lifetime
`'static`	Whole program
`T: 'static`	Owns its data (no short borrows)

ERRORS

CODE	MEANING
E0106	Missing lifetime specifier
E0597	Value does not live long enough
E0515	Returns reference to local
E0716	Temporary dropped while borrowed

⚠ COMMON MISTAKES

- Thinking `'a` makes a value live longer
- Using `'static` bounds when owned types would do
- Returning references to locals or temporaries

✓ TIPS & BEST PRACTICES

- Return owned values (`String`, `Vec`) when unsure
- Read the error's "borrowed value does not live long enough" arrow diagram
- Most code needs no explicit lifetimes

QUICK SUMMARY

- `'a` — names a borrow region
- ties output to inputs
- `'static` — whole program
- describe, never extend

Sheet 040

Lifetimes — Structs & Elision

structs holding &'a T · impl<'a> · three elision rules · '_ anonymous lifetime · T: 'a bounds · own instead of borrow

BORROWED DATA IN STRUCTS

A struct that holds a reference needs a lifetime parameter, which guarantees the struct cannot outlive the data it borrows. **Elision rules** let you skip annotations in common function signatures, and '_ marks an inferred lifetime. In practice, most structs should own their data; borrowing structs suit short-lived views like parsers.

KEY POINTS

- `struct Parser<'a> { input: &'a str }`
- Impl blocks repeat the parameter: `impl<'a> Parser<'a>`
- Elision: a `&self` method returns refs tied to self
- `'_` shows a borrow without naming it

STRUCTS WITH REFERENCES

```
struct Parser<'a> {
    input: &'a str,                         // borrows, never owns
    pos: usize,
}

impl<'a> Parser<'a> {
    fn new(input: &'a str) -> Self { Parser { input, pos: 0 } }

    fn next_word(&mut self) -> Option<&'a str> {   // tied to input, not self
        let rest = self.input[self.pos..].trim_start();
        if rest.is_empty() { return None; }
        let start = self.input.len() - rest.len();
        let end = rest.find(' ').map_or(self.input.len(), |i| start + i);
        self.pos = end;
        Some(&self.input[start..end])
    }
}

let text = String::from("parse these words");
let mut p = Parser::new(&text);
while let Some(w) = p.next_word() {
    print!("[{w}] ");                       // [parse] [these] [words]
}

// Elision: these two signatures are the same
fn first_word(s: &str) -> &str { s.split(' ').next().unwrap_or("") }
fn first_word2<'a>(s: &'a str) -> &'a str { s.split(' ').next().unwrap_or("") }

impl Parser<'_> {                           // '_ when the name isn't needed
    fn remaining(&self) -> &str { &self.input[self.pos..] }   // tied to self
}

struct Holder<'a, T: 'a> { item: &'a T }    // T must outlive 'a

struct Config { name: String }              // usually better: own the data
```

CODE EXPLAINED

→ `Parser<'a>` cannot outlive the string it reads

→ `next_word` returns `&'a str`, so words outlive the `&mut self` borrow

→ Elision fills in lifetimes for one-input functions

→ `impl Parser<'_>` skips naming an unused lifetime

COMING FROM…

↔ **C++** a struct holding a `string_view` can outlive its buffer

↔ **Java / Go** objects hold references; the GC prevents dangling

↔ **Python** same as Java; no concept of borrowed fields

THE RUST WAY

⇄ `struct User<'a> { name: &'a str }` in long-lived data → `name: String`

⇄ naming lifetimes the compiler can elide → let elision work

ELISION RULES

RULE	EFFECT
1	Each input ref gets its own lifetime
2	One input lifetime → output gets it
3	&self / &mut self → output gets self's
Otherwise	Annotate (E0106)

WHEN TO BORROW IN STRUCTS

SITUATION	CHOICE
Parser / iterator over input	Borrow &'a
Config, models, entities	Own (String, Vec)
Shared across threads	Arc<T>
Sometimes owned	Cow<'a, T>

⚠ COMMON MISTAKES

- Adding lifetimes to every struct and spreading them through the codebase
- Returning `&str` tied to `&mut self` when it should be tied to `'a`
- Self-referential structs (not expressible safely)

✓ TIPS & BEST PRACTICES

- Start with owned fields; add borrowing only for performance
- Use `'_` to keep signatures short
- Read rule 3 when a method's returned reference keeps `self` borrowed

QUICK SUMMARY

- struct Foo<'a> { r: &'a T }
- `impl<'a>` repeats it
- three elision rules
- prefer owned fields

Sheet 041

panic! & Unrecoverable Errors

`panic!` · `unwrap()` / `expect()` · `unreachable!` / `todo!` / `unimplemented!` · `assert!` / `assert_eq!` / `debug_assert!` · `RUST_BACKTRACE=1` · `panic = "abort"`

BUGS, NOT EXPECTED FAILURES

A **panic** stops the current thread, unwinding the stack and running destructors. Use it for **bugs and broken invariants**, not for errors a caller could handle (those return `Result`). `unwrap()` and `expect()` panic on `None`/`Err`; `expect("why")` documents the assumption.

KEY POINTS

- A panic in `main`'s thread ends the program with exit code 101
- Other threads' panics show up as `Err` from `join()`
- `RUST_BACKTRACE=1` prints a stack trace
- `panic = "abort"` in Cargo.toml skips unwinding (smaller binary)

PANICKING MACROS

```
fn divide(a: i32, b: i32) -> i32 {
    if b == 0 { panic!("division by zero: {a}/{b}"); }
    a / b
}

let v = vec![1, 2, 3];
// let x = v[10];              // panic: index out of bounds: the len is 3…

let port: u16 = "8080".parse().expect("PORT must be a number");
let first = v.first().unwrap();             // fine when you just checked

fn grade(n: u8) -> char {
    match n {
        90..=100 => 'A',
        0..=89 => 'B',
        _ => unreachable!("score {n} out of range"),
    }
}
fn later() -> u32 { todo!("implement caching") }   // compiles, panics if called

assert!(v.len() == 3, "expected 3 items, got {}", v.len());
assert_eq!(divide(10, 2), 5);
assert_ne!(port, 0);
debug_assert!(v.windows(2).all(|w| w[0] <= w[1]));   // debug builds only

let r = std::panic::catch_unwind(|| divide(1, 0));
println!("{}", r.is_err());                 // true (rarely needed)

// Terminal:
// $ RUST_BACKTRACE=1 cargo run
// thread 'main' panicked at src/main.rs:2:17:
// division by zero: 1/0

// Cargo.toml
// [profile.release]
// panic = "abort"
```

CODE EXPLAINED

→ `panic!` takes a format string like `println!`

→ `expect` adds a message explaining why the value should exist

→ `unreachable!` marks branches that indicate a bug

→ `debug_assert!` is removed in release builds

COMING FROM…

↔ **Go** `panic` / `recover`, also for bugs only

↔ **Java** unchecked exceptions (`IllegalStateException`)

↔ **Python** `assert` and uncaught exceptions

THE RUST WAY

⇄ `unwrap()` on user input → return `Result` and use `?`

⇄ `unwrap()` on an invariant → `expect("reason it holds")`

MACROS

MACRO	USE
`panic!(…)`	Explicit bug
`unreachable!()`	Impossible branch
`todo!()`	Not written yet
`unimplemented!()`	Intentionally missing
`assert! / assert_eq!`	Check invariant
`debug_assert!`	Debug-only check

PANICS IMPLICITLY

CODE	WHEN
`v[i]`	Out of bounds
`x / 0`	Integer divide by zero
`a + b`	Overflow (debug)
`opt.unwrap()`	None
`&s[a..b]`	Not a char boundary
`RefCell::borrow_mut`	Already borrowed

⚠ COMMON MISTAKES

- Using panics for validation errors
- Bare `unwrap()` hiding why a value must exist
- Relying on `catch_unwind` for control flow

✓ TIPS & BEST PRACTICES

- Write `expect` messages as the expected state: "config loaded"
- Ban `unwrap` in libraries with `clippy::unwrap_used`
- Use `todo!()` to sketch code that still compiles

QUICK SUMMARY

- panic = bug
- `expect` > `unwrap`
- `unreachable!` / `todo!`
- `RUST_BACKTRACE=1`

Option<T>

A VALUE THAT MAY BE ABSENT

Rust has no null. A value that might be missing has type `Option<T>`: either `Some(value)` or `None`. The compiler forces you to handle `None` before using the value. Combinator methods like `map`, `and_then` and `unwrap_or` transform options without nested `match` blocks.

KEY POINTS

- `Option` is an enum in the prelude: `Some` / `None` need no import
- `?` on Option returns `None` early
- `ok_or` converts to `Result` with an error
- `as_ref()` / `as_deref()` borrow the inner value

UNDER THE HOOD

- `Option<&T>`, `Option<Box<T>>`, `Option<NonZeroU32>` use the null/zero **niche**: same size as T
- `Option<u32>` needs a tag byte plus padding: 8 bytes
- Combinators inline to the same code as a hand-written `match`
- `?` desugars to a `match` that returns `None`

WORKING WITH OPTION

```
fn find_user(id: u32) -> Option<&'static str> {
    match id { 1 => Some("Ada"), 2 => Some("Bob"), _ => None }
}

let name = find_user(1);
match name {
    Some(n) => println!("found {n}"),
    None => println!("no user"),
}

println!("{}", find_user(9).unwrap_or("guest"));        // guest
println!("{}", find_user(9).unwrap_or_else(|| "anon")); // lazy default
let count: u32 = None.unwrap_or_default();              // 0

let len = find_user(1).map(|n| n.len());                // Some(3)
let initial = find_user(2).and_then(|n| n.chars().next());   // Some('B')
let long = find_user(1).filter(|n| n.len() > 5);        // None
println!("{}", find_user(1).is_some_and(|n| n.starts_with('A')));   // true

let res: Result<&str, String> = find_user(7).ok_or("not found".to_string());

fn first_char_upper(id: u32) -> Option<char> {
    let n = find_user(id)?;                 // None → return None
    Some(n.chars().next()?.to_ascii_uppercase())
}

let mut slot: Option<String> = Some("job".into());
let job = slot.take();                      // slot = None, job = Some("job")
let title: Option<String> = Some("Dr".into());
let t: Option<&str> = title.as_deref();     // Option<String> → Option<&str>
let filled: &mut String = slot.get_or_insert_with(|| "new".into());   // "new"
```

CODE EXPLAINED

- `match` forces both cases to be handled
- `unwrap_or_else` only runs the closure when needed
- `and_then` chains steps that also return Option
- `take()` moves the value out and leaves `None`

COMING FROM…

- **Java** `Optional<T>` with `map`, `orElse`, `orElseGet`
- **Kotlin / Swift** nullable types `T?` with `?.` and `?:`
- **Python** `None` with no compile-time checks; `Optional[T]` hints only

THE RUST WAY

- `if x.is_some() { x.unwrap() }` → `if let Some(v) = x`
- `.map(…).unwrap_or(…)` → `.map_or(default, f)`

GET THE VALUE

METHOD	IF NONE
`unwrap()`	Panic
`expect(msg)`	Panic with msg
`unwrap_or(v)`	v
`unwrap_or_else(f)`	f()
`unwrap_or_default()`	Default
`?`	Return None

TRANSFORM

METHOD	DOES
`map(f)`	Some(f(x))
`and_then(f)`	f returns Option
`filter(p)`	Keep if p
`or / or_else`	Fallback Option
`ok_or(e)`	→ Result
`zip(other)`	Some((a, b))
`take() / replace(v)`	Move out

⚠ COMMON MISTAKES

- `unwrap()` everywhere
- `unwrap_or(expensive())` evaluating eagerly (use `_else`)
- Matching `&Option<String>` and moving the String out

✓ TIPS & BEST PRACTICES

- Return Option from lookups, Result from failures
- Use `as_deref()` for `Option<String>` → `Option<&str>`
- Use `let … else` to unwrap at the top of functions

QUICK SUMMARY

- no null: `Option<T>`
- `Some` / `None`
- `map` / `and_then` / `unwrap_or`
- `?` returns None early

Result<T, E> & the ? Operator

RECOVERABLE ERRORS AS VALUES

Functions that can fail return `Result<T, E>`: `Ok(value)` or `Err(error)`. The **? operator** returns the error to the caller if there is one and unwraps the value otherwise, so error handling stays one character per call. `Box<dyn Error>` is a quick catch-all error type for applications.

KEY POINTS

- `Result` is `#[must_use]`: ignoring one warns
- `?` converts the error with `From::from`
- `?` only works in functions returning Result/Option
- `map_err` changes the error type

UNDER THE HOOD

- `expr?` desugars to `match Try::branch(expr) { Continue(v) => v, Break(r) => return FromResidual::from_residual(r) }`
- For Result, the residual path calls `From::from(err)`
- No exceptions or unwinding: an Err is an ordinary return value
- `Result<T, E>` size ≈ max(T, E) + tag

RESULT AND ?

```
use std::fs;
use std::num::ParseIntError;

fn parse_double(s: &str) -> Result<i32, ParseIntError> {
    let n: i32 = s.trim().parse()?;          // Err → return early
    Ok(n * 2)
}
println!("{:?}", parse_double("21"));        // Ok(42)
println!("{:?}", parse_double("x"));
                              // Err(ParseIntError { kind: InvalidDigit })

fn read_port(path: &str) -> Result<u16, String> {
    let text = fs::read_to_string(path).map_err(|e| format!("{path}: {e}"))?;
    text.trim().parse::<u16>().map_err(|e| e.to_string())
}

fn load() -> Result<u16, Box<dyn std::error::Error>> {
    let text = fs::read_to_string("port.txt")?;    // io::Error → Box
    let port = text.trim().parse::<u16>()?;        // ParseIntError → Box
    Ok(port)
}

let maybe: Option<i32> = parse_double("5").ok();   // Result → Option
let total: Result<Vec<i32>, _> =
    ["1", "2", "x"].iter().map(|s| s.parse::<i32>()).collect();
println!("{}", total.is_err());                // true: stops at first Err
let n2 = parse_double("8").inspect_err(|e| eprintln!("{e}")).unwrap_or(0);

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let port = load()?;                        // error printed, exit code 1
    println!("listening on {port}");
    Ok(())
}
```

CODE EXPLAINED

- → `?` unwraps Ok or returns the Err immediately
- → `map_err` adds context or converts the error type
- → `Box<dyn Error>` accepts any error via `From`
- → Collecting into `Result<Vec<_>, _>` stops at the first error

COMING FROM…

- ↔ **Go** `if err != nil { return err }` (? is the one-character version)
- ↔ **Java** checked exceptions with `throws`; Result is explicit in the type
- ↔ **Python** `try` / `except`; errors are invisible in signatures

THE RUST WAY

- ⇄ `match r { Ok(v) => v, Err(e) => return Err(e.into()) }` → `r?`
- ⇄ `.unwrap()` in library code → return `Result`

METHODS

METHOD	DOES
`?`	Propagate Err
`map(f)`	Transform Ok
`map_err(f)`	Transform Err
`and_then(f)`	Chain fallible steps
`unwrap_or / _else / _default`	Fallback
`ok() / err()`	→ Option
`is_ok() / is_err()`	bool
`inspect_err(f)`	Log, pass through

ERROR TYPE CHOICE

CONTEXT	TYPE
`Quick app / script`	Box<dyn Error>
`Application`	anyhow::Result
`Library`	Custom enum / thiserror
`Simple messages`	String (prototypes)

⚠ COMMON MISTAKES

- Using `?` in a function returning `()`
- Error types that do not convert (implement `From`)
- Ignoring a `Result` with `let _ =` without a reason

✓ TIPS & BEST PRACTICES

- Add context with `map_err` or anyhow's `.context()`
- Make `main` return `Result` in CLI tools
- Collect into `Result<Vec<T>, E>` to validate a batch

QUICK SUMMARY

- `Ok` / `Err`
- `?` — propagate
- `map_err` — convert
- `Box<dyn Error>` — catch-all

Custom Error Types

error enums · `impl Display` · `impl std::error::Error` · `source()` · `From` for `?` conversion · `#[non_exhaustive]`

DESIGNING YOUR OWN ERRORS

Libraries usually define an **error enum** with one variant per failure kind, so callers can `match` on it. Implement `Display` for a human message, `std::error::Error` for interoperability, and `From<OtherError>` so `?` converts underlying errors automatically. `source()` exposes the wrapped cause.

KEY POINTS

- Variants can carry data: `NotFound(String)`
- `Error` requires `Debug + Display`
- `From` impls make `?` work across error types
- `#[non_exhaustive]` lets you add variants later

AN ERROR ENUM

```rust
use std::{error::Error, fmt, fs, io, num::ParseIntError};

#[derive(Debug)]
#[non_exhaustive]
pub enum ConfigError {
    Io(io::Error),
    BadPort(ParseIntError),
}

impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            Self::Io(e) => write!(f, "could not read config: {e}"),
            Self::BadPort(e) => write!(f, "invalid port: {e}"),
        }
    }
}
impl Error for ConfigError {
    fn source(&self) -> Option<&(dyn Error + 'static)> {
        match self { Self::Io(e) => Some(e), Self::BadPort(e) => Some(e) }
    }
}
impl From<io::Error> for ConfigError {
    fn from(e: io::Error) -> Self { Self::Io(e) }
}
impl From<ParseIntError> for ConfigError {
    fn from(e: ParseIntError) -> Self { Self::BadPort(e) }
}

fn load_port(path: &str) -> Result<u16, ConfigError> {
    let text = fs::read_to_string(path)?;   // io::Error → ConfigError
    Ok(text.trim().parse()?)                // ParseIntError → ConfigError
}

match load_port("port.txt") {
    Err(ConfigError::BadPort(e)) => eprintln!("fix the port: {e}"),
    Err(e) => eprintln!("{e}"),
    Ok(p) => println!("port {p}"),
}
```

CODE EXPLAINED

→ Each variant names one way loading can fail

→ `Display` gives the message users see

→ `source()` lets reporters print the cause chain

→ `From` impls let `?` convert io and parse errors

COMING FROM…

↔ **Java** custom exception classes with a cause

↔ **Python** exception subclasses; `raise … from e` sets the cause

↔ **Go** sentinel errors + `errors.Is` / `errors.As`, `%w` wrapping

THE RUST WAY

⇄ writing Display/From by hand → `thiserror` (sheet 045)

⇄ `String` errors in a library → an enum callers can match

PIECES

IMPL	PURPOSE
`#[derive(Debug)]`	Required by Error
`Display`	User-facing message
`Error`	Standard interface
`source()`	Underlying cause
`From<E>`	Enables ?
`#[non_exhaustive]`	Future variants

⚠ COMMON MISTAKES

- Display messages that repeat the source message (shown twice in chains)
- One giant error enum for the whole crate
- Forgetting `From`, then writing `map_err` everywhere

✓ TIPS & BEST PRACTICES

- Keep messages lowercase without trailing periods (std style)
- Group errors by module or operation
- Expose the source instead of formatting it into the message

QUICK SUMMARY

- error enum per failure kind
- `Display` + `Error`
- `source()` — cause chain
- `From` — powers `?`

Sheet 045

thiserror & anyhow

#[derive(Error)] · #[error("…")] · #[from] · anyhow::Result · .context() / with_context · bail! / ensure! · library vs application rule

THE TWO ERROR CRATES

thiserror derives `Display`, `Error` and `From` for your error enums, ideal for **libraries** whose callers need to match on errors. **anyhow** gives applications one easy error type, `anyhow::Error`, that wraps anything and adds **context** messages. The common rule: thiserror in libraries, anyhow in binaries.

KEY POINTS

- `cargo add thiserror anyhow`
- `#[from]` generates the `From` impl and sets `source`
- `.context("…")` wraps an error with a message
- `{:#}` prints the full chain on one line; `{:?}` adds a backtrace

THISERROR AND ANYHOW

```rust
// ---- library: thiserror ----
use thiserror::Error;

#[derive(Debug, Error)]
pub enum StoreError {
    #[error("item {0} not found")]
    NotFound(u64),
    #[error("database error")]
    Db(#[from] std::io::Error),
    #[error("invalid name {name:?}: {reason}")]
    Invalid { name: String, reason: &'static str },
}
pub fn get(id: u64) -> Result<String, StoreError> {
    if id == 0 { return Err(StoreError::NotFound(id)); }
    let data = std::fs::read_to_string("store.db")?;   // #[from] converts
    Ok(data)
}

// ---- application: anyhow ----
use anyhow::{bail, ensure, Context, Result};

fn load_config(path: &str) -> Result<u16> {
    let text = std::fs::read_to_string(path)
        .with_context(|| format!("reading {path}"))?;
    let port: u16 = text.trim().parse().context("port is not a number")?;
    ensure!(port >= 1024, "port {port} is privileged");
    if port == 9999 { bail!("port 9999 is reserved"); }
    Ok(port)
}

fn main() -> Result<()> {
    let port = load_config("port.txt")?;
    let item = get(7).context("loading item 7")?;
    println!("{port} {item}");
    Ok(())
}
```

CODE EXPLAINED

- `#[error("…")]` writes the Display impl; `{0}` and `{name}` use fields
- `#[from]` adds `From<io::Error>` and `source()`
- `with_context` builds the message only on failure
- `bail!` returns an error; `ensure!` is a checked assert

COMING FROM…

- **Java** exception wrapping: `throw new X("msg", cause)` ≈ `.context()`
- **Go** `fmt.Errorf("reading %s: %w", path, err)`
- **Python** `raise ConfigError("msg") from e`

THE RUST WAY

- `Box<dyn Error>` in apps → `anyhow::Result`
- `anyhow::Error` in a public library API → thiserror enum

THISERROR

ATTRIBUTE	EFFECT
`#[derive(Error)]`	Implements Error
`#[error("…")]`	Display
`#[from]`	From + source
`#[source]`	source only
`#[error(transparent)]`	Forward Display/source

ANYHOW

ITEM	USE
`anyhow::Result<T>`	Result<T, anyhow::Error>
`.context(msg)`	Add message
`.with_context(|| …)`	Lazy message
`bail!(…)`	Return Err
`ensure!(cond, …)`	Err if false
`anyhow!(…)`	Make an error
`e.downcast_ref::<T>()`	Inspect cause

⚠ COMMON MISTAKES

- Returning `anyhow::Error` from a library
- Context messages that restate the cause instead of the action
- Using `context(format!(…))` (allocates even on success)

✓ TIPS & BEST PRACTICES

- Describe what you were doing: "reading config.toml"
- Print errors with `{:#}` for one-line chains in logs
- Use `#[error(transparent)]` to wrap another crate's error

QUICK SUMMARY

- thiserror — libraries
- anyhow — applications
- `#[from]` / `.context()`
- `bail!` / `ensure!`

Sheet 046

Defining Functions

`fn name(x: i32) -> i32` · implicit return · early `return` · unit return · snake_case · functions are items (order doesn't matter)

FUNCTIONS

Functions are declared with `fn`, and every parameter and return type must be written out. The last expression without a semicolon is the return value; use `return` only for early exits. A function with no `->` returns `()`. Functions can be declared in any order and even nested inside other functions.

KEY POINTS

- Names are snake_case; types are CamelCase
- Parameter types are never inferred
- `-> !` means the function never returns
- Nested fns cannot capture local variables (use closures)

FUNCTIONS

```
fn main() {
    greet("Ada");                       // defined below: order doesn't matter
    println!("{}", add(2, 3));          // 5
    println!("{}", classify(-4));       // negative
    println!("{:?}", divide(7, 0));     // None
    println!("{}", hypot(3.0, 4.0));    // 5

    fn helper(x: i32) -> i32 { x * 10 }   // nested function
    println!("{}", helper(4));          // 40
}

fn greet(name: &str) {                  // returns ()
    println!("Hello, {name}!");
}

fn add(a: i32, b: i32) -> i32 {
    a + b                               // implicit return
}

fn classify(n: i32) -> &'static str {
    if n < 0 {
        return "negative";              // early return
    }
    if n == 0 { "zero" } else { "positive" }
}

fn divide(a: i32, b: i32) -> Option<i32> {
    if b == 0 { return None; }
    Some(a / b)
}

fn hypot(x: f64, y: f64) -> f64 { (x * x + y * y).sqrt() }

fn min_max(v: &[i32]) -> (i32, i32) {   // return several values as a tuple
    (*v.iter().min().unwrap(), *v.iter().max().unwrap())
}
```

CODE EXPLAINED

- → `main` calls functions declared later in the file
- → The last expression `a + b` is returned
- → `return` exits early from inside an `if`
- → Return a tuple or struct for several values

COMING FROM…

- ↔ **Go** `func add(a, b int) int` with explicit `return`
- ↔ **Python** `def add(a, b):` with optional type hints
- ↔ **Java** methods live in classes; Rust functions can be free-standing

THE RUST WAY

- ⇄ `return x;` as the last line → just `x`
- ⇄ many related free functions → methods in an `impl` block

SIGNATURE PARTS

PART	EXAMPLE
Name	fn parse_line
Generics	fn f<T>
Params	(a: i32, s: &str)
Return	-> Result<u8, E>
No return	(implicit ())
Never returns	-> !
Where clause	where T: Display

⚠ COMMON MISTAKES

- A trailing `;` on the return expression ("expected i32, found ()")
- Omitting parameter types
- Very long functions instead of small helpers

✓ TIPS & BEST PRACTICES

- Keep functions short and named for what they return
- Return `Option` / `Result` instead of sentinel values
- Use `#[must_use]` on functions whose result matters

QUICK SUMMARY

- `fn name(p: T) -> R`
- last expression returns
- `return` for early exit
- any declaration order

Sheet 047

Parameters & Arguments

by value (move) · &T / &mut T params · mut bindings · impl Trait params · no default args or overloading · Option / builder instead

HOW ARGUMENTS ARE PASSED

The parameter type decides ownership: T takes ownership (move or copy), &T borrows for reading and &mut T borrows for changing. `mut x: T` only makes the local binding mutable. Rust has **no default arguments and no overloading**; use `Option` parameters, separate function names, traits or a builder.

KEY POINTS

- The caller writes `&x` / `&mut x` explicitly
- `impl Trait` accepts any type implementing the trait
- `impl Into<String>` accepts both `&str` and `String`
- Patterns can destructure parameters

PARAMETER KINDS

```
fn consume(v: Vec<i32>) -> usize { v.len() }    // takes ownership
fn inspect(v: &[i32]) -> i32 { v.iter().sum() } // borrows
fn extend(v: &mut Vec<i32>) { v.push(99); }     // mutable borrow
fn countdown(mut n: u32) { while n > 0 { n -= 1; } }   // local mut

let mut data = vec![1, 2, 3];
println!("{}", inspect(&data));             // 6
extend(&mut data);
println!("{}", consume(data));              // 4; data moved

fn show(item: impl std::fmt::Display) { println!("[{item}]"); }
show(42);
show("text");

struct User { name: String }
fn new_user(name: impl Into<String>) -> User { User { name: name.into() } }
let a = new_user("ada");                    // &str
let b = new_user(String::from("bob"));      // String

// No defaults: use Option…
fn connect(host: &str, port: Option<u16>) -> String {
    format!("{host}:{}", port.unwrap_or(80))
}
println!("{}", connect("example.com", None));   // example.com:80

// …or separate names instead of overloading
fn area_square(s: f64) -> f64 { s * s }
fn area_rect(w: f64, h: f64) -> f64 { w * h }

fn print_point((x, y): (i32, i32)) { println!("({x}, {y})"); }
print_point((3, 4));
```

CODE EXPLAINED

- → `consume` moves `data`; it cannot be used afterwards
- → `&mut data` is visible at the call site
- → `impl Display` takes integers and strings
- → `impl Into<String>` avoids forcing callers to allocate

COMING FROM…

- ↔ **C++** default arguments and overloading are both allowed
- ↔ **Python** keyword and default arguments: `def connect(host, port=80)`
- ↔ **Java** overloading; no defaults (same workaround: builders)

THE RUST WAY

- ⇄ many `bool` parameters → an options struct or enum
- ⇄ boolean flag parameter `f(true)` → two functions or an enum

PARAMETER TYPES

TYPE	CALLER PASSES	CALLEE CAN
`T`	`x (moves)`	`Own, drop`
`&T`	`&x`	`Read`
`&mut T`	`&mut x`	`Modify`
`mut x: T`	`x`	`Modify local copy`
`impl Trait`	`Any impl`	`Use trait methods`
`impl Into<T>`	`Convertible`	`.into()`

⚠ COMMON MISTAKES

- Taking `String` when `&str` is enough
- Taking `&mut` when only reading
- Trying to overload by type (E0428 duplicate definition)

✓ TIPS & BEST PRACTICES

- Choose the weakest parameter: `&T` over `&mut T` over `T`
- Use builders for 4+ optional settings (sheet 106)
- Use `impl AsRef<Path>` for path parameters

QUICK SUMMARY

- `T` moves · `&T` reads · `&mut T` edits
- no defaults / overloading
- `Option` params / builders
- `impl Into<String>`

Closures

`|x| x + 1` · type inference · capture by ref / mut ref / value · `move` closures · closures in variables · closures in iterators

ANONYMOUS FUNCTIONS THAT CAPTURE

A closure is an anonymous function written `|args| body` that can **capture** variables from its surroundings. The compiler picks the least intrusive capture: borrow, mutable borrow, or move. The `move` keyword forces captured values to move into the closure, which is required for threads and returning closures.

KEY POINTS

- Parameter and return types are usually inferred
- Once inferred, a closure's types are fixed
- Each closure has its own unique, unnamed type
- Edition 2021 captures individual fields, not whole structs

UNDER THE HOOD

- A closure compiles to an anonymous struct holding its captures
- Calling it is a direct call to a method on that struct (inlinable)
- A closure with no captures is zero-sized and coerces to a `fn` pointer
- `move` changes captures from references to owned fields

CLOSURES

```
let add_one = |x: i32| x + 1;
let square = |x| x * x;                    // inferred from first use
println!("{} {}", add_one(4), square(5));   // 5 25

let factor = 3;
let scale = |x: i32| x * factor;           // borrows factor
println!("{}", scale(10));                 // 30

let mut count = 0;
let mut inc = || count += 1;               // mutable borrow
inc();
inc();
println!("{count}");                       // 2

let name = String::from("Ada");
let greet = move || println!("hi {name}"); // name moved in
greet();
// println!("{name}");                     // E0382: moved into closure

let nums = vec![1, 2, 3, 4, 5, 6];
let threshold = 3;
let big: Vec<i32> = nums.iter().copied().filter(|&n| n > threshold).collect();
let mut sorted = vec!["bb", "a", "ccc"];
sorted.sort_by_key(|s| s.len());           // ["a", "bb", "ccc"]

let handle = std::thread::spawn(move || nums.len());   // move required
println!("{}", handle.join().unwrap());    // 6

let multi_line = |a: i32, b: i32| -> i32 {
    let s = a + b;
    s * 2
};
println!("{}", multi_line(1, 2));          // 6
```

CODE EXPLAINED

- → `|x| x * x` gets its type from the first call
- → `scale` borrows `factor`; `inc` borrows `count` mutably
- → `move` transfers `name` into the closure
- → Threads require `move` because they may outlive the scope

COMING FROM…

- ↔ **C++** lambdas with explicit capture lists `[&]`, `[=]`, `[x = std::move(x)]`
- ↔ **Java / JavaScript** lambdas / arrow functions capture effectively-final or any variables
- ↔ **Python** `lambda x: x + 1` (single expression only)

THE RUST WAY

- ⇄ `.map(|x| foo(x))` → `.map(foo)`
- ⇄ `clone()` before a `move` closure only when the original is still needed

CAPTURE MODES

USE INSIDE	CAPTURE	TRAIT
Read only	&T	Fn
Mutate	&mut T	FnMut
Move out / consume	T	FnOnce
move keyword	T (all)	Depends on use

SYNTAX

FORM	EXAMPLE
No args	|| 42
Typed	|x: i32| x + 1
Return type	|x| -> i32 { x }
Block body	|x| { … }
Pattern arg	|&(a, b)| a + b
Move	move || …

⚠ COMMON MISTAKES

- Calling a mutating closure without declaring it `mut`
- Using a variable after moving it into a `move` closure
- Calling one closure with two different argument types

✓ TIPS & BEST PRACTICES

- Pass function names directly when no capture is needed
- Clone into a new binding just before `move` to keep the original
- Keep closures short; name complex ones as functions

QUICK SUMMARY

- `|args| body`
- captures by ref, mut ref or value
- `move` — take ownership
- unique type per closure

Sheet 049

Fn, FnMut & FnOnce

Fn · FnMut · FnOnce · how captures pick the trait · `F: Fn(i32) -> i32` · `where` clauses · `fn` pointers vs closures

THE THREE CLOSURE TRAITS

Every closure implements one or more of three traits. **FnOnce** can be called once (it may consume captures), **FnMut** can be called many times and may mutate captures, and **Fn** can be called many times through a shared reference. Functions that accept closures state which trait they need; ask for the weakest one you can.

KEY POINTS

- Every Fn is also FnMut, and every FnMut is FnOnce
- Accept `FnOnce` if you call it once (most flexible for callers)
- `fn(i32) -> i32` is a function pointer type
- Plain functions implement all three traits

ACCEPTING CLOSURES

```
fn apply<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 { f(x) }

fn repeat<F>(n: usize, mut f: F)
where
    F: FnMut(usize),
{
    for i in 0..n { f(i); }
}

fn run_once<F: FnOnce() -> String>(f: F) -> String { f() }

println!("{}", apply(|x| x * 2, 21));        // 42

let mut log = Vec::new();
repeat(3, |i| log.push(i));                  // FnMut: mutates log
println!("{log:?}");                         // [0, 1, 2]

let name = String::from("Ada");
let msg = run_once(move || name + "!");      // FnOnce: consumes name
println!("{msg}");                           // Ada!

fn double(x: i32) -> i32 { x * 2 }
println!("{}", apply(double, 5));            // functions work too: 10

let ops: Vec<fn(i32) -> i32> = vec![double, |x| x + 1];   // no captures
println!("{}", ops.iter().fold(3, |acc, f| f(acc)));      // 7

let consume = move || drop(log);             // FnOnce only
consume();
// consume();                          // E0382: closure moved by first call

fn make_counter() -> impl FnMut() -> u32 {
    let mut c = 0;
    move || { c += 1; c }
}
let mut next = make_counter();
println!("{} {}", next(), next());           // 1 2
```

CODE EXPLAINED

- → `apply` needs `Fn`: it only reads its captures
- → `repeat` needs `FnMut` because the closure pushes to `log`
- → `run_once` accepts closures that consume what they capture
- → Closures without captures coerce to `fn` pointers

COMING FROM…

- ↔ **C++** `std::function<int(int)>` or a template parameter
- ↔ **Java** `Function<T,R>`, `Consumer<T>`, `Supplier<T>`
- ↔ **Python** any callable; no compile-time distinction

THE RUST WAY

- ⇄ `F: Fn()` for a callback called once → `F: FnOnce()`
- ⇄ long inline bounds → `where` clause

TRAITS

TRAIT	CALL VIA	TIMES	CAPTURES
FnOnce	self	Once	May consume
FnMut	&mut self	Many	May mutate
Fn	&self	Many	Read only
fn ptr	value	Many	None

⚠ COMMON MISTAKES

- Requiring `Fn` when callers need to mutate state
- Forgetting `mut f: F` to call an FnMut
- Expecting a capturing closure to coerce to `fn`

✓ TIPS & BEST PRACTICES

- Accept `impl FnOnce` for one-shot callbacks
- Use `where` clauses for readability
- Use `fn` pointers for tables of plain functions

QUICK SUMMARY

- FnOnce ⊇ FnMut ⊇ Fn
- captures decide the trait
- ask for the weakest
- `fn` — pointer, no captures

Sheet 050

Higher-Order Functions

closures as arguments · returning `impl Fn` · `Box<dyn Fn>` · `fn(i32) -> i32` pointers · composing functions · callbacks

FUNCTIONS THAT TAKE OR RETURN FUNCTIONS

Pass closures with generic parameters (`impl Fn`) for zero-cost static dispatch. Return a single closure type with `impl Fn`. When you need to store **different** closures together or choose one at runtime, box them as `Box<dyn Fn>`, paying for a heap allocation and an indirect call.

KEY POINTS

- `impl Fn` in return position = one concrete hidden type
- Two different closures need `Box<dyn Fn>` to share a type
- Returned closures usually need `move`
- Iterators are the most common higher-order API

PASSING, RETURNING, STORING

```
fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n                          // captures n by value
}
let add5 = make_adder(5);
println!("{}", add5(10));                   // 15

fn compose<A, B, C>(f: impl Fn(A) -> B, g: impl Fn(B) -> C) -> impl Fn(A) -> C {
    move |x| g(f(x))
}
let inc_then_double = compose(|x: i32| x + 1, |x| x * 2);
println!("{}", inc_then_double(4));         // 10

fn op(name: &str) -> Box<dyn Fn(f64, f64) -> f64> {
    match name {
        "add" => Box::new(|a, b| a + b),
        "pow" => Box::new(f64::powf),
        _ => Box::new(|_, _| f64::NAN),
    }
}
println!("{}", op("pow")(2.0, 10.0));       // 1024

struct Button {
    label: String,
    on_click: Vec<Box<dyn Fn(&str)>>,       // list of callbacks
}
impl Button {
    fn click(&self) { for cb in &self.on_click { cb(&self.label); } }
}
let b = Button {
    label: "OK".into(),
    on_click: vec![Box::new(|l| println!("clicked {l}")),
                   Box::new(|l| println!("log: {l}"))],
};
b.click();

let pipeline: [fn(i32) -> i32; 3] = [|x| x + 1, |x| x * 10, |x| x - 3];
println!("{}", pipeline.iter().fold(1, |v, f| f(v)));   // 17
```

CODE EXPLAINED

- → `make_adder` returns a closure that remembers `n`
- → `compose` combines two functions into one
- → `op` returns different closures, so it boxes them
- → A `Vec<Box<dyn Fn>>` stores callbacks of different types

COMING FROM…

- ↔ **JavaScript** functions are first-class values; no boxing decisions
- ↔ **Python** decorators and `functools.partial`
- ↔ **C++** `std::function` ≈ `Box<dyn Fn>`; templates ≈ `impl Fn`

THE RUST WAY

- ⇄ `Box<dyn Fn>` parameters → `impl Fn` (no allocation)
- ⇄ returning `Box<dyn Fn>` for a single closure → `impl Fn`

CHOOSING A FORM

FORM	DISPATCH	USE
`impl Fn (param)`	Static	Most callbacks
`impl Fn (return)`	Static	One closure type
`Box<dyn Fn>`	Dynamic	Mixed closures, storage
`&dyn Fn`	Dynamic	Borrowed, no alloc
`fn(T) -> U`	Pointer	No captures

⚠ COMMON MISTAKES

- Returning a closure without `move` (borrows a local: E0373)
- Trying to return two different closures as `impl Fn`
- Boxing closures when generics would do

✓ TIPS & BEST PRACTICES

- Add `+ Send + 'static` to boxed callbacks used across threads
- Prefer iterator adapters over hand-written HOFs
- Use `&dyn Fn` to avoid generics bloat without allocating

QUICK SUMMARY

- `impl Fn` — static, zero-cost
- `Box<dyn Fn>` — mixed / stored
- `move` for returned closures
- compose small functions

Sheet 051

const fn, Recursion & Attributes

const fn · recursion & stack depth · no guaranteed tail calls · #[inline] · #[must_use] · diverging -> !

COMPILE-TIME AND SPECIAL FUNCTIONS

A `const fn` can run at compile time to initialize constants, and also at runtime like any function. Recursion works, but Rust does **not guarantee tail-call optimization**, so deep recursion can overflow the stack; iterate instead. Attributes like `#[inline]` and `#[must_use]` give hints and warnings.

KEY POINTS

- `const fn` bodies are limited (no heap, no traits calls in most cases)
- Main thread stack is usually 8 MB; spawned threads default to 2 MB
- `#[must_use]` warns when a result is ignored
- `#[inline]` matters mostly across crates

CONST FN, RECURSION, ATTRIBUTES

```
const fn kib(n: usize) -> usize { n * 1024 }
const BUF: usize = kib(64);                 // computed at compile time

const fn fib_const(n: u32) -> u64 {
    let (mut a, mut b, mut i) = (0, 1, 0);
    while i < n { let t = a + b; a = b; b = t; i += 1; }
    a
}
const FIB_50: u64 = fib_const(50);          // 12586269025

fn factorial(n: u64) -> u64 {
    if n <= 1 { 1 } else { n * factorial(n - 1) }   // recursion
}

fn sum_to(n: u64) -> u64 { (1..=n).sum() }  // iteration: no stack risk
// fn deep(n: u64) -> u64 { if n == 0 { 0 } else { 1 + deep(n - 1) } }
// deep(10_000_000)                         // stack overflow

fn depth(tree: &Node) -> usize {
    1 + tree.children.iter().map(depth).max().unwrap_or(0)
}
struct Node { children: Vec<Node> }

#[must_use = "the new string is returned; the input is unchanged"]
fn shout(s: &str) -> String { s.to_uppercase() }
// shout("hi");                             // warning: unused return value
#[inline]
fn square(x: i32) -> i32 { x * x }
#[cold]
fn report(msg: &str) -> ! {                 // rarely called, never returns
    eprintln!("fatal: {msg}");
    std::process::exit(1)
}

println!("{} {}", BUF, FIB_50);             // 65536 12586269025
println!("{}", factorial(20));              // 2432902008176640000
```

CODE EXPLAINED

- → `kib(64)` runs during compilation
- → `const fn` can use loops but not heap allocation
- → Recursion suits trees; use loops for long linear work
- → `-> !` tells the compiler `report` never returns

COMING FROM…

- ↔ **C++** `constexpr` / `consteval` functions
- ↔ **Python** recursion limit of 1000 by default
- ↔ **Go / Java** no compile-time function execution in the language

THE RUST WAY

- ⇄ deep linear recursion → a loop or an explicit stack
- ⇄ `#[inline(always)]` everywhere → measure first

ATTRIBUTES

ATTRIBUTE	EFFECT
`#[inline]`	Allow cross-crate inlining
`#[inline(always)] / (never)`	Strong hint
`#[cold]`	Unlikely path
`#[must_use]`	Warn if ignored
`#[track_caller]`	Panic points to caller
`#[deprecated]`	Warn on use
`#[doc(hidden)]`	Hide from docs

⚠ COMMON MISTAKES

- Assuming tail recursion will be optimized
- Calling non-const functions inside `const fn`
- Ignoring `#[must_use]` warnings on Results

✓ TIPS & BEST PRACTICES

- Use `stacker` or a bigger thread stack for deep trees if needed
- Put `#[must_use]` on builders and pure functions
- Use `#[track_caller]` on helpers that panic

QUICK SUMMARY

- `const fn` — compile-time capable
- no guaranteed TCO
- `#[must_use]` / `#[inline]`
- `-> !` diverges

Arrays

`[i32; 5]` · `[0; 100]` · fixed size on the stack · `len()` · arrays vs slices vs `Vec` · `array::map` · 2-D arrays

FIXED-SIZE SEQUENCES

An array `[T; N]` holds exactly N values of one type, and N is part of the type: `[i32; 3]` and `[i32; 4]` are different types. Arrays live inline (usually on the stack) with no heap allocation. Indexing is bounds-checked. Use a `Vec` when the length is not known at compile time.

KEY POINTS

- `[value; N]` repeats a value N times (needs Copy or a const)
- Arrays coerce to slices: `&arr` → `&[T]`
- Arrays implement Debug, PartialEq, Ord, Hash, Default
- Arrays iterate by value with `for x in arr` (2021+)

UNDER THE HOOD

- `[T; N]` is N values laid out back to back: `size_of = N * size_of::<T>()`
- No length is stored; N is known at compile time
- Constant-index accesses are bounds-checked at compile time
- Large arrays on the stack can overflow it; use `vec!` or `Box<[T; N]>`

ARRAYS

```
let primes: [u32; 5] = [2, 3, 5, 7, 11];
let zeros = [0u8; 100];                      // 100 zero bytes
let names = ["Ada", "Bob", "Cy"];            // [&str; 3]

println!("{}", primes[0]);                   // 2
println!("{}", primes.len());                // 5
println!("{:?}", primes.get(9));             // None
// let x = primes[9];                 // compile error: index out of bounds

let doubled = primes.map(|p| p * 2);         // [4, 6, 10, 14, 22]
let total: u32 = primes.iter().sum();        // 28
println!("{}", primes.contains(&7));         // true

let mut grid = [[0; 3]; 2];                  // 2 rows × 3 columns
grid[1][2] = 9;
println!("{grid:?}");                        // [[0, 0, 0], [0, 0, 9]]

fn sum(xs: &[u32]) -> u32 { xs.iter().sum() }
println!("{}", sum(&primes));                // array → slice

let [first, second, ..] = primes;            // destructure
let squares: [usize; 4] = std::array::from_fn(|i| i * i);   // [0, 1, 4, 9]

println!("{}", std::mem::size_of_val(&zeros));   // 100
let v: Vec<u32> = primes.to_vec();           // copy to the heap
let back: [u32; 5] = v.try_into().unwrap(); // Vec → array (length checked)
let mut sorted = [3, 1, 2];
sorted.sort();
for p in primes { print!("{p} "); }          // by value
```

CODE EXPLAINED

- → The length 5 is part of the type `[u32; 5]`
- → `get` returns an Option instead of panicking
- → `[[0; 3]; 2]` builds a 2-D array
- → `from_fn` builds an array from an index function

COMING FROM…

- ↔ **C / C++** `int a[5]` / `std::array<int, 5>` (Rust adds bounds checks)
- ↔ **Go** `[5]int` arrays are values, like Rust's
- ↔ **Java / Python** arrays/lists are heap objects with runtime length

THE RUST WAY

- ⇄ `Vec` for a fixed set of values → array `[T; N]`
- ⇄ `for i in 0..arr.len()` → `for x in arr` / `arr.iter()`

COMPARE

ITEM	ARRAY	SLICE	VEC
Type	`[T; N]`	`&[T]`	`Vec<T>`
Size	Compile time	Runtime	Runtime, grows
Owns	Yes	No	Yes
Storage	Inline/stack	Borrowed	Heap
Resize	No	No	Yes

⚠ COMMON MISTAKES

- Using a runtime value as N (`[0; n]` needs a const)
- Huge stack arrays causing stack overflow
- Expecting arrays of different lengths to be the same type

✓ TIPS & BEST PRACTICES

- Use const generics for functions over any N: `fn f<const N: usize>(a: [T; N])`
- Use `array::from_fn` for computed arrays
- Use `chunks_exact` to view a slice as fixed-size pieces

QUICK SUMMARY

- `[T; N]` — fixed length
- `[v; N]` — repeat
- inline, no heap
- coerces to `&[T]`

Vec — Basics

GROWABLE ARRAYS

`Vec<T>` is Rust's growable array: contiguous elements on the heap, with fast indexing and amortized O(1) `push` at the end. `v[i]` panics when out of range, while `v.get(i)` returns `Option<&T>`. When the Vec runs out of room it reallocates, usually doubling its capacity.

KEY POINTS

- `vec![1, 2, 3]` and `vec![0; n]` are the usual constructors
- `pop` returns `Option<T>`
- A Vec derefs to a slice, so all slice methods work
- Reallocation invalidates references (the borrow checker prevents this)

UNDER THE HOOD

- A Vec is three words on the stack: pointer, capacity, length (24 bytes)
- `Vec::new()` does not allocate until the first push
- Growth is amortized: capacity roughly doubles on reallocation
- Dropping a Vec drops each element, then frees the buffer

VEC BASICS

```
let mut v: Vec<i32> = Vec::new();           // no allocation yet
v.push(10);
v.push(20);
v.push(30);
let w = vec![1, 2, 3];
let zeros = vec![0.0; 5];                   // [0.0, 0.0, 0.0, 0.0, 0.0]

println!("{}", v[1]);                       // 20
println!("{:?}", v.get(1));                 // Some(20)
println!("{:?}", v.get(99));                // None
// println!("{}", v[99]);                   // panic: index out of bounds

println!("{:?}", v.pop());                  // Some(30)
println!("{} {}", v.len(), v.is_empty());   // 2 false
println!("{:?} {:?}", v.first(), v.last()); // Some(10) Some(20)

if let Some(x) = v.get_mut(0) { *x += 1; }  // v = [11, 20]
v[1] = 99;                                  // v = [11, 99]

let mut big = Vec::with_capacity(1000);     // one allocation
for i in 0..1000 { big.push(i); }
println!("{} {}", big.len(), big.capacity());   // 1000 1000

let mut grow = Vec::new();
for i in 0..5 {
    grow.push(i);
    print!("{} ", grow.capacity());         // 4 4 4 4 8 (implementation detail)
}
println!();
grow.shrink_to_fit();
let from_iter: Vec<char> = "abc".chars().collect();
let nested: Vec<Vec<u8>> = vec![vec![0; 3]; 2];
```

CODE EXPLAINED

- → `Vec::new()` waits to allocate until the first `push`
- → `get` returns `Option<&T>`; `[]` panics
- → `get_mut` gives `Option<&mut T>` for safe edits
- → `with_capacity` reserves space up front

COMING FROM…

- ↔ **C++** `std::vector` (same layout: ptr, size, capacity)
- ↔ **Java** `ArrayList<Integer>` (boxed elements unless primitive arrays)
- ↔ **Python / Go** `list` / slices with `append`

THE RUST WAY

- ⇄ `v[i]` with an index from user input → `v.get(i)`
- ⇄ pushing in a loop with a known count → `with_capacity` or `collect()`

CREATE

CODE	RESULT
`Vec::new()`	Empty
`vec![1, 2]`	Literal
`vec![0; n]`	n copies
`Vec::with_capacity(n)`	Reserved
`iter.collect()`	From iterator
`slice.to_vec()`	Copy a slice

ACCESS

METHOD	RETURNS
`v[i]`	T (panics)
`get(i)`	Option<&T>
`get_mut(i)`	Option<&mut T>
`first / last`	Option<&T>
`pop()`	Option<T>
`len / capacity`	usize

⚠ COMMON MISTAKES

- Holding `&v[0]` while pushing (E0502)
- Indexing with `i32` (indexes are `usize`)
- Calling `v.len() - 1` on an empty Vec (underflow)

✓ TIPS & BEST PRACTICES

- Use `collect()` instead of push loops when possible
- Use `last()` instead of `v[v.len() - 1]`
- Use `Box<[T]>` for a fixed buffer you will never grow

QUICK SUMMARY

- heap, growable, contiguous
- `push` / `pop`
- `v[i]` panics · `get(i)` → Option
- `with_capacity` pre-sizes

Vec — Methods

MODIFYING A VEC

Removing or inserting in the middle of a Vec shifts later elements, so it costs O(n). `swap_remove` is O(1) when order does not matter. `retain` filters in place, `dedup` removes **consecutive** duplicates, and `drain` removes a range while yielding the removed items.

KEY POINTS

- `remove(i)` and `insert(i, x)` panic if i is out of range
- `extend` takes any iterator; `append` moves from another Vec
- `dedup` only removes adjacent duplicates: sort first
- `clear` keeps the capacity

VEC METHODS

```
let mut v = vec![1, 2, 3, 4, 5];

v.insert(0, 0);                          // [0, 1, 2, 3, 4, 5]
let removed = v.remove(1);               // 1 → [0, 2, 3, 4, 5]
let swapped = v.swap_remove(0);          // 0 → [5, 2, 3, 4] (last moved in)
v.extend([6, 7]);                        // [5, 2, 3, 4, 6, 7]
v.extend_from_slice(&[8]);

let mut other = vec![100, 200];
v.append(&mut other);                    // other is now empty
println!("{v:?} {other:?}");

v.retain(|&x| x % 2 == 0);          // keep evens: [2, 4, 6, 8, 100, 200]
let tail = v.split_off(4);          // v = [2, 4, 6, 8], tail = [100, 200]
v.truncate(2);                           // [2, 4]
println!("{}", v.contains(&4));          // true

let mut words = vec!["b", "a", "b", "c", "a"];
words.sort();
words.dedup();                           // ["a", "b", "c"]

let mut nums = vec![10, 20, 30, 40, 50];
let middle: Vec<i32> = nums.drain(1..4).collect();   // [20, 30, 40]
println!("{nums:?}");                    // [10, 50]

let mut people = vec![("ada", 36), ("bob", 25), ("ada", 40)];
people.dedup_by_key(|p| p.0);            // removes only adjacent "ada" dupes
nums.resize(4, 0);                       // [10, 50, 0, 0]
nums.reverse();
nums.clear();                            // len 0, capacity kept
let joined = [1, 2, 3].map(|x| x.to_string()).join("-");   // "1-2-3"
```

CODE EXPLAINED

- → `swap_remove` fills the hole with the last element
- → `append` moves all items and leaves `other` empty
- → `retain` keeps only items matching the predicate
- → `drain(1..4)` removes and returns a range

COMING FROM…

- ↔ **Python** `list.insert`, `pop(i)`, list comprehension for filtering
- ↔ **Java** `ArrayList.add(i, x)`, `remove(i)`, `removeIf`
- ↔ **C++** `erase` / `insert` / `std::erase_if` (C++20)

THE RUST WAY

- ⇄ removing in a loop by index → `retain`
- ⇄ `v.remove(i)` when order is irrelevant → `swap_remove(i)`

METHODS

METHOD	COST	DOES
`push / pop`	`O(1)`	End
`insert / remove`	`O(n)`	Middle
`swap_remove`	`O(1)`	Unordered remove
`retain(f)`	`O(n)`	Filter in place
`dedup`	`O(n)`	Adjacent dupes
`drain(r)`	`O(n)`	Remove range
`extend / append`	`O(k)`	Add many
`truncate / clear`	`O(k)`	Shorten

⚠ COMMON MISTAKES

- `dedup` on unsorted data expecting all duplicates gone
- Removing by index while iterating forward (skips items)
- Calling `remove(0)` repeatedly (O(n²); use `VecDeque`)

✓ TIPS & BEST PRACTICES

- Sort then `dedup` to make values unique, or collect into a HashSet
- Use `drain(..)` to move every item out while keeping capacity
- Use `retain_mut` to modify and filter in one pass

QUICK SUMMARY

- `insert` / `remove` — O(n)
- `swap_remove` — O(1)
- `retain` / `dedup`
- `drain` / `split_off`

Sheet 055

HashMap — Basics

HashMap::new() · insert / get / remove · contains_key · random iteration order · HashMap::from([…]) · key / value ownership

KEY-VALUE STORAGE

`HashMap<K, V>` maps keys to values with average O(1) lookups. Keys must implement `Eq + Hash`. `insert` takes ownership of the key and value and returns the old value, `get` returns `Option<&V>`, and iteration order is **unspecified** and varies between runs.

KEY POINTS

- Import with `use std::collections::HashMap;`
- Default hasher is SipHash 1-3: resistant to HashDoS, not the fastest
- `get` accepts a borrowed key: `map.get("k")` for `String` keys
- Indexing `map["k"]` panics if missing

UNDER THE HOOD

- std uses a SwissTable layout (hashbrown): control bytes + slots in one allocation
- Each map gets a random seed (`RandomState`), so iteration order differs per run
- `HashMap::new()` does not allocate until the first insert
- Swap the hasher for speed with `ahash` / `FxHashMap` when DoS is not a concern

HASHMAP

```
use std::collections::HashMap;

let mut ages: HashMap<String, u32> = HashMap::new();
ages.insert("Ada".to_string(), 36);
ages.insert("Bob".to_string(), 25);
let old = ages.insert("Bob".to_string(), 26);   // Some(25): replaced

println!("{:?}", ages.get("Ada"));          // Some(36)
println!("{:?}", ages.get("Zed"));          // None
println!("{}", ages["Ada"]);                // 36 (panics if missing)
println!("{}", ages.contains_key("Bob"));   // true
println!("{}", ages.len());                 // 2

if let Some(age) = ages.get_mut("Ada") { *age += 1; }
let removed = ages.remove("Bob");           // Some(26)

let caps = HashMap::from([("fr", "Paris"), ("jp", "Tokyo")]);
let squares: HashMap<u32, u32> = (1..=4).map(|n| (n, n * n)).collect();

for (country, city) in &caps {              // order is not defined
    println!("{country}: {city}");
}
let mut keys: Vec<_> = squares.keys().copied().collect();
keys.sort();                                // sort for stable output
println!("{keys:?}");                       // [1, 2, 3, 4]
let total: u32 = squares.values().sum();    // 30

let key = String::from("Cy");
ages.insert(key, 50);                       // key moved into the map
// println!("{key}");                       // E0382
let n = ages.get("Cy").copied().unwrap_or(0);   // Option<&u32> → u32
ages.retain(|_, v| *v > 40);
```

CODE EXPLAINED

- → `insert` returns the previous value if the key existed
- → `get("Ada")` works with `String` keys thanks to `Borrow`
- → `collect()` builds a map from `(key, value)` pairs
- → Sort keys when you need deterministic output

COMING FROM…

- ↔ **Python** `dict` (keeps insertion order; Rust's HashMap does not)
- ↔ **Java** `HashMap<K,V>` ; `get` returns null instead of Option
- ↔ **Go** `map[K]V` ; `v, ok := m[k]` ≈ `get`

THE RUST WAY

- ⇄ `if map.contains_key(k) { map[k] }` → `if let Some(v) = map.get(k)`
- ⇄ need sorted or stable order → `BTreeMap` (sheet 056) or `indexmap`

METHODS

METHOD	RETURNS
`insert(k, v)`	Option<V> (old)
`get(&k)`	Option<&V>
`get_mut(&k)`	Option<&mut V>
`remove(&k)`	Option<V>
`contains_key(&k)`	bool
`entry(k)`	Entry (sheet 056)
`keys / values / iter`	Iterators
`retain(f)`	Filter in place

⚠ COMMON MISTAKES

- Relying on iteration order (it changes between runs)
- Using `map[k]` for keys that may be missing
- Floats as keys (f64 is not `Eq` / `Hash`)

✓ TIPS & BEST PRACTICES

- Use the entry API for insert-or-update (sheet 056)
- Use `&str` lookups on `String` keys
- Use `indexmap` when insertion order matters

QUICK SUMMARY

- `HashMap<K, V>` , O(1) average
- `insert` / `get` / `remove`
- `get` → `Option<&V>`
- no defined order

Entry API & BTreeMap

INSERT-OR-UPDATE AND SORTED MAPS

The **entry API** looks up a key once and lets you insert a default or update the existing value, avoiding double lookups and borrow problems. **BTreeMap** is a map kept in sorted key order, with O(log n) operations, ordered iteration, range queries and min/max access.

KEY POINTS

- `entry(k)` takes the key by value
- `or_insert` returns `&mut V`
- `or_default()` uses `V::default()` (0, empty Vec…)
- BTreeMap keys need `Ord`, not `Hash`

ENTRY API AND BTREEMAP

```
use std::collections::{BTreeMap, HashMap};

let text = "the cat and the hat and the bat";
let mut counts: HashMap<&str, usize> = HashMap::new();
for word in text.split_whitespace() {
    *counts.entry(word).or_insert(0) += 1;  // word-count pattern
}
println!("{}", counts["the"]);              // 3

let mut groups: HashMap<usize, Vec<&str>> = HashMap::new();
for w in ["hi", "yo", "hey", "cat"] {
    groups.entry(w.len()).or_default().push(w);   // group by length
}
println!("{:?}", groups[&3]);               // ["hey", "cat"]

let mut stock = HashMap::from([("apple", 5)]);
stock.entry("apple").and_modify(|n| *n += 10).or_insert(1);   // 15
stock.entry("pear").and_modify(|n| *n += 10).or_insert(1);    // 1
let cache = stock.entry("kiwi").or_insert_with(|| expensive());

let mut scores = BTreeMap::new();
scores.insert(72, "C");
scores.insert(95, "A");
scores.insert(85, "B");
for (score, grade) in &scores { print!("{score}{grade} "); }   // 72C 85B 95A
println!();
println!("{:?}", scores.first_key_value());  // Some((72, "C"))
println!("{:?}", scores.last_key_value());   // Some((95, "A"))
let b_or_better: Vec<_> = scores.range(80..).map(|(k, _)| k).collect();
println!("{b_or_better:?}");                 // [85, 95]
let sorted_counts: BTreeMap<_, _> = counts.into_iter().collect(); // by word
fn expensive() -> i32 { 42 }
```

CODE EXPLAINED

- → `entry(word).or_insert(0)` returns a `&mut usize` to increment
- → `or_default()` creates an empty Vec on first use
- → `and_modify` updates existing; `or_insert` handles new keys
- → BTreeMap iterates in key order and supports `range`

COMING FROM…

- ↔ **Python** `collections.Counter`, `defaultdict(list)`, `dict.setdefault`
- ↔ **Java** `merge(k, 1, Integer::sum)`, `computeIfAbsent`, `TreeMap`
- ↔ **C++** `map[k]++` default-inserts; `std::map` is ordered like BTreeMap

THE RUST WAY

- ⇄ `if let Some(v) = m.get_mut(k) { … } else { m.insert(k, …) }` → `entry`
- ⇄ sorting a HashMap's keys every time → `BTreeMap`

ENTRY METHODS

METHOD	DOES
`or_insert(v)`	Insert v if missing
`or_insert_with(f)`	Lazy default
`or_default()`	Default::default()
`and_modify(f)`	Update if present
`key()`	The key
`Entry::Occupied / Vacant`	Match for full control

HASHMAP VS BTREEMAP

ITEM	HASHMAP	BTREEMAP
Lookup	O(1) avg	O(log n)
Order	None	Sorted
Key needs	Eq + Hash	Ord
Range queries	No	Yes
Min / max	Scan	O(log n)

⚠ COMMON MISTAKES

- `or_insert(expensive())` running even when the key exists
- Holding the `&mut` from `or_insert` while touching the map again
- Using BTreeMap only to print sorted output once (sort instead)

✓ TIPS & BEST PRACTICES

- Use `entry` for counters, groupings and caches
- Use BTreeMap for leaderboards, time series and ranges
- Collect into BTreeMap to sort a HashMap for display

QUICK SUMMARY

- `entry().or_insert()`
- `or_default` / `and_modify`
- BTreeMap — sorted keys
- `range` / `first_key_value`

HashSet & BTreeSet

HashSet · BTreeSet · insert returns bool · union / intersection / difference / symmetric_difference · is_subset · dedup via collect

COLLECTIONS OF UNIQUE VALUES

A set stores each value at most once. **HashSet** has O(1) average operations with no order, and **BTreeSet** keeps values sorted. `insert` returns `false` when the value was already present, which makes "have I seen this?" checks one call. Set operations return lazy iterators.

KEY POINTS

- Same key rules as maps: `Eq + Hash` or `Ord`
- Operators work on references: `&a | &b`, `&a & &b`, `&a - &b`
- `collect::<HashSet<_>>()` removes duplicates
- `contains` takes a borrowed value

SETS

```
use std::collections::{BTreeSet, HashSet};

let mut seen = HashSet::new();
for x in [3, 1, 3, 2, 1] {
    if !seen.insert(x) { print!("dup {x} "); }   // dup 3 dup 1
}
println!("{}", seen.len());                  // 3

let a: HashSet<i32> = [1, 2, 3, 4].into();
let b: HashSet<i32> = [3, 4, 5].into();

let mut u: Vec<_> = a.union(&b).copied().collect();
u.sort();                                    // [1, 2, 3, 4, 5]
let i: HashSet<_> = a.intersection(&b).collect();   // {3, 4}
let d: HashSet<_> = &a - &b;                 // {1, 2}
let x: HashSet<_> = &a ^ &b;                 // {1, 2, 5}
println!("{:?}", u);

println!("{}", a.contains(&2));              // true
let small: HashSet<i32> = [3, 4].into();
println!("{}", small.is_subset(&a));         // true
println!("{}", a.is_disjoint(&HashSet::from([9])));   // true

let words = ["pear", "apple", "pear", "fig"];
let unique: BTreeSet<&str> = words.into_iter().collect();
println!("{unique:?}");                      // {"apple", "fig", "pear"}
println!("{:?}", unique.first());            // Some("apple")

let mut tags = HashSet::from(["rust".to_string()]);
println!("{}", tags.contains("rust"));       // &str lookup on String set
tags.remove("rust");
tags.retain(|t| t.len() > 3);
```

CODE EXPLAINED

→ `insert` returns false for values already in the set

→ Set methods borrow both sets and yield references

→ `&a - &b` builds a new set with the difference

→ Collecting into a BTreeSet dedups and sorts

COMING FROM…

↔ **Python** `set` with `|`, `&`, `-`, `^` operators

↔ **Java** `HashSet` / `TreeSet` with `retainAll`, `removeAll`

↔ **Go** no set type; `map[T]struct{}`

THE RUST WAY

⇄ `if !v.contains(&x) { v.push(x) }` on a Vec → `HashSet`

⇄ `contains` check then `insert` → just check `insert`'s return value

OPERATIONS

METHOD	OPERATOR	RESULT
union	&a | &b	In either
intersection	&a & &b	In both
difference	&a - &b	In a only
symmetric_difference	&a ^ &b	In exactly one
is_subset / is_superset	—	bool
is_disjoint	—	bool

⚠ COMMON MISTAKES

- Expecting a HashSet to print in order
- Calling `contains` then `insert` (two lookups)
- Storing f64 values (not Hash)

✓ TIPS & BEST PRACTICES

- Use a BTreeSet for sorted unique output
- Collect into a set to dedup without sorting
- Use `HashSet<&str>` to dedup borrowed text cheaply

QUICK SUMMARY

- unique values
- `insert` → bool (new?)
- `union` / `intersection` / `difference`
- BTreeSet — sorted

Sheet 058

VecDeque, BinaryHeap & Choosing

VecDeque `push_front` / `pop_back` · queues & stacks · `BinaryHeap` max-heap · `Reverse` for min-heap · `LinkedList` (rarely) · choosing a collection

QUEUES AND PRIORITY QUEUES

VecDeque is a ring buffer with O(1) push and pop at **both** ends, the standard queue. **BinaryHeap** is a priority queue that always pops the **largest** item; wrap items in `Reverse` for a min-heap. `LinkedList` exists but is rarely the right choice.

KEY POINTS

- A `Vec` is already a good stack (`push` / `pop`)
- VecDeque supports indexing and `make_contiguous`
- BinaryHeap iteration order is arbitrary; only `pop` is ordered
- Heap items need `Ord`

QUEUES AND HEAPS

```
use std::collections::{BinaryHeap, VecDeque};
use std::cmp::Reverse;

let mut q = VecDeque::new();
q.push_back("a");                       // enqueue
q.push_back("b");
q.push_front("z");                      // front: z a b
println!("{:?}", q.pop_front());        // Some("z") (dequeue)
println!("{:?} {}", q.back(), q[0]);    // Some("b") a
q.rotate_left(1);                       // b a

let mut stack = vec![1, 2];             // Vec as a stack
stack.push(3);
println!("{:?}", stack.pop());          // Some(3)

let mut heap = BinaryHeap::new();
for x in [5, 1, 8, 3] { heap.push(x); }
println!("{:?}", heap.peek());          // Some(8)
println!("{:?}", heap.pop());           // Some(8)
println!("{:?}", heap.into_sorted_vec()); // [1, 3, 5]

let mut min = BinaryHeap::new();
for x in [5, 1, 8] { min.push(Reverse(x)); }
println!("{:?}", min.pop().map(|Reverse(x)| x));   // Some(1)

let mut tasks = BinaryHeap::new();      // (priority, name)
tasks.push((2, "write"));
tasks.push((9, "deploy"));
tasks.push((5, "test"));
while let Some((p, name)) = tasks.pop() {
    print!("{name}:{p} ");              // deploy:9 test:5 write:2
}
println!();

let window: VecDeque<i32> = (1..=5).collect();
let last3: Vec<_> = window.iter().rev().take(3).collect();   // [5, 4, 3]
```

CODE EXPLAINED

- `push_front` / `pop_front` are O(1) on VecDeque
- `peek` shows the largest without removing it
- `Reverse(x)` flips ordering for a min-heap
- Tuples order by the first field, so they work as (priority, item)

COMING FROM…

- **Python** `collections.deque` and `heapq` (a min-heap)
- **Java** `ArrayDeque` and `PriorityQueue` (a min-heap)
- **C++** `std::deque` and `std::priority_queue` (a max-heap, like Rust)

THE RUST WAY

- `Vec::remove(0)` for a queue → `VecDeque::pop_front()`
- sorting after every insert → `BinaryHeap`

CHOOSING A COLLECTION

NEED	USE
List, stack, indexing	Vec
Queue / deque	VecDeque
Lookup by key	HashMap
Sorted keys, ranges	BTreeMap
Unique values	HashSet / BTreeSet
Highest/lowest first	BinaryHeap
O(1) splice, rare	LinkedList

⚠ COMMON MISTAKES

- Assuming BinaryHeap iterates in sorted order
- Using LinkedList for performance (poor cache use)
- Forgetting `Reverse` and getting a max-heap

✓ TIPS & BEST PRACTICES

- Default to Vec and HashMap; switch with a reason
- Use `(Reverse(cost), node)` in Dijkstra-style searches
- Use VecDeque for sliding windows and BFS

QUICK SUMMARY

- VecDeque — both ends O(1)
- BinaryHeap — max first
- `Reverse` — min-heap
- Vec for stacks

Iterators

Iterator & next() · iter() / iter_mut() / into_iter() · laziness · IntoIterator · implementing Iterator · iter::from_fn / successors

THE ITERATOR TRAIT

An iterator is any type with a `next()` method returning `Option<Item>`. Collections offer three: `iter()` yields `&T`, `iter_mut()` yields `&mut T`, and `into_iter()` yields owned `T`. Adapters are **lazy**: nothing runs until a consumer such as `for`, `collect` or `sum` pulls items.

KEY POINTS

- Implement only `next()`; 70+ methods come free
- `for x in coll` calls `IntoIterator::into_iter`
- An unconsumed adapter chain does nothing (compiler warns)
- Iterators can be infinite: `(0..)`, `repeat`, `successors`

UNDER THE HOOD

- Adapter chains compile to a single loop: no intermediate collections
- `map`, `filter` are structs wrapping the previous iterator
- LLVM often vectorizes slice iterator loops
- Iterating a slice skips per-element bounds checks that indexing needs

USING AND WRITING ITERATORS

```
let v = vec![1, 2, 3];
let mut it = v.iter();
println!("{:?}", it.next());                // Some(1)
println!("{:?}", it.next());                // Some(2)
println!("{:?}", it.next());                // Some(3)
println!("{:?}", it.next());                // None

let lazy = v.iter().map(|x| { println!("map {x}"); x * 2 });  // nothing yet
let doubled: Vec<i32> = lazy.collect();     // now it runs

let mut owned = vec![String::from("a")];
for s in owned.iter_mut() { s.push('!'); }  // &mut String
for s in owned.into_iter() { println!("{s}"); }   // String, owned moved

struct Countdown(u32);
impl Iterator for Countdown {
    type Item = u32;
    fn next(&mut self) -> Option<u32> {
        if self.0 == 0 { return None; }
        self.0 -= 1;
        Some(self.0 + 1)
    }
}
let c: Vec<u32> = Countdown(3).collect();   // [3, 2, 1]
println!("{}", Countdown(4).sum::<u32>());  // 10 (free method)

let mut n = 0;
let evens = std::iter::from_fn(|| { n += 2; (n <= 6).then_some(n) });
println!("{:?}", evens.collect::<Vec<_>>());   // [2, 4, 6]
let pow2: Vec<u32> = std::iter::successors(Some(1u32), |x| x.checked_mul(2))
    .take(5).collect();                     // [1, 2, 4, 8, 16]
let first_big = (1..).map(|x| x * x).find(|&x| x > 50);   // Some(64)
```

CODE EXPLAINED

- → Each `next()` call returns the following item or None
- → The `map` closure runs only when `collect` pulls items
- → Implementing `next` gives `sum`, `collect` and the rest
- → `(1..)` is infinite; `find` stops it

COMING FROM…

- ↔ **Java** `Stream` (lazy) and `Iterator` (hasNext/next)
- ↔ **Python** `__next__` / generators; `StopIteration` instead of None
- ↔ **C++** iterator pairs / C++20 ranges with lazy views

THE RUST WAY

- ⇄ building temporary Vecs between steps → one adapter chain
- ⇄ index loops over collections → iterators

THREE WAYS TO ITERATE

METHOD	ITEM	COLLECTION AFTER
`iter()`	`&T`	Usable
`iter_mut()`	`&mut T`	Usable, modified
`into_iter()`	`T`	Moved
`drain(..)`	`T`	Empty, usable

SOURCES

SOURCE	YIELDS
`0..n / (0..)`	Numbers
`iter::repeat(x)`	x forever
`iter::once(x)`	One item
`iter::from_fn(f)`	Until f → None
`iter::successors(s, f)`	Each from the last
`s.chars() / lines()`	Text pieces

⚠ COMMON MISTAKES

- Building an adapter chain and never consuming it
- Calling `into_iter()` and then using the collection
- Expecting side effects in `map` to run in a lazy chain

✓ TIPS & BEST PRACTICES

- Implement `Iterator` for your own sequences
- Return `impl Iterator<Item = T>` from functions (sheet 106)
- Use `by_ref()` to consume part of an iterator and keep the rest

QUICK SUMMARY

- `next()` → `Option<Item>`
- `iter` / `iter_mut` / `into_iter`
- adapters are lazy
- implement `next`, get the rest

Sheet 060

Iterator Adapters & Consumers

map / filter / filter_map · enumerate / zip / chain · take / skip / take_while · flat_map / flatten · peekable · fold / sum / count · collect::<Vec<_>>()

TRANSFORMING SEQUENCES

Adapters take an iterator and return a new lazy iterator: `map` transforms, `filter` selects, `filter_map` does both, `flat_map` flattens nested results. **Consumers** drive the chain and produce a result: `collect`, `sum`, `count`, `fold`, `min`, `max`. `collect` can build a Vec, String, HashMap, HashSet or `Result<Vec<_>, E>`.

KEY POINTS

- `filter` closures receive `&&T` for `iter()` chains: use `|&&x|` or `|x| **x`
- `copied()` / `cloned()` turn `&T` into `T`
- `sum` and `collect` usually need a type annotation
- `rev` needs a double-ended iterator

ADAPTER CHAINS

```
let nums = [1, 2, 3, 4, 5, 6, 7, 8];

let even_squares: Vec<i32> = nums.iter()
    .filter(|&&n| n % 2 == 0)
    .map(|n| n * n)
    .collect();                              // [4, 16, 36, 64]

let parsed: Vec<i32> = ["1", "x", "3"].iter()
    .filter_map(|s| s.parse().ok())
    .collect();                              // [1, 3]

let total: i32 = nums.iter().sum();          // 36
let product: i64 = (1..=10).product();       // 3628800
let odd_count = nums.iter().filter(|n| *n % 2 == 1).count();   // 4
let csv = nums.iter().map(|n| n.to_string()).collect::<Vec<_>>().join(",");

for (i, n) in nums.iter().enumerate().skip(6) { print!("{i}:{n} "); } // 6:7 7:8
let pairs: Vec<(char, i32)> = "abc".chars().zip(1..).collect();   // [('a',1),…]
let both: Vec<i32> = [1, 2].into_iter().chain([9]).collect();     // [1, 2, 9]
let head: Vec<_> = nums.iter().take_while(|&&n| n < 4).collect(); // [1, 2, 3]
let words: Vec<&str> = ["a b", "c"].iter().flat_map(|s| s.split(' ')).collect();
let flat: Vec<i32> = vec![vec![1], vec![2, 3]].into_iter().flatten().collect();

let sentence = nums.iter().fold(String::new(), |mut acc, n| {
    if !acc.is_empty() { acc.push('-'); }
    acc += &n.to_string();
    acc
});                                          // "1-2-3-4-5-6-7-8"

let mut it = [1, 1, 2, 3, 3].into_iter().peekable();
while let Some(x) = it.next() {
    if it.peek() == Some(&x) { print!("dup{x} "); }   // dup1 dup3
}
let (small, big): (Vec<i32>, Vec<i32>) = nums.into_iter().partition(|&n| n < 5);
println!("{:?}", nums.iter().max());         // Some(8)
```

CODE EXPLAINED

- → `filter(|&&n| …)` destructures the double reference
- → `filter_map` keeps only the `Some` results
- → `zip(1..)` numbers items from 1
- → `partition` splits into two collections by a predicate

COMING FROM…

- ↔ **Java** `stream().filter().map().collect(Collectors.toList())`
- ↔ **Python** comprehensions, `map`, `filter`, `itertools`
- ↔ **JavaScript** `arr.filter().map().reduce()` (eager, not lazy)

THE RUST WAY

- ⇄ `.filter(…).map(…)` with Options → `filter_map`
- ⇄ `.map(…).flatten()` → `flat_map`

ADAPTERS

ADAPTER	DOES
`map / filter`	Transform / select
`filter_map`	Both, via Option
`enumerate`	(index, item)
`zip / chain`	Pair / concatenate
`take / skip (_while)`	Limit
`flat_map / flatten`	Un-nest
`peekable`	Look ahead
`rev / step_by`	Order / stride
`inspect`	Debug peek

CONSUMERS

CONSUMER	RETURNS
`collect()`	Vec, String, Map…
`sum / product`	Number
`count`	usize
`fold(init, f)`	Accumulated
`min / max (_by_key)`	Option
`any / all`	bool
`find / position`	Option
`partition`	(A, B)
`for_each`	()

⚠ COMMON MISTAKES

- `filter(|n| n % 2 == 0)` on `&&i32` (type error: use `*n` or `&&n`)
- Forgetting a type for `collect()` or `sum()`
- Collecting into a Vec just to call `.len()` (use `count()`)

✓ TIPS & BEST PRACTICES

- Break long chains over lines, one adapter per line
- Use `inspect(|x| dbg!(x))` to debug chains
- Use `itertools` for `join`, `group_by`, `sorted`, `unique`

QUICK SUMMARY

- adapters: map, filter, zip…
- consumers: collect, sum, fold…
- `filter_map` / `flat_map`
- collect into many types

Sheet 061

Sorting & Searching

sort / sort_unstable · sort_by / sort_by_key · cmp::Reverse · binary_search · position / find / any / all · max_by_key / min_by

ORDERING AND FINDING DATA

`sort` is a stable sort and `sort_unstable` is usually faster and uses no extra memory. `sort_by_key` sorts by a computed key and `sort_by` takes a comparison function. `binary_search` works on sorted slices and returns `Ok(index)` or `Err(insert_position)`. Iterators provide `find`, `position`, `any`, `all` and `max_by_key`.

KEY POINTS

- Sorting needs `Ord`; floats need `sort_by(f64::total_cmp)`
- `sort_by_cached_key` computes each key once
- `Reverse(key)` sorts descending
- `is_sorted()` checks order (Rust 1.82)

SORTING AND SEARCHING

```
use std::cmp::Reverse;

let mut v = vec![5, 2, 9, 1, 7];
v.sort();                                  // [1, 2, 5, 7, 9]
v.sort_unstable_by(|a, b| b.cmp(a));       // [9, 7, 5, 2, 1]
v.sort_by_key(|&x| Reverse(x));            // descending again

let mut words = vec!["banana", "Apple", "fig", "cherry"];
words.sort_by_key(|w| w.len());     // ["fig", "Apple", "banana", "cherry"]
words.sort_by_key(|w| w.to_lowercase());   // allocates on every compare
words.sort_by_cached_key(|w| w.to_lowercase());   // allocates once per item

#[derive(Debug)]
struct P { name: &'static str, age: u32 }
let mut people = vec![P { name: "Bo", age: 30 }, P { name: "Al", age: 30 },
                      P { name: "Cy", age: 25 }];
people.sort_by(|a, b| b.age.cmp(&a.age).then_with(|| a.name.cmp(b.name)));
                                           // Al 30, Bo 30, Cy 25

let mut f = vec![2.5, -1.0, 9.75];
f.sort_by(f64::total_cmp);                 // [-1.0, 2.5, 9.75]

let sorted = [1, 3, 5, 7, 9];
println!("{:?}", sorted.binary_search(&7)); // Ok(3)
println!("{:?}", sorted.binary_search(&4)); // Err(2) → insert at 2
println!("{}", sorted.partition_point(|&x| x < 6));   // 3

println!("{:?}", v.iter().position(|&x| x == 5));     // Some(2)
println!("{:?}", v.iter().find(|&&x| x < 3));         // Some(2)
println!("{}", v.iter().any(|&x| x > 8));             // true
println!("{}", v.iter().all(|&x| x > 0));             // true
let oldest = people.iter().max_by_key(|p| p.age).unwrap();   // last max: Bo
let youngest = people.iter().min_by_key(|p| p.age).unwrap(); // Cy
println!("{} {}", oldest.name, sorted.is_sorted());   // Bo true
```

CODE EXPLAINED

- → `sort_unstable_by(|a, b| b.cmp(a))` sorts descending
- → `sort_by_cached_key` avoids recomputing expensive keys
- → `then_with` adds a tie-breaker
- → `binary_search` returns where a missing item would go

COMING FROM…

- ↔ **Python** `sorted(xs, key=…, reverse=True)` ; `bisect` for binary search
- ↔ **Java** `list.sort(Comparator.comparing(P::age).reversed())`
- ↔ **C++** `std::sort` (unstable) / `std::stable_sort` ; `lower_bound`

THE RUST WAY

- ⇄ `sort_by(|a, b| a.partial_cmp(b).unwrap())` → `sort_by(f64::total_cmp)`
- ⇄ `sort_by_key` with an allocating key → `sort_by_cached_key`

SORT METHODS

METHOD	STABLE	NOTES
`sort()`	Yes	Needs Ord
`sort_unstable()`	No	Faster, in place
`sort_by(f)`	Yes	Custom compare
`sort_by_key(f)`	Yes	Key per compare
`sort_by_cached_key(f)`	Yes	Key once per item
`select_nth_unstable(k)`	No	Kth element, O(n)

SEARCH

METHOD	RETURNS
`contains(&x)`	bool
`binary_search(&x)`	Result<usize, usize>
`partition_point(p)`	usize
`iter().position(p)`	Option<usize>
`iter().find(p)`	Option<&T>
`iter().any / all`	bool
`max_by_key / min_by_key`	Option<&T>

⚠ COMMON MISTAKES

- `binary_search` on unsorted data (meaningless result)
- Sorting floats with `sort()` (E0277: f64 is not Ord)
- Expecting `max_by_key` to return the first max (it returns the last)

✓ TIPS & BEST PRACTICES

- Use `sort_unstable` for primitives
- Use `Reverse` inside key tuples: `(Reverse(score), name)`
- Use `select_nth_unstable` for medians and top-k

QUICK SUMMARY

- `sort` stable · `sort_unstable` fast
- `sort_by_key` / `Reverse`
- `binary_search` → Ok/Err
- `find` / `position` / `any`

Structs

GROUPING DATA

A **struct** groups named fields into a new type. **Tuple structs** have unnamed fields (`struct Meters(f64)`) and are ideal for newtypes. **Unit structs** have no fields and serve as markers. All fields must be initialized, and the whole instance is either mutable or immutable.

KEY POINTS

- Field init shorthand: `User { name, age }`
- Struct update `..other` copies or moves remaining fields
- Mutability applies to the whole binding, not single fields
- Derive common traits: `Debug, Clone, PartialEq`

DEFINING STRUCTS

```
#[derive(Debug, Clone, PartialEq)]
struct User {
    name: String,
    email: String,
    age: u32,
    active: bool,
}

struct Meters(f64);                          // tuple struct (newtype)
struct Rgb(u8, u8, u8);
struct Marker;                               // unit struct

fn new_user(name: String, email: String) -> User {
    User { name, email, age: 0, active: true }   // field init shorthand
}

let mut u = new_user("Ada".into(), "ada@x.io".into());
u.age = 36;                                  // whole binding is mut
println!("{} {}", u.name, u.age);            // Ada 36
println!("{u:?}");                           // User { name: "Ada", … }

let u2 = User { email: "bob@x.io".into(), ..u.clone() };   // rest from u
let User { name, age, .. } = &u2;            // destructure
println!("{name} {age}");                    // Ada 36

let d = Meters(42.5);
let c = Rgb(255, 128, 0);
println!("{} {}", d.0, c.1);                 // 42.5 128
let Rgb(r, g, b) = c;

let _m = Marker;
println!("{}", std::mem::size_of::<Marker>());   // 0

let p = (u.name.clone(), u.age);             // tuple vs struct: names win
let same = u == u.clone();                   // PartialEq derived: true
```

CODE EXPLAINED

→ Every field must be set when creating a struct

→ `..u.clone()` fills the remaining fields

→ Tuple structs access fields by position

→ A unit struct takes zero bytes

COMING FROM…

↔ **C++** `struct` with public members; Rust fields are private outside the module

↔ **Java** `record User(String name, …)` ≈ derive Debug/Clone/PartialEq

↔ **Python** `@dataclass` ; **Go** `type User struct { … }`

THE RUST WAY

⇄ `let x = Point { x: x, y: y }` → `Point { x, y }`

⇄ raw `f64` for distances → newtype `Meters(f64)`

KINDS

KIND	EXAMPLE
Named fields	struct P { x: i32 }
Tuple struct	struct P(i32, i32);
Newtype	struct Id(u64);
Unit struct	struct Marker;
Generic	struct Wrap<T>(T);

SYNTAX

CODE	MEANING
`P { x, y }`	Shorthand
`P { x: 1, ..p }`	Update syntax
`p.x / t.0`	Field access
`let P { x, .. } = p;`	Destructure
`#[derive(…)]`	Generate traits

⚠ COMMON MISTAKES

- Making one field `mut` (not possible: use the whole binding or Cell)
- `..other` moving a String out of `other` and then using it
- Forgetting `#[derive(Debug)]` before printing with `{:?}`

✓ TIPS & BEST PRACTICES

- Use newtypes to prevent mixing units and IDs
- Implement `Default` and use `..Default::default()`
- Keep fields private and expose methods (sheet 084)

QUICK SUMMARY

- named, tuple, unit structs
- shorthand `P { x, y }`
- update `..other`
- derive Debug, Clone, PartialEq

Sheet 063

Methods & impl Blocks

impl · &self / &mut self / self · associated functions new() · Self · multiple impl blocks · auto-ref on method calls

BEHAVIOR FOR YOUR TYPES

Methods live in `impl` blocks. The first parameter decides access: `&self` reads, `&mut self` modifies, `self` consumes the value. Functions without `self` are **associated functions**, called with `Type::name()`, and new is the conventional constructor. `Self` refers to the type being implemented.

KEY POINTS

- `p.area()` auto-borrows: same as `Point::area(&p)`
- There are no constructors; `new` is just a convention
- A type can have many `impl` blocks
- Methods consuming `self` suit builders and conversions

IMPL BLOCKS

```
#[derive(Debug)]
struct Rect { w: f64, h: f64 }

impl Rect {
    pub fn new(w: f64, h: f64) -> Self { Self { w, h } }   // associated fn
    pub fn square(s: f64) -> Self { Self::new(s, s) }

    pub fn area(&self) -> f64 { self.w * self.h }          // read
    pub fn scale(&mut self, k: f64) { self.w *= k; self.h *= k; }   // modify
    pub fn into_tuple(self) -> (f64, f64) { (self.w, self.h) }    // consume

    pub fn fits_in(&self, other: &Rect) -> bool {
        self.w <= other.w && self.h <= other.h
    }
}

impl Rect {                                 // a second impl block is fine
    pub const UNIT: Rect = Rect { w: 1.0, h: 1.0 };   // associated const
}

let mut r = Rect::new(3.0, 4.0);
println!("{}", r.area());                   // 12
r.scale(2.0);                               // auto &mut r
println!("{:?}", r);                        // Rect { w: 6.0, h: 8.0 }
println!("{}", Rect::square(2.0).fits_in(&r));   // true
println!("{}", Rect::area(&Rect::UNIT));    // 1: explicit form

let t = r.into_tuple();                     // r moved
// r.area();                                // E0382: borrow of moved value

struct Counter { n: u32 }
impl Counter {
    fn tick(&mut self) -> &mut Self { self.n += 1; self }   // chainable
}
let mut c = Counter { n: 0 };
c.tick().tick().tick();
println!("{}", c.n);                        // 3
```

CODE EXPLAINED

- → `new` and `square` are called with `Rect::`
- → `r.scale(2.0)` borrows `r` mutably automatically
- → `into_tuple(self)` consumes the Rect
- → Returning `&mut Self` allows method chaining

COMING FROM…

- ↔ **C++ / Java** methods inside the class body; `this` is implicit
- ↔ **Python** `def area(self)` with explicit self, like Rust
- ↔ **Go** methods with receivers: `func (r *Rect) Scale(k float64)`

THE RUST WAY

- ⇄ `fn get_area(&self)` → `fn area(&self)` (no `get_` prefix)
- ⇄ `new()` with no arguments → also implement `Default`

RECEIVERS

RECEIVER	MEANS	CALL
`&self`	Borrow	`r.area()`
`&mut self`	Mutable borrow	`r.scale(2.0)`
`self`	Consume	`r.into_tuple()`
`mut self`	Consume, modify	`builder.port(80)`
(none)	Associated fn	`Rect::new()`
`self: Box<Self>`	Boxed self	`b.run()`

⚠ COMMON MISTAKES

- Calling a `&mut self` method on an immutable binding
- Using `self` when `&self` would do (moves the caller's value)
- Expecting `new` to be special

✓ TIPS & BEST PRACTICES

- Name conversions `into_*` (consume), `as_*` (borrow), `to_*` (copy)
- Return `Self` from constructors
- Group methods into several impl blocks by topic

QUICK SUMMARY

- `impl Type { … }`
- `&self` · `&mut self` · `self`
- `Type::new()` convention
- `Self` = this type

Sheet 064

Enums with Data

variants with data · struct-like variants · `impl` on enums · `Option` & `Result` are enums · C-like enums & discriminants · `match` on enums

SUM TYPES

A Rust `enum` is a type that is **one of several variants**, and each variant can carry its own data. This models states and messages precisely: a value cannot be in an impossible combination. `match` must handle every variant, so adding one shows you every place to update. `Option` and `Result` are ordinary enums.

KEY POINTS

- Variants can be unit, tuple-like or struct-like
- Enums can have methods and implement traits
- C-like enums can have explicit discriminants and `as` casts
- Use enums for state machines and protocol messages

UNDER THE HOOD

- Layout: a tag (discriminant) + space for the largest variant
- **Niche optimization**: `Option<&T>`, `Option<Box<T>>` reuse the null value as the tag
- Size check: `size_of::<Option<bool>>()` is 1
- A `match` on an enum typically compiles to a jump table on the tag

ENUMS

```
#[derive(Debug)]
enum Shape {
    Circle(f64),                        // tuple-like
    Rect { w: f64, h: f64 },            // struct-like
    Point,                              // unit
}
impl Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Circle(r) => std::f64::consts::PI * r * r,
            Shape::Rect { w, h } => w * h,
            Shape::Point => 0.0,
        }
    }
}
let shapes = vec![Shape::Circle(1.0), Shape::Rect { w: 2.0, h: 3.0 },
                  Shape::Point];
let total: f64 = shapes.iter().map(Shape::area).sum();   // 9.14…

enum Msg { Quit, Move { x: i32, y: i32 }, Write(String) }
fn handle(m: Msg) {
    match m {
        Msg::Quit => println!("bye"),
        Msg::Move { x, y } => println!("to {x},{y}"),
        Msg::Write(text) => println!("{text}"),
    }
}
handle(Msg::Move { x: 1, y: 2 });

#[derive(Debug, Clone, Copy, PartialEq)]
enum Level { Low = 1, Mid = 5, High = 10 }  // C-like, explicit values
println!("{}", Level::Mid as i32);          // 5

use std::mem::size_of;
println!("{}", size_of::<Option<Box<i32>>>());   // 8: niche, no extra tag
println!("{}", size_of::<Option<u64>>());        // 16: tag + padding
```

CODE EXPLAINED

→ Each variant holds different data

→ `match self` handles every variant in `area`

→ Explicit discriminants allow `as i32` casts

→ Option<Box<T>> uses the null niche, so no tag byte is needed

COMING FROM…

↔ **C++** `std::variant` + `std::visit` (much clunkier)

↔ **Java** sealed interfaces + records + pattern switch (Java 21)

↔ **Go / Python** no sum types; use interfaces, classes or tags

THE RUST WAY

⇄ a struct with `kind: String` and optional fields → an enum with data

⇄ `bool` flags that conflict → enum variants

VARIANT KINDS

KIND	DEFINE	BUILD
Unit	`Quit`	`Msg::Quit`
Tuple	`Write(String)`	`Msg::Write(s)`
Struct	`Move { x: i32 }`	`Msg::Move { x: 1 }`
C-like	`Low = 1`	`Level::Low as i32`

SIZES

TYPE	BYTES
`Option<bool>`	1
`Option<&T>`	8
`Option<Box<T>>`	8
`Option<u64>`	16
`Result<u32, ()>`	8

⚠ COMMON MISTAKES

- A `_ =>` arm on your own enum hiding new variants
- Large variant making every value big (box the big one)
- Casting a data-carrying enum with `as` (only C-like enums)

✓ TIPS & BEST PRACTICES

- Box large variant payloads: `Big(Box<Data>)`
- Use `#[non_exhaustive]` on public enums that may grow
- Use `strum` to iterate variants or parse names

QUICK SUMMARY

- one of several variants
- variants carry data
- exhaustive `match`
- niche-optimized layouts

Traits — Basics

`trait` definitions · `impl Trait for Type` · trait methods · orphan rule · `impl Trait` params · `&dyn Trait` preview

SHARED BEHAVIOR

A **trait** declares methods that types can implement, like an interface. Implement it with `impl Trait for Type`. Functions accept "anything implementing the trait" with `impl Trait` (static dispatch) or `&dyn Trait` (dynamic dispatch). The **orphan rule** requires either the trait or the type to be defined in your crate.

KEY POINTS

- Traits must be in scope (`use`) to call their methods
- You can implement your traits for std types like `i32`
- You cannot implement `Display` for `Vec<T>` (both foreign)
- Wrap foreign types in a newtype to get around the orphan rule

DEFINING AND IMPLEMENTING TRAITS

```
trait Describe {
    fn name(&self) -> String;
    fn describe(&self) -> String;
}
struct Dog { breed: String }
struct Robot { id: u32 }
impl Describe for Dog {
    fn name(&self) -> String { "dog".into() }
    fn describe(&self) -> String { format!("a {} dog", self.breed) }
}
impl Describe for Robot {
    fn name(&self) -> String { format!("robot-{}", self.id) }
    fn describe(&self) -> String { "a metal friend".into() }
}
impl Describe for i32 {                    // our trait, std type: allowed
    fn name(&self) -> String { "number".into() }
    fn describe(&self) -> String { format!("the number {self}") }
}
fn introduce(x: &impl Describe) {          // static dispatch
    println!("{}: {}", x.name(), x.describe());
}
introduce(&Dog { breed: "corgi".into() });  // dog: a corgi dog
introduce(&7);                              // number: the number 7

let things: Vec<Box<dyn Describe>> = vec![  // dynamic dispatch (sheet 070)
    Box::new(Robot { id: 9 }),
    Box::new(Dog { breed: "lab".into() }),
];
for t in &things { println!("{}", t.describe()); }

// impl std::fmt::Display for Vec<i32> {}   // E0117: orphan rule
struct List(Vec<i32>);                      // newtype workaround
impl std::fmt::Display for List {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "[{} items]", self.0.len())
    }
}
```

CODE EXPLAINED

→ `Describe` lists methods every implementor must provide

→ Each type supplies its own method bodies

→ `&impl Describe` accepts any implementing type

→ A newtype lets you implement foreign traits for foreign types

COMING FROM…

↔ **Java** interfaces; Rust traits can also be added to existing types

↔ **Go** interfaces are implicit; Rust impls are explicit

↔ **C++** concepts (C++20) for static, virtual base classes for dynamic

THE RUST WAY

⇄ inheritance hierarchies → traits plus composition

⇄ methods taking a concrete type when any implementor would do → `impl Trait`

PIECES

SYNTAX	MEANING
`trait T { fn f(&self); }`	Declare
`impl T for X { … }`	Implement
`fn g(x: &impl T)`	Accept any T
`fn g<U: T>(x: &U)`	Same, generic form
`&dyn T / Box<dyn T>`	Trait object
`use crate::T;`	Bring methods into scope

ORPHAN RULE

TRAIT	TYPE	ALLOWED
Yours	Yours	Yes
Yours	Foreign	Yes
Foreign	Yours	Yes
Foreign	Foreign	No (use newtype)

⚠ COMMON MISTAKES

- "no method named X found": the trait is not imported
- Trying to implement a foreign trait for a foreign type (E0117)
- Modeling class hierarchies with trait inheritance

✓ TIPS & BEST PRACTICES

- Keep traits small and focused
- Prefer `impl Trait` parameters; use `dyn` for mixed collections
- Provide default methods to reduce boilerplate (sheet 066)

QUICK SUMMARY

- trait = shared behavior
- `impl Trait for Type`
- `impl Trait` / `dyn Trait`
- orphan rule → newtype

Sheet 066

Default Methods, derive & Supertraits

default method bodies · overriding defaults · `#[derive(…)]` · supertraits `trait A: B` · blanket impls · marker traits

REUSING TRAIT CODE

Trait methods can have **default bodies** that implementors get for free or override. `#[derive]` generates impls of standard traits automatically. A **supertrait** (`trait Pretty: Display`) requires implementors to also implement another trait. **Blanket impls** implement a trait for every type meeting a bound.

KEY POINTS

- Default methods can call required methods
- `#[derive]` works for Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Default
- Marker traits have no methods (`Copy` , `Send` , `Eq`)
- std has blanket impls like `impl<T: Display> ToString for T`

DEFAULTS, SUPERTRAITS, BLANKET IMPLS

```
use std::fmt::Display;

trait Greet {
    fn name(&self) -> String;               // required
    fn hello(&self) -> String {             // default body
        format!("Hello, {}!", self.name())
    }
}

struct En; struct Es;
impl Greet for En { fn name(&self) -> String { "Ada".into() } }
impl Greet for Es {
    fn name(&self) -> String { "Ana".into() }
    fn hello(&self) -> String { format!("¡Hola, {}!", self.name()) } // override
}
println!("{} {}", En.hello(), Es.hello());  // Hello, Ada! ¡Hola, Ana!

trait Labeled: Display {                    // supertrait
    fn label(&self) -> String { format!("<{self}>") }   // can use Display
}
struct Tag(&'static str);
impl Display for Tag {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "{}", self.0)
    }
}
impl Labeled for Tag {}                     // all defaults
println!("{}", Tag("new").label());         // <new>

trait Shout { fn shout(&self) -> String; }
impl<T: Display> Shout for T {              // blanket impl
    fn shout(&self) -> String { self.to_string().to_uppercase() }
}
println!("{} {}", "hi".shout(), 42_i32.shout());   // HI 42

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Default)]
struct Id(u32);
println!("{:?} {}", Id::default(), Id(1) < Id(2));   // Id(0) true
```

CODE EXPLAINED

- → `En` uses the default `hello` ; `Es` overrides it
- → `Labeled: Display` lets its default use `{self}`
- → The blanket impl gives `shout` to every Display type
- → One derive line generates nine trait impls

COMING FROM…

- ↔ **Java** `default` interface methods; interfaces can extend interfaces
- ↔ **C++** base classes with virtual functions and default implementations
- ↔ **Python** mixins and ABCs with concrete methods

THE RUST WAY

- ⇄ copy-pasted method bodies across impls → a default method
- ⇄ hand-written `PartialEq` / `Hash` matching fields → `#[derive]`

DERIVABLE TRAITS

TRAIT	GIVES
`Debug`	{:?}
`Clone / Copy`	.clone() / implicit copy
`PartialEq / Eq`	=
`Hash`	Map/set key
`PartialOrd / Ord`	< > sort
`Default`	T::default()

TRAIT FEATURES

FEATURE	SYNTAX
`Default method`	fn f(&self) { … }
`Supertrait`	trait A: B + C
`Blanket impl`	impl<T: B> A for T
`Marker`	trait Marker {}
`Empty impl`	impl A for X {}

⚠ COMMON MISTAKES

- Overlapping blanket impls (E0119: conflicting implementations)
- Deriving `Eq` on a struct with f64 fields (not allowed)
- Forgetting to implement the supertrait first

✓ TIPS & BEST PRACTICES

- Put shared logic in default methods built on a few required ones
- Derive generously on data types
- Add blanket impls carefully in libraries: they limit future impls

QUICK SUMMARY

- default bodies + overrides
- `#[derive(…)]`
- supertraits `A: B`
- blanket `impl<T: B> A for T`

Sheet 067

Common Standard Traits

Debug / Display · Clone / Copy · PartialEq / Eq / Hash · PartialOrd / Ord · Default · From / Into · AsRef / Borrow

TRAITS EVERY TYPE SHOULD CONSIDER

A handful of std traits make your types work with the rest of Rust: printing (`Debug`, `Display`), copying (`Clone`, `Copy`), comparing and hashing (`PartialEq`, `Eq`, `Hash`, `Ord`), defaults (`Default`) and conversions (`From`, `AsRef`). Implementing them unlocks `{}`, `==`, sorting, HashMap keys and `.into()`.

KEY POINTS

- `Display` is never derived; write it by hand
- `Hash` must agree with `Eq`: equal values hash equally
- Implement `From`; `Into` comes free
- `AsRef<str>` accepts String, &str and more

IMPLEMENTING STD TRAITS

```
use std::fmt;
use std::collections::HashMap;

#[derive(Debug, Clone, PartialEq, Eq, Hash, PartialOrd, Ord, Default)]
struct Email(String);
impl fmt::Display for Email {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "<{}>", self.0)
    }
}
impl From<&str> for Email {
    fn from(s: &str) -> Self { Email(s.to_lowercase()) }
}
impl AsRef<str> for Email {
    fn as_ref(&self) -> &str { &self.0 }
}

let e: Email = "Ada@X.io".into();           // From → Into
println!("{e}");                            // <ada@x.io>
println!("{e:?}");                          // Email("ada@x.io")

let mut logins: HashMap<Email, u32> = HashMap::new();   // needs Eq + Hash
*logins.entry(e.clone()).or_default() += 1;

let mut list = vec![Email::from("b@x"), Email::from("a@x")];
list.sort();                                // needs Ord
println!("{}", list[0]);                    // <a@x>

fn domain<S: AsRef<str>>(s: S) -> String {
    s.as_ref().split('@').nth(1).unwrap_or("").to_string()
}
println!("{} {}", domain(&e), domain("x@y.com"));   // x.io y.com
```

CODE EXPLAINED

- → `Display` controls `{}`; `Debug` (derived) controls `{:?}`
- → `From<&str>` makes `"…".into()` work
- → `Eq + Hash` make `Email` a HashMap key; `Ord` enables sort
- → `domain` accepts both `&Email` and `&str` through `AsRef<str>`

COMING FROM…

- ↔ **Java** `toString`, `equals` / `hashCode`, `Comparable`
- ↔ **Python** `__str__` / `__repr__`, `__eq__` / `__hash__`, `__lt__`
- ↔ **C++** `operator<<`, `operator==`, `std::hash` specialization

THE RUST WAY

- ⇄ a `to_string()` method → `impl Display` (gives `to_string` free)
- ⇄ implementing `Into` → implement `From` instead

TRAITS

TRAIT	ENABLES	DERIVE?
Debug	{:?}	Yes
Display	{} / to_string	No
Clone / Copy	clone / copy	Yes
PartialEq / Eq	==	Yes
Hash	HashMap keys	Yes
PartialOrd / Ord	< sort max	Yes
Default	default()	Yes
From / Into	.into()	No
AsRef / Borrow	Cheap views	No

⚠ COMMON MISTAKES

- Manual `Hash` that disagrees with `PartialEq`
- Implementing `Display` with a trailing newline
- Implementing `Into` directly (blocks the blanket impl benefit)

✓ TIPS & BEST PRACTICES

- Derive `Debug` on every type
- Implement `Display` for user-facing types and errors
- Use `Borrow` for map keys that should look up by `&str`

QUICK SUMMARY

- `Debug` derive · `Display` by hand
- `Eq + Hash` → map keys
- `Ord` → sort
- `From` → `.into()`

Generics

generic functions `fn largest<T>` · generic structs & enums · `impl<T>` · monomorphization · const generics `[T; N]` · `PhantomData`

CODE THAT WORKS FOR MANY TYPES

Generics let one definition work for many types: `fn largest<T: PartialOrd>(v: &[T]) -> &T`. Structs and enums can be generic too (`Vec<T>`, `Option<T>`). The compiler generates a specialized copy for each concrete type used, so generics have **zero runtime cost**. **Const generics** parameterize over values like array lengths.

KEY POINTS

- Type parameters are CamelCase single letters by convention
- Bounds (`T: Trait`) say what the code may do with T
- `impl<T> Point<T>` vs `impl Point<f64>` (specific type only)
- `PhantomData<T>` marks an unused type parameter

UNDER THE HOOD

- **Monomorphization**: `largest::<i32>` and `largest::<f64>` become separate machine-code functions
- Calls are static and inlinable, as fast as hand-written code
- Cost: longer compile times and larger binaries
- Const generics are also monomorphized per `N`

GENERIC FUNCTIONS AND TYPES

```
fn largest<T: PartialOrd>(items: &[T]) -> &T {
    let mut max = &items[0];
    for it in items { if it > max { max = it; } }
    max
}
println!("{}", largest(&[3, 9, 2]));        // 9
println!("{}", largest(&["b", "z", "a"]));  // z

#[derive(Debug)]
struct Point<T> { x: T, y: T }

impl<T: std::ops::Add<Output = T> + Copy> Point<T> {
    fn sum(&self) -> T { self.x + self.y }
}
impl Point<f64> {                           // only for f64
    fn dist(&self) -> f64 { (self.x * self.x + self.y * self.y).sqrt() }
}
let pi = Point { x: 1, y: 2 };
let pf = Point { x: 3.0, y: 4.0 };
println!("{} {}", pi.sum(), pf.dist());     // 3 5

struct Pair<A, B> { a: A, b: B }
let p = Pair { a: "id", b: 7u8 };

fn sum_all<const N: usize>(arr: [i32; N]) -> i32 { arr.iter().sum() }
println!("{}", sum_all([1, 2, 3]));         // 6 (N = 3)

struct Matrix<const R: usize, const C: usize>([[f32; C]; R]);
let m: Matrix<2, 3> = Matrix([[0.0; 3]; 2]);

use std::marker::PhantomData;
struct Meters;
struct Length<Unit> { value: f64, _unit: PhantomData<Unit> }
let len: Length<Meters> = Length { value: 5.0, _unit: PhantomData };
```

CODE EXPLAINED

- `T: PartialOrd` allows `>` on items
- `impl Point<f64>` adds a method only for f64 points
- Const generic `N` is inferred from the array length
- `PhantomData<Unit>` tags a type without storing data

COMING FROM…

- **C++** templates (also monomorphized), checked at instantiation
- **Java** generics with type erasure: one runtime copy, boxed values
- **Go** generics (1.18+) with type sets; partly dictionary-based

THE RUST WAY

- copy-pasted functions for i32/f64 → one generic function
- `Box<dyn Trait>` where one type is used per call site → generics

SYNTAX

FORM	EXAMPLE
Function	fn f<T>(x: T)
Struct	struct S<T> { v: T }
Enum	enum E<T> { A(T) }
Impl for all	impl<T> S<T>
Impl for one	impl S<f64>
Const generic	fn f<const N: usize>
Turbofish	f::<i32>(…)

GENERIC VS DYN

ITEM	GENERICS	DYN TRAIT
Dispatch	Static	Vtable
Speed	Inlinable	Indirect call
Binary size	Larger	Smaller
Mixed types in one Vec	No	Yes

⚠ COMMON MISTAKES

- Using an operation without the bound (E0369 binary operation cannot be applied)
- Over-generic APIs that are hard to read
- Expecting runtime type info about T

✓ TIPS & BEST PRACTICES

- Start concrete, generalize when a second type appears
- Put long bounds in `where` clauses (sheet 069)
- Use const generics for fixed-size buffers and matrices

QUICK SUMMARY

- `<T>` on fns, structs, enums
- bounds limit what T can do
- monomorphized: zero cost
- const generics `<const N: usize>`

Sheet 069

Trait Bounds & where

`T: Display + Clone` · `where` clauses · `impl Trait` in argument & return position · conditional impls · `?Sized`

CONSTRAINING GENERICS

A **trait bound** like `T: Display + Clone` limits a generic to types implementing those traits, which lets the body call their methods. Long bounds move to a `where` clause. `impl Trait` is shorthand in argument position and an opaque return type in return position. Bounds on an `impl` block create **conditional** methods.

KEY POINTS

- `+` combines bounds
- `where` also allows bounds on associated types
- Return-position `impl Trait` hides one concrete type
- Generics are `Sized` by default; `?Sized` relaxes that

BOUNDS AND WHERE

```
use std::fmt::{Debug, Display};
use std::hash::Hash;
use std::collections::HashMap;

fn show_twice<T: Display + Clone>(x: T) -> String {
    let y = x.clone();
    format!("{x} {y}")
}

fn count_items<I, K>(items: I) -> HashMap<K, usize>
where
    I: IntoIterator<Item = K>,
    K: Eq + Hash,
{
    let mut m = HashMap::new();
    for k in items { *m.entry(k).or_insert(0) += 1; }
    m
}
println!("{:?}", count_items("abca".chars()).get(&'a'));   // Some(2)

fn log_all(items: &[impl Debug]) { for i in items { println!("{i:?}"); } }

fn evens(limit: u32) -> impl Iterator<Item = u32> {   // opaque return type
    (0..limit).filter(|n| n % 2 == 0)
}
println!("{:?}", evens(7).collect::<Vec<_>>());   // [0, 2, 4, 6]

struct Wrapper<T>(T);
impl<T: Display> Wrapper<T> {               // method exists only if T: Display
    fn print(&self) { println!("[{}]", self.0); }
}
Wrapper(5).print();                         // [5]
// Wrapper(vec![1]).print();                // E0599: bounds not satisfied

fn describe<T: Debug + ?Sized>(x: &T) { println!("{x:?}"); }
describe("a str");                          // T = str (unsized) is OK
describe(&[1, 2][..]);                      // T = [i32]
```

CODE EXPLAINED

- → `T: Display + Clone` allows `{}` and `.clone()`
- → The `where` clause keeps the signature readable
- → `impl Iterator` hides the exact filter type
- → `impl<T: Display>` adds `print` only where T is printable

COMING FROM…

- ↔ **C++20** `requires` clauses and concepts
- ↔ **Java** `<T extends Comparable<T>>` bounded types
- ↔ **TypeScript** `<T extends Shape>` constraints

THE RUST WAY

- ⇄ `fn f<T: A + B + C, U: D>(…)` → `where` clause
- ⇄ returning `Box<dyn Iterator>` for one type → `impl Iterator`

FORMS

SYNTAX	MEANING
`<T: A>`	Inline bound
`<T: A + B>`	Multiple
`where T: A`	Where clause
`x: impl A`	Anonymous generic
`-> impl A`	Opaque return
`T: ?Sized`	Allow str, [T], dyn
`T: 'static`	No short borrows
`I: Iterator<Item = u8>`	Associated type bound

⚠ COMMON MISTAKES

- Returning two different types from `-> impl Trait`
- Missing bound errors (E0277: trait bound not satisfied)
- Unnecessary bounds on struct definitions (put them on impls)

✓ TIPS & BEST PRACTICES

- Bound impls, not structs, unless the struct needs it
- Use `impl Trait` args for simple cases; named generics when reused
- Read E0277 help: it names the missing trait

QUICK SUMMARY

- `T: A + B`
- `where` for long bounds
- `impl Trait` arg / return
- conditional `impl<T: A>`

Sheet 070

Trait Objects & dyn

dyn Trait · Box<dyn Trait> / &dyn Trait · dynamic dispatch & vtables · dyn compatibility · Vec<Box<dyn Shape>> · generics vs trait objects

RUNTIME POLYMORPHISM

A **trait object** `dyn Trait` lets one variable hold any type implementing the trait, chosen at runtime. It must sit behind a pointer: `&dyn Trait`, `Box<dyn Trait>` or `Arc<dyn Trait>`. Method calls go through a **vtable**. Only **dyn-compatible** traits (formerly "object safe") can be used this way.

KEY POINTS

- Use for heterogeneous collections and plugin-style APIs
- Not dyn-compatible: generic methods, `-> Self`, `Self: Sized` requirements
- Add `+ Send + Sync` for thread-safe objects
- Downcasting needs `std::any::Any`

UNDER THE HOOD

- `&dyn Trait` is a fat pointer: data pointer + vtable pointer (16 bytes)
- The vtable holds drop, size, align and one entry per method
- Calls are indirect and usually not inlined
- One vtable per (type, trait) pair, stored in read-only data

DYN TRAIT

```
trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> &str { "shape" }
}
struct Circle(f64);
struct Square(f64);
impl Shape for Circle {
    fn area(&self) -> f64 { std::f64::consts::PI * self.0 * self.0 }
    fn name(&self) -> &str { "circle" }
}
impl Shape for Square { fn area(&self) -> f64 { self.0 * self.0 } }

let shapes: Vec<Box<dyn Shape>> =
    vec![Box::new(Circle(1.0)), Box::new(Square(2.0))];
for s in &shapes {
    println!("{} {:.2}", s.name(), s.area());   // circle 3.14 / shape 4.00
}
let total: f64 = shapes.iter().map(|s| s.area()).sum();

fn print_area(s: &dyn Shape) { println!("{:.1}", s.area()); }   // borrowed
print_area(&Square(3.0));                    // 9.0

fn make(kind: &str) -> Box<dyn Shape> {      // type chosen at runtime
    if kind == "c" { Box::new(Circle(2.0)) } else { Box::new(Square(2.0)) }
}

use std::sync::Arc;
let shared: Arc<dyn Shape + Send + Sync> = Arc::new(Circle(0.5));

trait Cloner { fn dup(&self) -> Self; }      // returns Self → not dyn compatible
// let x: Box<dyn Cloner>;                   // E0038

use std::any::Any;
let items: Vec<Box<dyn Any>> = vec![Box::new(5i32), Box::new("hi")];
for i in &items {
    if let Some(n) = i.downcast_ref::<i32>() { println!("int {n}"); }
}
println!("{}", std::mem::size_of::<&dyn Shape>());   // 16
```

CODE EXPLAINED

- → `Vec<Box<dyn Shape>>` holds different shape types together
- → `&dyn Shape` borrows a trait object without allocating
- → `make` decides the concrete type at runtime
- → Methods returning `Self` make a trait not dyn-compatible (E0038)

COMING FROM…

- ↔ **C++** virtual functions and `std::unique_ptr<Base>`
- ↔ **Java** interface references (every call is virtual)
- ↔ **Go** interface values (also data + itab pointers)

THE RUST WAY

- ⇄ `Box<dyn Trait>` param used with one type → `impl Trait`
- ⇄ an enum is closed and known → enum instead of `dyn` (faster, matchable)

GENERICS VS DYN VS ENUM

NEED	USE
One type per call site	Generics / impl Trait
Mixed types, open set	Box<dyn Trait>
Mixed types, closed set	enum
Plugins / callbacks	Box<dyn Fn>
Shared across threads	Arc<dyn T + Send + Sync>

NOT DYN COMPATIBLE

TRAIT HAS	EXAMPLE
Generic method	fn f<T>(&self)
Returns Self	fn dup(&self) -> Self
Static fn without self	fn new() -> Self
Self: Sized supertrait	trait T: Sized

⚠ COMMON MISTAKES

- `Vec<dyn Shape>` without a pointer (E0277: size not known)
- Trying to clone a `Box<dyn Trait>` (use `dyn-clone`)
- Defaulting to `dyn` when generics or enums would be faster

✓ TIPS & BEST PRACTICES

- Add `where Self: Sized` to exclude a method from the vtable
- Use enums when you control every variant
- Use `Box<dyn Error + Send + Sync>` for thread-safe errors

QUICK SUMMARY

- `dyn Trait` behind a pointer
- vtable = dynamic dispatch
- fat pointer: data + vtable
- dyn compatibility rules

Sheet 071

Associated Types & Consts

type Item · associated consts · Iterator<Item = T> · generic trait vs associated type · <T as Trait>::f · default type parameters

TYPES AND CONSTANTS INSIDE TRAITS

An **associated type** is a placeholder type each implementor fixes once, like `Iterator::Item`. Use it when a type has exactly one natural choice; use a **generic** trait (`From<T>`) when one type can implement it many times. Traits can also declare **associated constants**, and fully qualified syntax resolves name clashes.

KEY POINTS

- Refer to it as `Self::Item` inside the trait
- Bound it from outside: `I: Iterator<Item = u8>`
- Associated consts: `const ID: u32;`
- Default type params: `trait Add<Rhs = Self>`

ASSOCIATED ITEMS

```
trait Container {
    type Item;                              // associated type
    const CAPACITY: usize;                  // associated const
    fn get(&self, i: usize) -> Option<&Self::Item>;
    fn first(&self) -> Option<&Self::Item> { self.get(0) }
}

struct Names(Vec<String>);
impl Container for Names {
    type Item = String;
    const CAPACITY: usize = 100;
    fn get(&self, i: usize) -> Option<&String> { self.0.get(i) }
}

let n = Names(vec!["Ada".into()]);
println!("{:?} {}", n.first(), Names::CAPACITY);   // Some("Ada") 100

fn print_first<C: Container>(c: &C) where C::Item: std::fmt::Debug {
    println!("{:?}", c.first());
}

fn sum_bytes<I: Iterator<Item = u8>>(it: I) -> u32 { it.map(u32::from).sum() }
println!("{}", sum_bytes("AB".bytes()));    // 131

// Generic trait: one type implements it many times
struct Celsius(f64);
impl From<f64> for Celsius { fn from(v: f64) -> Self { Celsius(v) } }
impl From<i32> for Celsius { fn from(v: i32) -> Self { Celsius(v as f64) } }

trait Pilot { fn fly(&self) -> &str; }
trait Wizard { fn fly(&self) -> &str; }
struct Human;
impl Pilot for Human { fn fly(&self) -> &str { "captain speaking" } }
impl Wizard for Human { fn fly(&self) -> &str { "up!" } }
let h = Human;
println!("{}", Pilot::fly(&h));             // captain speaking
println!("{}", <Human as Wizard>::fly(&h)); // up!
```

CODE EXPLAINED

- → Each implementor picks `Item` once
- → `Names::CAPACITY` reads an associated const
- → `Iterator<Item = u8>` bounds the associated type
- → `<Human as Wizard>::fly` picks between same-named methods

COMING FROM…

- ↔ **C++** member typedefs: `std::vector<T>::value_type`
- ↔ **Swift** `associatedtype` in protocols (very similar)
- ↔ **Java** no equivalent; extra generic parameters instead

THE RUST WAY

- ⇄ `trait Container<T>` with one impl per type → associated `type Item`
- ⇄ magic numbers per type → associated consts

ASSOCIATED TYPE VS GENERIC

ITEM	TYPE ITEM	TRAIT<T>
Impls per type	One	Many
Caller specifies	No	Often
Example	Iterator, Deref	From<T>, Add<Rhs>
Bound syntax	I: Iterator<Item = u8>	T: From<u8>

QUALIFIED SYNTAX

FORM	USE
Trait::f(&x)	Pick a trait
<T as Trait>::f()	Fully qualified
<T as Trait>::Item	Associated type
T::CONST	Associated const

⚠ COMMON MISTAKES

- Using a generic parameter where an associated type fits (forces annotations)
- Ambiguous method calls when two traits share a name (E0034)
- Forgetting to set the associated type in the impl

✓ TIPS & BEST PRACTICES

- Ask "can a type implement this more than once?" to choose
- Use `where C::Item: Bound` for associated type bounds
- Use associated consts for per-type configuration

QUICK SUMMARY

- `type Item` — one per impl
- `const X: T` in traits
- `Iterator<Item = T>` bounds
- `<T as Trait>::f`

Box<T>

OWNING HEAP POINTER

Box<T> puts a value on the heap and owns it: when the Box is dropped, the value is dropped and the memory freed. Use it for **recursive types** (whose size would otherwise be infinite), **trait objects** (Box<dyn Trait>), and moving large values cheaply. A Box derefs to T, so you use it almost like the value itself.

KEY POINTS

- `*b` moves the value out of a Box (for owned boxes)
- `Box<[T]>` / `Box<str>` are fixed-size heap buffers
- Box has no runtime overhead beyond the allocation
- `Box::leak` gives a `&'static mut T` (memory never freed)

UNDER THE HOOD

- `Box<T>` is a single non-null pointer (8 bytes); `Box<dyn T>` / `Box<[T]>` are fat (16)
- `Option<Box<T>>` uses the null niche: still 8 bytes
- `Box::new(x)` may build x on the stack, then copy it to the heap
- Drop calls `T::drop`, then frees with the global allocator

USING BOX

```
let b = Box::new(5);                        // i32 on the heap
println!("{}", *b + 1);                     // 6
println!("{}", b);                          // Display passes through: 5

#[derive(Debug)]
enum List { Cons(i32, Box<List>), Nil }     // recursive: needs indirection
use List::{Cons, Nil};
let list = Cons(1, Box::new(Cons(2, Box::new(Nil))));
println!("{list:?}");                       // Cons(1, Cons(2, Nil))

#[derive(Debug)]
struct Node { val: i32, left: Option<Box<Node>>, right: Option<Box<Node>> }
let leaf = |v| Some(Box::new(Node { val: v, left: None, right: None }));
let root = Node { val: 2, left: leaf(1), right: leaf(3) };
fn sum(n: &Option<Box<Node>>) -> i32 {
    n.as_ref().map_or(0, |b| b.val + sum(&b.left) + sum(&b.right))
}
println!("{}", root.val + sum(&root.left) + sum(&root.right));   // 6

trait Animal { fn speak(&self) -> &str; }
struct Cat;
impl Animal for Cat { fn speak(&self) -> &str { "meow" } }
let pet: Box<dyn Animal> = Box::new(Cat);
println!("{}", pet.speak());

let big = Box::new([0u8; 1_000_000]);       // 1 MB moved by pointer, not copied
let moved = big;                            // copies 8 bytes

let s: Box<str> = "fixed".into();           // no spare capacity
let v: Box<[i32]> = vec![1, 2, 3].into_boxed_slice();
let inner: i32 = *Box::new(7);              // move out of the box
let forever: &'static mut Vec<u8> = Box::leak(Box::new(Vec::new()));
```

CODE EXPLAINED

- → `*b` reads through the Box; Display works directly
- → `Box<List>` gives the enum a known size
- → Trees use `Option<Box<Node>>` for optional children
- → Moving a Box copies only the pointer

COMING FROM…

- ↔ **C++** `std::unique_ptr<T>` (same ownership semantics)
- ↔ **Java / Python / Go** every object is already heap-allocated and GC-managed
- ↔ **C** `malloc` + `free`, done automatically by Box's drop

THE RUST WAY

- ⇄ `Box<Vec<T>>` → `Vec<T>` (already on the heap)
- ⇄ `Box<T>` for small Copy values → plain `T`

USES

USE	EXAMPLE
Recursive type	Cons(i32, Box<List>)
Trait object	Box<dyn Animal>
Large value	Box<[u8; 1_000_000]>
Fixed buffer	Box<[T]>, Box<str>
Error type	Box<dyn Error>
Leak to static	Box::leak(b)

SIZES (64-BIT)

TYPE	BYTES
Box<T>	8
Option<Box<T>>	8
Box<[T]>	16
Box<dyn Trait>	16
Vec<T>	24

⚠ COMMON MISTAKES

- Boxing everything out of habit
- Recursive struct without Box (E0072: recursive type has infinite size)
- Very large `Box::new([0; N])` overflowing the stack first (use `vec!`)

✓ TIPS & BEST PRACTICES

- Use `vec![0; n].into_boxed_slice()` for big zeroed buffers
- Box the largest enum variant to shrink the enum
- Prefer `Box<dyn Error + Send + Sync>` in threaded code

QUICK SUMMARY

- `Box<T>` — owned heap value
- recursive types
- `Box<dyn Trait>`
- drop frees the memory

Rc<T> & Weak

Rc::new · Rc::clone · strong_count · single-thread shared ownership · Weak / upgrade() · reference cycles · parent pointers

REFERENCE-COUNTED SHARED OWNERSHIP

Rc<T> lets several owners share one read-only value on a single thread. Each `Rc::clone` increments a count; the value is dropped when the last owner goes away. `Weak<T>` is a non-owning pointer that does not keep the value alive, used for parent links and caches to avoid **reference cycles**, which would leak memory.

KEY POINTS

- `Rc` gives shared `&T` only; add `RefCell` for mutation
- `Rc` is not `Send`: use `Arc` across threads
- `Weak::upgrade()` returns `Option<Rc<T>>`
- Cycles of strong `Rc`s are never freed

UNDER THE HOOD

- `Rc::new` allocates one block: strong count, weak count, then the value
- `Rc<T>` is one pointer (8 bytes); clone is a non-atomic increment
- Drop decrements; at strong = 0 the value drops, at weak = 0 the block frees
- `Arc` is the same layout with atomic counts

RC AND WEAK

```
use std::rc::{Rc, Weak};
use std::cell::RefCell;

let shared = Rc::new(String::from("config"));
let a = Rc::clone(&shared);                 // count 2 (cheap)
let b = Rc::clone(&shared);                 // count 3
println!("{}", Rc::strong_count(&shared));  // 3
drop(a);
println!("{}", Rc::strong_count(&shared));  // 2

#[derive(Debug)]
struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,            // weak: child doesn't own parent
    children: RefCell<Vec<Rc<Node>>>,       // strong: parent owns children
}
fn node(name: &str, kids: Vec<Rc<Node>>) -> Rc<Node> {
    Rc::new(Node { name: name.into(), parent: RefCell::new(Weak::new()),
                   children: RefCell::new(kids) })
}
let leaf = node("leaf", vec![]);
let branch = node("branch", vec![Rc::clone(&leaf)]);
*leaf.parent.borrow_mut() = Rc::downgrade(&branch);
if let Some(p) = leaf.parent.borrow().upgrade() {
    println!("{} -> {}", leaf.name, p.name);    // leaf -> branch
}
println!("{} {}", Rc::strong_count(&branch), Rc::weak_count(&branch));   // 1 1
drop(branch);
println!("{}", leaf.parent.borrow().upgrade().is_none());   // true

let mut unique = Rc::new(5);
*Rc::make_mut(&mut unique) += 1;            // clone-on-write if shared
println!("{:?}", Rc::try_unwrap(unique));   // Ok(6): last owner gets value
```

CODE EXPLAINED

- → `Rc::clone` shares the same String; no copy
- → Children hold `Weak` parents, so there is no cycle
- → `upgrade()` returns None once the parent is dropped
- → `make_mut` clones only when other owners exist

COMING FROM…

- ↔ **C++** `std::shared_ptr` / `std::weak_ptr` (atomic counts by default)
- ↔ **Python** reference counting + a cycle collector; Rust has no collector
- ↔ **Java / Go** tracing GC handles cycles automatically

THE RUST WAY

- ⇄ `shared.clone()` → `Rc::clone(&shared)` (shows it is cheap)
- ⇄ parent and child both holding `Rc` → parent strong, child `Weak`

API

CALL	DOES
`Rc::new(v)`	Allocate, count 1
`Rc::clone(&rc)`	Count + 1
`Rc::strong_count`	Owners
`Rc::downgrade(&rc)`	→ Weak
`weak.upgrade()`	Option<Rc<T>>
`Rc::ptr_eq`	Same allocation?
`Rc::make_mut`	Clone-on-write
`Rc::try_unwrap`	Take if unique

⚠ COMMON MISTAKES

- Strong cycles (parent ↔ child) leaking memory
- Expecting `Rc<T>` to allow mutation (needs `RefCell`)
- Sending an `Rc` to another thread (E0277: Rc cannot be sent)

✓ TIPS & BEST PRACTICES

- Use `Rc<str>` / `Rc<[T]>` for shared immutable data
- Consider arena or index-based graphs (`Vec<Node>` + ids)
- Use `Arc` only when threads are involved

QUICK SUMMARY

- `Rc<T>` — shared, single thread
- `Rc::clone` bumps a count
- `Weak` — no ownership
- avoid strong cycles

Cell, RefCell & Interior Mutability

interior mutability · `Cell` `get` / `set` · `RefCell` `borrow()` / `borrow_mut()` · runtime borrow panics · `Rc<RefCell<T>>` · `OnceCell`

MUTATING THROUGH A SHARED REFERENCE

Interior mutability lets you change data behind a &T. `Cell<T>` swaps whole values (best for Copy types) with no borrowing at all. `RefCell<T>` hands out borrows checked **at runtime**: breaking the rules panics instead of failing to compile. `Rc<RefCell<T>>` is the classic shared, mutable, single-threaded value.

KEY POINTS

- Neither Cell nor RefCell is `Sync` (single thread only)
- Thread-safe versions: `Mutex`, `RwLock`, atomics
- `try_borrow_mut` returns a Result instead of panicking
- `OnceCell` sets a value once, then reads it

CELL AND REFCELL

```
use std::cell::{Cell, OnceCell, RefCell};
use std::rc::Rc;

struct Counter { hits: Cell<u32> }
impl Counter {
    fn hit(&self) { self.hits.set(self.hits.get() + 1); } // &self, yet mutates
}
let c = Counter { hits: Cell::new(0) };
c.hit();
c.hit();
println!("{}", c.hits.get());               // 2

let log = RefCell::new(Vec::new());
log.borrow_mut().push("start");             // mutable borrow, released at ;
log.borrow_mut().push("end");
println!("{:?}", log.borrow());             // ["start", "end"]
println!("{}", log.borrow().len());         // many shared borrows OK

let r1 = log.borrow_mut();
// let r2 = log.borrow_mut();               // panic: already mutably borrowed
println!("{}", log.try_borrow().is_err());  // true
drop(r1);

let shared = Rc::new(RefCell::new(vec![1, 2]));
let other = Rc::clone(&shared);
other.borrow_mut().push(3);                 // mutate through one owner
println!("{:?}", shared.borrow());          // [1, 2, 3] seen by all

struct Lazy { value: OnceCell<String> }
impl Lazy {
    fn get(&self) -> &str {
        self.value.get_or_init(|| { println!("computing"); "ready".into() })
    }
}
let l = Lazy { value: OnceCell::new() };
println!("{} {}", l.get(), l.get());        // computing (once), ready ready

let flag = Cell::new(false);
let old = flag.replace(true);               // swap in, return old
```

CODE EXPLAINED

- → `Cell::set` changes the value through `&self`
- → Each `borrow_mut()` ends at the end of its statement
- → Two live `borrow_mut`s panic at runtime
- → `Rc<RefCell<…>>` lets every owner mutate the shared Vec

COMING FROM…

- ↔ **C++** `mutable` members; no runtime borrow checks
- ↔ **Java / Python** everything is mutable through any reference
- ↔ **Go** no equivalent; any pointer can mutate

THE RUST WAY

- ⇄ `RefCell` for Copy fields → `Cell` (no borrow tracking)
- ⇄ `Rc<RefCell<T>>` everywhere → restructure ownership or pass `&mut`

TYPES

TYPE	ACCESS	CHECK	THREADS
`Cell<T>`	`get / set / replace`	None needed	No
`RefCell<T>`	`borrow / borrow_mut`	Runtime (panic)	No
`OnceCell<T>`	`get_or_init`	Set once	No
`Mutex<T>`	`lock()`	Runtime (blocks)	Yes
`RwLock<T>`	`read / write`	Runtime (blocks)	Yes
`OnceLock<T>`	`get_or_init`	Set once	Yes

⚠ COMMON MISTAKES

- Holding a `borrow()` while calling a function that does `borrow_mut()` (panic)
- Using RefCell to dodge every borrow error
- Expecting `Rc<RefCell<T>>` to work across threads

✓ TIPS & BEST PRACTICES

- Keep RefCell borrows short; drop guards quickly
- Use `try_borrow_mut` where re-entrancy is possible
- Prefer `Cell` for counters and flags

QUICK SUMMARY

- mutate through `&T`
- `Cell` — whole-value swaps
- `RefCell` — runtime-checked borrows
- `Rc<RefCell<T>>` — shared + mutable

Deref & Drop

SMART POINTER TRAITS

`Deref` lets a type act like a reference: `*x` and method calls pass through to the target, and **deref coercion** turns `&String` into `&str` or `&Box<T>` into `&T` automatically. `Drop` runs cleanup code when a value goes out of scope, the basis of **RAII**: files close, locks release and memory frees without manual calls.

KEY POINTS

- You cannot call `x.drop()` directly; use `drop(x)`
- Fields drop after the struct's own `drop`, in declaration order
- Locals drop in reverse order of declaration
- Implement Deref only for smart-pointer-like types

DEREF AND DROP

```
use std::ops::{Deref, DerefMut};

struct Wrapper(Vec<i32>);
impl Deref for Wrapper {
    type Target = Vec<i32>;
    fn deref(&self) -> &Vec<i32> { &self.0 }
}
impl DerefMut for Wrapper {
    fn deref_mut(&mut self) -> &mut Vec<i32> { &mut self.0 }
}
let mut w = Wrapper(vec![3, 1, 2]);
w.push(4);                                  // Vec method via DerefMut
w.sort();                                   // slice method via two derefs
println!("{} {:?}", w.len(), *w);           // 4 [1, 2, 3, 4]

fn takes_str(s: &str) -> usize { s.len() }
let boxed: Box<String> = Box::new("hello".into());
println!("{}", takes_str(&boxed));          // &Box<String> → &String → &str

struct Guard(&'static str);
impl Drop for Guard {
    fn drop(&mut self) { println!("drop {}", self.0); }
}
{
    let _a = Guard("a");
    let _b = Guard("b");
    println!("in scope");
}                                           // drop b, then drop a

let early = Guard("early");
drop(early);                                // drop early (now)
println!("after early");

let _ = Guard("temp");                      // `_` drops immediately: drop temp
let _keep = Guard("kept");                  // lives to end of scope
```

CODE EXPLAINED

- → `Deref` exposes Vec methods on Wrapper
- → Coercion follows `&Box<String>` down to `&str`
- → Locals drop in reverse order: b, then a
- → `let _ =` drops at once; `_keep` lives to scope end

COMING FROM…

- ↔ **C++** destructors and RAII (same idea); `operator*` / `operator->`
- ↔ **Java** try-with-resources / `AutoCloseable`
- ↔ **Python** `with` context managers; `__del__` is unreliable

THE RUST WAY

- ⇄ explicit `close()` methods callers can forget → `impl Drop`
- ⇄ `impl Deref` to fake inheritance → composition + explicit methods

COERCIONS

FROM	TO
`&String`	&str
`&Vec<T>`	&[T]
`&Box<T>`	&T
`&Rc<T>` / `&Arc<T>`	&T
`&mut T`	&T
`&[T; N]`	&[T]

DROP ORDER

CASE	ORDER
`Local variables`	Reverse declaration
`Struct fields`	Declaration order
`Vec elements`	First to last
`Temporaries`	End of statement
`let _ = x`	Immediately

⚠ COMMON MISTAKES

- `let _ = lock.lock()` dropping the guard instantly
- Calling `x.drop()` (E0040: explicit destructor call)
- Panicking inside `drop` (can abort the process)

✓ TIPS & BEST PRACTICES

- Name guards `_guard` so they live to scope end
- Use `drop(x)` to release locks and files early
- Keep `Drop` impls small and infallible

QUICK SUMMARY

- `Deref` — act like `&T`
- deref coercion
- `Drop` — RAII cleanup
- reverse declaration drop order

Sheet 076

Cow & Choosing a Pointer

Cow<'a, str> · to_mut() / into_owned() · Box vs Rc vs Arc · Rc<RefCell> vs Arc<Mutex> · Pin preview

CLONE ON WRITE AND POINTER CHOICE

Cow<'a, T> ("clone on write") holds **either** a borrowed &T or an owned value, so a function can return the input unchanged without allocating and only allocate when it must modify. The rest of this sheet is a decision guide for Box, Rc, Arc and their interior-mutability pairings.

KEY POINTS

- `Cow::Borrowed(&x)` / `Cow::Owned(v)`
- `to_mut()` clones only if currently borrowed
- `Pin<P>` guarantees a value will not move (needed by async futures)
- Start with plain ownership; reach for pointers when needed

COW AND POINTER CHOICES

```
use std::borrow::Cow;

fn normalize(input: &str) -> Cow<'_, str> {
    if input.contains(' ') {
        Cow::Owned(input.replace(' ', "_"))  // allocate only if needed
    } else {
        Cow::Borrowed(input)                 // zero-copy path
    }
}
let a = normalize("no_spaces");
let b = normalize("has spaces");
println!("{a} {b}");                         // no_spaces has_spaces
println!("{}", matches!(a, Cow::Borrowed(_)));   // true

let mut c: Cow<[i32]> = Cow::Borrowed(&[1, 2, 3]);
c.to_mut().push(4);                          // clones into a Vec here
let owned: Vec<i32> = c.into_owned();

let lossy = String::from_utf8_lossy(b"ok");  // std returns Cow<str>

use std::rc::Rc;
use std::cell::RefCell;
use std::sync::{Arc, Mutex};

let single_owner = Box::new(5);                      // one owner
let shared_read = Rc::new(vec![1, 2]);               // many owners, 1 thread
let shared_mut = Rc::new(RefCell::new(0));           // + mutation
*shared_mut.borrow_mut() += 1;
let thread_read = Arc::new(String::from("cfg"));     // many threads
let thread_mut = Arc::new(Mutex::new(0));            // + mutation
*thread_mut.lock().unwrap() += 1;

use std::pin::Pin;
let pinned: Pin<Box<i32>> = Box::pin(10);   // won't move again
```

CODE EXPLAINED

- → `normalize` borrows when no change is needed
- → `to_mut()` turns a borrowed Cow into an owned one
- → std APIs like `from_utf8_lossy` return Cow
- → Each pointer adds one capability: sharing, threads, mutation

COMING FROM…

- ↔ **C++** no std Cow; copy-on-write strings were removed in C++11
- ↔ **Swift** arrays/strings are copy-on-write automatically
- ↔ **Python / Java** immutable strings; sharing is via the GC

THE RUST WAY

- ⇄ returning `String` from a function that usually doesn't change input → `Cow<str>`
- ⇄ `Arc<Mutex<T>>` in single-threaded code → `Rc<RefCell<T>>` or plain `&mut`

CHOOSING A POINTER

NEED	USE
One owner, heap	Box<T>
Many owners, one thread	Rc<T>
… and mutation	Rc<RefCell<T>>
Many owners, threads	Arc<T>
… and mutation	Arc<Mutex<T>> / Arc<RwLock<T>>
Maybe owned	Cow<'a, T>
Must not move	Pin<Box<T>>
Non-owning back link	Weak<T>

⚠ COMMON MISTAKES

- Reaching for `Rc<RefCell>` before trying plain ownership
- Using `Arc` and `Mutex` where threads never share data
- Returning `Cow` from APIs that always allocate anyway

✓ TIPS & BEST PRACTICES

- Accept `impl Into<Cow<'a, str>>` for flexible constructors
- Use indices into a Vec instead of pointer graphs
- Learn `Pin` when writing futures or self-referential types

QUICK SUMMARY

- `Cow` — borrowed or owned
- `to_mut()` clones lazily
- Box · Rc · Arc ladder
- RefCell (1 thread) · Mutex (threads)

Reading Files

`fs::read_to_string` · `File::open` · `BufReader` & `lines()` · `read_to_end` · `io::Result` · `?` in file code

GETTING DATA IN

For small files, `fs::read_to_string` reads everything into a `String` and `fs::read` into `Vec<u8>`. For large files, wrap a `File` in a `BufReader` and iterate `lines()` so memory stays flat. All I/O returns `io::Result<T>`, so `?` handles errors cleanly.

READING

```
use std::fs::{self, File};
use std::io::{self, BufRead, BufReader, Read};

fn main() -> io::Result<()> {
    let text = fs::read_to_string("notes.txt")?;      // whole file → String
    println!("{} lines", text.lines().count());

    let bytes: Vec<u8> = fs::read("logo.png")?;        // whole file → bytes
    println!("{} bytes", bytes.len());

    let file = File::open("big.log")?;
    let reader = BufReader::new(file);
    for (n, line) in reader.lines().enumerate() {     // streams line by line
        let line = line?;
        if line.contains("ERROR") { println!("{}: {line}", n + 1); }
    }

    let mut f = File::open("data.bin")?;
    let mut header = [0u8; 4];
    f.read_exact(&mut header)?;                       // exactly 4 bytes
    let mut rest = Vec::new();
    f.read_to_end(&mut rest)?;

    match fs::read_to_string("missing.txt") {
        Ok(s) => println!("{s}"),
        Err(e) if e.kind() == io::ErrorKind::NotFound => println!("no file"),
        Err(e) => return Err(e),
    }

    let nums: Vec<i64> = fs::read_to_string("nums.txt")?
        .lines()
        .filter_map(|l| l.trim().parse().ok())
        .collect();
    println!("sum = {}", nums.iter().sum::<i64>());
    Ok(())
}
```

KEY POINTS

- `read_to_string` fails on invalid UTF-8
- `lines()` strips `\n` and `\r\n`; each line is `io::Result<String>`
- Files close automatically when dropped
- `io::ErrorKind::NotFound` lets you match specific errors

CODE EXPLAINED

- → `read_to_string` is the one-liner for small text files
- → `BufReader::lines` reads one line at a time
- → `read_exact` fills a fixed buffer or errors
- → Match on `e.kind()` to handle a missing file

COMING FROM…

- ↔ **Python** `open(p).read()` / `for line in open(p):`
- ↔ **Java** `Files.readString(path)` / `Files.lines(path)`
- ↔ **Go** `os.ReadFile` / `bufio.Scanner`

THE RUST WAY

- ⇄ `read_to_string` on multi-GB logs → `BufReader::lines()`
- ⇄ `.unwrap()` on file opens → `?` with a `Result` return

FUNCTIONS

CALL	RETURNS
`fs::read_to_string(p)`	io::Result<String>
`fs::read(p)`	io::Result<Vec<u8>>
`File::open(p)`	io::Result<File>
`BufReader::new(f).lines()`	Iterator<io::Result<String>>
`f.read_to_end(&mut v)`	io::Result<usize>
`f.read_exact(&mut buf)`	io::Result<()>

ERRORKIND

KIND	MEANING
`NotFound`	No such file
`PermissionDenied`	No access
`AlreadyExists`	create_new failed
`InvalidData`	Bad UTF-8 etc.
`UnexpectedEof`	File too short

⚠ COMMON MISTAKES

- Reading huge files into memory at once
- Unbuffered small reads on a `File` (many syscalls)
- Ignoring the `io::Result` inside `lines()`

✓ TIPS & BEST PRACTICES

- Use `BufReader` for anything read piece by piece
- Add the path to error messages (`map_err` / anyhow `context`)
- Use `memmap2` for very large read-only files

QUICK SUMMARY

- `fs::read_to_string` — small files
- `BufReader::lines` — big files
- `io::Result` + `?`
- match `ErrorKind`

Writing Files

`fs::write` · `File::create` · `OpenOptions` (append, create) · `BufWriter` · `write!` / `writeln!` · `flush()` / `sync_all`

GETTING DATA OUT

`fs::write` creates or truncates a file and writes everything in one call. `File::create` gives a handle for repeated writes; wrap it in `BufWriter` to batch small writes. `OpenOptions` controls append, create-new and truncate behavior. `write!`/`writeln!` format directly into any writer.

KEY POINTS

- `File::create` truncates an existing file
- `BufWriter` flushes on drop but **ignores errors** there: call `flush()`
- `sync_all()` forces data to disk
- Write to a temp file and rename for atomic replacement

WRITING

```rust
use std::fs::{self, File, OpenOptions};
use std::io::{self, BufWriter, Write};

fn main() -> io::Result<()> {
    fs::write("hello.txt", "Hello, file!\n")?;      // create or overwrite
    fs::write("data.bin", [0xDE, 0xAD, 0xBE, 0xEF])?;

    let mut f = File::create("report.txt")?;        // truncates
    writeln!(f, "Report")?;
    writeln!(f, "{:<10}{:>5}", "apples", 12)?;
    f.write_all(b"raw bytes\n")?;

    let mut log = OpenOptions::new()
        .create(true)
        .append(true)
        .open("app.log")?;                          // append mode
    writeln!(log, "started")?;

    let out = File::create("numbers.csv")?;
    let mut w = BufWriter::new(out);                // batch small writes
    for i in 0..10_000 {
        writeln!(w, "{i},{}", i * i)?;
    }
    w.flush()?;                                     // surface write errors

    match OpenOptions::new().write(true).create_new(true).open("once.lock") {
        Ok(_) => println!("created lock"),
        Err(e) if e.kind() == io::ErrorKind::AlreadyExists => println!("busy"),
        Err(e) => return Err(e),
    }

    let tmp = "config.toml.tmp";                    // atomic replace
    fs::write(tmp, "port = 8080\n")?;
    fs::rename(tmp, "config.toml")?;

    let f2 = File::create("important.dat")?;
    f2.sync_all()?;                                 // fsync to disk
    Ok(())
}
```

CODE EXPLAINED

→ `fs::write` takes any bytes or string

→ `writeln!` formats straight into the file

→ `OpenOptions` with `append(true)` adds to the end

→ Write-then-rename keeps readers from seeing half a file

COMING FROM…

↔ **Python** `open(p, "w")` / `"a"` modes with `with`

↔ **Java** `Files.writeString`, `BufferedWriter`, `StandardOpenOption.APPEND`

↔ **Go** `os.WriteFile`, `os.OpenFile(p, os.O_APPEND|…)`, `bufio.Writer`

THE RUST WAY

⇄ many `write!` calls on a raw File → `BufWriter`

⇄ relying on drop to flush → call `flush()` and check it

OPENOPTIONS

OPTION	EFFECT
`read(true)`	Readable
`write(true)`	Writable
`append(true)`	Write at end
`truncate(true)`	Empty first
`create(true)`	Create if missing
`create_new(true)`	Fail if exists

WRITE CALLS

CALL	USE
`fs::write(p, data)`	Whole file
`f.write_all(&bytes)`	All bytes
`write!(f, …)`	Formatted
`w.flush()`	Push buffer
`f.sync_all()`	fsync

⚠ COMMON MISTAKES

- Forgetting `flush()` on BufWriter and losing errors
- Using `write` instead of `write_all` (may write partially)
- Opening with `File::create` when appending was intended

✓ TIPS & BEST PRACTICES

- Use temp file + `rename` for config and state files
- Use `create_new(true)` for lock files
- Use `tempfile` crate for scratch files

QUICK SUMMARY

- `fs::write` — one call
- `File::create` truncates
- `OpenOptions` — append / create_new
- `BufWriter` + `flush()`

Paths & Directories

WORKING WITH THE FILE SYSTEM

`Path` is a borrowed path (like `&str`) and `PathBuf` is owned and growable (like `String`). They handle separators per platform. `join` builds paths, and methods like `extension`, `file_name` and `parent` take them apart. `std::fs` creates, lists, renames and deletes files and directories.

KEY POINTS

- Paths are not always UTF-8: `to_str()` returns Option
- `display()` prints a path lossily
- Accept `impl AsRef<Path>` to take &str, String or PathBuf
- `read_dir` order is not sorted

PATHS AND DIRECTORIES

```
use std::fs;
use std::path::{Path, PathBuf};

let p = Path::new("/home/ada/docs/report.final.pdf");
println!("{:?}", p.file_name());            // Some("report.final.pdf")
println!("{:?}", p.file_stem());            // Some("report.final")
println!("{:?}", p.extension());            // Some("pdf")
println!("{:?}", p.parent());               // Some("/home/ada/docs")

let mut out = PathBuf::from("build");
out.push("assets");                         // build/assets
out.push("app.js");
out.set_extension("min.js");                // build/assets/app.min.js
println!("{}", out.display());
let cfg = Path::new("config").join("app.toml");   // config/app.toml

fs::create_dir_all("build/assets")?;        // mkdir -p
fs::write(&out, "…")?;
println!("{}", out.exists());               // true

for entry in fs::read_dir("build/assets")? {
    let entry = entry?;
    let meta = entry.metadata()?;
    println!("{:?} {} bytes", entry.file_name(), meta.len());
}

let mut rs_files: Vec<PathBuf> = fs::read_dir("src")?
    .filter_map(|e| e.ok().map(|e| e.path()))
    .filter(|p| p.extension().is_some_and(|x| x == "rs"))
    .collect();
rs_files.sort();
fs::rename(&out, "build/app.js")?;
fs::copy("build/app.js", "build/app.bak")?;
fs::remove_file("build/app.bak")?;
fs::remove_dir_all("build")?;               // rm -rf: careful
```

CODE EXPLAINED

- → `file_stem` drops only the last extension
- → `push` appends components with the right separator
- → `create_dir_all` makes all missing parents
- → `read_dir` yields entries with names and metadata

COMING FROM…

- ↔ **Python** `pathlib.Path` (`/` operator, `.suffix`, `.iterdir()`)
- ↔ **Java** `Path.of`, `resolve`, `Files.list`, `Files.createDirectories`
- ↔ **Go** `path/filepath.Join`, `os.ReadDir`, `os.MkdirAll`

THE RUST WAY

- ⇄ `format!("{}/{}", dir, name)` → `Path::new(dir).join(name)`
- ⇄ `fn f(p: &str)` for paths → `fn f(p: impl AsRef<Path>)`

PATH METHODS

METHOD	RETURNS
`join(x)`	PathBuf
`parent()`	Option<&Path>
`file_name()`	Option<&OsStr>
`file_stem / extension`	Option<&OsStr>
`exists / is_file / is_dir`	bool
`to_str()`	Option<&str>
`display()`	Printable
`components()`	Iterator

FS FUNCTIONS

FUNCTION	SHELL
`create_dir_all`	mkdir -p
`read_dir`	ls
`metadata`	stat
`copy`	cp
`rename`	mv
`remove_file`	rm
`remove_dir_all`	rm -rf

⚠ COMMON MISTAKES

- Building paths with string concatenation and `/`
- Unwrapping `to_str()` on non-UTF-8 paths
- Expecting `read_dir` results in sorted order

✓ TIPS & BEST PRACTICES

- Use `walkdir` for recursive traversal
- Use `dirs` / `directories` crates for home and config folders
- Sort `read_dir` output when order matters

QUICK SUMMARY

- `Path` borrowed · `PathBuf` owned
- `join` / `push`
- `extension` / `file_stem`
- `fs::create_dir_all` / `read_dir`

stdin & stdout

CONSOLE INPUT AND OUTPUT

`io::stdin().read_line(&mut s)` reads one line **including** the trailing newline into a String, so call `trim()` before parsing. `stdin().lines()` iterates every line until EOF, which suits filters and pipes. For heavy output, lock stdout once and wrap it in a `BufWriter`.

KEY POINTS

- `read_line` appends; clear the buffer when reusing it
- `read_line` returns `Ok(0)` at EOF
- `print!` needs `flush()` before waiting for input
- stdout is line-buffered only when attached to a terminal

CONSOLE I/O

```
use std::io::{self, BufRead, BufWriter, Write};

fn main() -> io::Result<()> {
    print!("Your name: ");
    io::stdout().flush()?;                  // show the prompt now
    let mut name = String::new();
    io::stdin().read_line(&mut name)?;      // includes '\n'
    let name = name.trim();
    println!("Hello, {name}!");

    print!("Age: ");
    io::stdout().flush()?;
    let mut buf = String::new();
    io::stdin().read_line(&mut buf)?;
    match buf.trim().parse::<u32>() {
        Ok(age) => println!("Next year: {}", age + 1),
        Err(_) => eprintln!("not a number: {:?}", buf.trim()),
    }

    // Filter mode: cat input.txt | app
    let stdin = io::stdin();
    let mut total = 0i64;
    for line in stdin.lock().lines() {      // until EOF (Ctrl-D / Ctrl-Z)
        let line = line?;
        if let Ok(n) = line.trim().parse::<i64>() { total += n; }
    }

    let stdout = io::stdout();
    let mut out = BufWriter::new(stdout.lock());   // fast bulk output
    for i in 0..5 { writeln!(out, "row {i}")?; }
    writeln!(out, "total {total}")?;
    out.flush()?;

    let all: String = io::read_to_string(io::stdin())?;   // whole input
    eprintln!("{} bytes read", all.len());  // diagnostics to stderr
    Ok(())
}
```

CODE EXPLAINED

- → `flush()` makes `print!` text appear before input
- → `read_line` keeps the newline, so trim before parsing
- → `lock().lines()` reads until end of input
- → Diagnostics go to stderr so stdout stays clean for pipes

COMING FROM…

- ↔ **Python** `input("Name: ")` and `for line in sys.stdin:`
- ↔ **Java** `Scanner(System.in)` / `BufferedReader(InputStreamReader(System.in))`
- ↔ **C++** `std::getline(std::cin, s)` / `std::cout`

THE RUST WAY

- ⇄ parsing without `trim()` → `buf.trim().parse()`
- ⇄ many `println!` in a loop → locked `BufWriter`

INPUT

CALL	RETURNS
`stdin().read_line(&mut s)`	io::Result<usize>
`stdin().lines()`	Iterator<io::Result<String>>
`stdin().lock()`	StdinLock (BufRead)
`io::read_to_string(stdin())`	io::Result<String>

OUTPUT

CALL	TARGET
`print! / println!`	stdout
`eprint! / eprintln!`	stderr
`stdout().flush()`	Push prompt
`BufWriter::new(stdout().lock())`	Fast bulk
`writeln!(out, …)`	Any writer

⚠ COMMON MISTAKES

- Parsing input with the newline still attached
- Reusing a buffer with `read_line` without `clear()`
- Prompt invisible because `print!` was not flushed

✓ TIPS & BEST PRACTICES

- Use `dialoguer` / `inquire` for interactive prompts
- Use `is_terminal()` (`std::io::IsTerminal`) to detect pipes
- Exit with a non-zero code on errors (sheet 082)

QUICK SUMMARY

- `read_line` keeps `\n` → trim
- `stdin().lines()` until EOF
- flush before input
- lock + BufWriter for speed

Sheet 081

Serde & JSON

#[derive(Serialize, Deserialize)] · serde_json::to_string_pretty · from_str · Value / json! · #[serde(rename_all, default, skip)] · TOML / CSV via serde

SERIALIZATION FRAMEWORK

Serde converts Rust types to and from data formats. Derive `Serialize` and `Deserialize` on your structs, then use a format crate: `serde_json`, `toml`, `serde_yaml`, `csv`. Attributes rename fields, supply defaults and skip values. For unknown shapes, `serde_json::Value` holds any JSON.

KEY POINTS

- `cargo add serde -F derive` and `cargo add serde_json`
- Missing fields are errors unless `Option` or `#[serde(default)]`
- Unknown fields are ignored unless `deny_unknown_fields`
- Enums serialize as externally tagged by default

SERDE

```rust
use serde::{Deserialize, Serialize};
use serde_json::{json, Value};

#[derive(Debug, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
struct User {
    user_id: u64,
    display_name: String,
    #[serde(default)]
    tags: Vec<String>,
    #[serde(skip_serializing_if = "Option::is_none")]
    email: Option<String>,
    #[serde(skip)]
    cache: u32,
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let input = r#"{"userId": 7, "displayName": "Ada", "extra": true}"#;
    let u: User = serde_json::from_str(input)?;     // tags → [], email → None
    println!("{u:?}");

    let out = serde_json::to_string(&u)?;
    println!("{out}");          // {"userId":7,"displayName":"Ada","tags":[]}
    println!("{}", serde_json::to_string_pretty(&u)?);

    let v: Value = serde_json::from_str(r#"{"a": {"b": [1, 2, 3]}}"#)?;
    println!("{}", v["a"]["b"][1]);                 // 2

    let doc = json!({ "name": "Ferris", "langs": ["rust"], "age": 9 });
    #[derive(Deserialize)]
    struct Config { port: u16, debug: bool }
    let cfg: Config = toml::from_str("port = 8080\ndebug = true")?;
    Ok(())
}
```

CODE EXPLAINED

- → `rename_all = "camelCase"` maps `user_id` ↔ `userId`
- → `#[serde(default)]` fills missing fields
- → `Value` indexing returns `Null` for missing keys
- → The same derive works with `toml` and other formats

COMING FROM…

- ↔ **Python** `json.loads` / `json.dumps` ; Pydantic for typed models
- ↔ **Java** Jackson annotations (`@JsonProperty` , `@JsonIgnore`)
- ↔ **Go** `encoding/json` with struct tags `json:"userId"`

THE RUST WAY

- ⇄ navigating `Value` everywhere → typed structs with derive
- ⇄ manual field renames → `#[serde(rename_all = …)]`

ATTRIBUTES

ATTRIBUTE	EFFECT
`rename_all = "camelCase"`	Field naming
`rename = "id"`	One field
`default`	Use Default if missing
`skip`	Never (de)serialize
`skip_serializing_if = "…"`	Omit when
`flatten`	Inline nested struct
`deny_unknown_fields`	Strict input
`tag = "type"`	Internally tagged enum

FORMAT CRATES

CRATE	FORMAT
`serde_json`	JSON
`toml`	TOML
`serde_yaml / serde_yml`	YAML
`csv`	CSV
`bincode / postcard`	Binary
`rmp-serde`	MessagePack

⚠ COMMON MISTAKES

- Forgetting `features = ["derive"]` on serde
- Expecting missing JSON fields to default without `default`
- Indexing `Value` and unwrapping types blindly

✓ TIPS & BEST PRACTICES

- Use `Option<T>` for truly optional fields
- Use `serde_with` for custom formats (dates as strings, etc.)
- Deserialize borrowed data with `&'a str` fields for zero-copy

QUICK SUMMARY

- derive Serialize / Deserialize
- `from_str` / `to_string_pretty`
- `#[serde(…)]` attributes
- `Value` / `json!` for dynamic

CLI Arguments & Environment

env::args() · env::var · clap #[derive(Parser)] · #[arg(short, long)] · subcommands · ExitCode / process::exit

BUILDING COMMAND-LINE TOOLS

`std::env::args()` gives raw arguments (the first is the program name). For anything beyond trivial, use **clap**: derive `Parser` on a struct and get parsing, validation, `--help` and `--version` for free. Read configuration from `env::var`, and report success or failure with `ExitCode`.

KEY POINTS

- `cargo add clap -F derive`
- Doc comments become help text
- `env::var` returns `Result<String, VarError>`
- `process::exit` skips destructors; prefer returning `ExitCode`

CLAP AND ENV

```rust
use clap::{Parser, Subcommand};
use std::process::ExitCode;

/// Search files for a pattern
#[derive(Parser, Debug)]
#[command(version, about)]
struct Cli {
    /// Pattern to look for
    pattern: String,
    /// Files to search
    #[arg(required = true)]
    files: Vec<std::path::PathBuf>,
    /// Max matches to show
    #[arg(short, long, default_value_t = 10)]
    max: usize,
    #[command(subcommand)]
    cmd: Option<Cmd>,
}

#[derive(Subcommand, Debug)]
enum Cmd {
    /// Count matches only
    Count,
    /// Export results
    Export { #[arg(long)] json: bool },
}

fn main() -> ExitCode {
    let cli = Cli::parse();                  // exits with help on bad input
    let level = std::env::var("LOG_LEVEL").unwrap_or_else(|_| "info".into());
    if cli.files.iter().any(|f| !f.exists()) {
        eprintln!("error: missing file");
        return ExitCode::from(2);
    }
    println!("{cli:?} level={level}");
    ExitCode::SUCCESS
}
```

CODE EXPLAINED

- → Plain fields are positional; `#[arg(short, long)]` makes `-m` / `--max`
- → `default_value_t` sets a typed default
- → `Subcommand` enums model `tool count` / `tool export`
- → Returning `ExitCode` sets the process status

COMING FROM…

- ↔ **Python** `argparse` / `click` / `typer` (typer is closest to clap derive)
- ↔ **Java** picocli annotations
- ↔ **Go** `flag` package / cobra for subcommands

THE RUST WAY

- ⇄ hand-parsing `env::args()` → clap derive
- ⇄ `process::exit(1)` deep in code → return errors up to `main`

CLAP ATTRIBUTES

ATTRIBUTE	EFFECT
`#[command(version, about)]`	Metadata from Cargo.toml
`#[arg(short, long)]`	-x / --long
`#[arg(default_value_t = v)]`	Default
`#[arg(env = "VAR")]`	Fallback to env
`#[arg(value_enum)]`	Enum choices
`#[command(subcommand)]`	Subcommands

STD::ENV

CALL	RETURNS
`env::args()`	Iterator<String>
`env::var("K")`	Result<String, VarError>
`env::var_os("K")`	Option<OsString>
`env::set_var (unsafe in 2024)`	Set
`env::current_dir()`	io::Result<PathBuf>

⚠ COMMON MISTAKES

- Forgetting that `args()` includes the program name
- Calling `env::set_var` in multi-threaded code (now `unsafe` in edition 2024)
- `process::exit` skipping file flushes

✓ TIPS & BEST PRACTICES

- Use `#[arg(env = "…")]` for env fallbacks
- Use exit code 2 for usage errors, 1 for failures
- Test CLIs with `assert_cmd`

QUICK SUMMARY

- `env::args()` — raw
- clap `#[derive(Parser)]`
- `env::var` → Result
- return `ExitCode`

Modules

`mod` · inline vs file modules (`foo.rs`, `foo/mod.rs`) · `use` · `crate::` / `super::` / `self::` · `pub use` re-exports · glob imports

ORGANIZING CODE

A crate is a tree of **modules** rooted at `main.rs` or `lib.rs`. `mod foo;` tells the compiler to load `foo.rs` (or `foo/mod.rs`), and `mod foo { … }` declares one inline. `use` brings paths into scope. Paths start from `crate::` (root), `super::` (parent) or `self::` (current module).

KEY POINTS

- Files are not modules until declared with `mod`
- Prefer `foo.rs` + `foo/` folder over `foo/mod.rs` (2018+)
- `pub use` re-exports items at a nicer path
- Glob `use x::*` is fine for preludes and tests

MODULE TREE

```
// src/
// ├── main.rs
// ├── config.rs            mod config;
// └── net/
//     ├── mod.rs  (or net.rs beside the folder)
//     └── http.rs          pub mod http;   (inside net)

// main.rs
mod config;                                 // loads src/config.rs
mod net;                                // loads src/net.rs or src/net/mod.rs

mod util {                                  // inline module
    pub fn clamp01(x: f64) -> f64 { x.clamp(0.0, 1.0) }
    pub mod text {
        pub fn shout(s: &str) -> String { s.to_uppercase() }
        pub fn helper() -> f64 { super::clamp01(2.0) }   // parent module
    }
}

use util::text::shout;                      // bring into scope
use std::collections::{HashMap, HashSet};   // grouped import
use std::io::{self, Write};                 // self = std::io
use std::fmt::Result as FmtResult;          // rename

fn main() {
    println!("{}", shout("hi"));
    println!("{}", crate::util::clamp01(-1.0));   // absolute path
    println!("{}", util::text::helper());
    let _cfg = config::load();
}

// net.rs
// pub mod http;
// pub use http::Client;                    // re-export: net::Client

// lib.rs prelude pattern
// pub mod prelude { pub use crate::{Client, Config, Error}; }
// user code: use mycrate::prelude::*;
```

CODE EXPLAINED

→ `mod config;` pulls in `config.rs` as a child module

→ `super::clamp01` calls the parent module's function

→ Grouped and renamed imports reduce clutter

→ `pub use` exposes `net::Client` without the `http` level

COMING FROM…

↔ **Python** files are modules automatically; Rust needs `mod`

↔ **Java** packages follow folders; one public class per file

↔ **Go** one package per folder; capitalized names are exported

THE RUST WAY

⇄ deep `use a::b::c::d::E` paths in user code → `pub use` re-exports

⇄ `use super::super::x` chains → `use crate::path::x`

PATHS

PREFIX	STARTS AT
`crate::`	Crate root
`super::`	Parent module
`self::`	Current module
`std:: / name::`	External crate
`::name`	Extern crate (explicit)

FILES

DECLARATION	LOADS
`mod a; (in main.rs)`	src/a.rs or src/a/mod.rs
`mod b; (in a.rs)`	src/a/b.rs
`mod c { … }`	Inline
`#[path = "x.rs"] mod c;`	Custom path

⚠ COMMON MISTAKES

- Creating a file but never declaring `mod` for it
- Declaring the same file with `mod` in two places
- Mixing `foo.rs` and `foo/mod.rs` for one module (E0761)

✓ TIPS & BEST PRACTICES

- One module per concept; keep files under a few hundred lines
- Use `pub use` to design a flat public API
- Let rust-analyzer add `use` lines

QUICK SUMMARY

- `mod x;` loads x.rs
- `use` brings names in
- `crate::` / `super::` / `self::`
- `pub use` — re-export

Visibility

private by default · `pub` · `pub(crate)` / `pub(super)` · struct field visibility · public enum variants · API surface design

CONTROLLING ACCESS

Everything is **private** to its module (and that module's children) by default. `pub` makes an item visible to anyone who can see its parent, `pub(crate)` limits it to the current crate and `pub(super)` to the parent module. Struct fields are private individually; enum variants are public whenever the enum is.

KEY POINTS

- A `pub struct` with private fields cannot be built outside its module
- Private fields protect invariants: expose constructors and getters
- Child modules can see their ancestors' private items
- `#[non_exhaustive]` stops outside code from matching exhaustively

VISIBILITY LEVELS

```
mod bank {
    pub struct Account {
        pub owner: String,                  // readable and writable outside
        balance: i64,                       // private: protects the invariant
    }

    impl Account {
        pub fn new(owner: &str) -> Self {
            Account { owner: owner.into(), balance: 0 }
        }
        pub fn balance(&self) -> i64 { self.balance }   // getter
        pub fn deposit(&mut self, amt: i64) -> Result<(), String> {
            if amt <= 0 { return Err("amount must be positive".into()); }
            self.balance += amt;
            audit::log(amt);
            Ok(())
        }
    }

    pub enum Kind { Checking, Savings }     // variants public automatically
    pub(crate) fn internal_rate() -> f64 { 0.02 }   // crate-wide only

    mod audit {
        pub(super) fn log(amt: i64) { println!("audit {amt}"); } // parent only
    }
}

use bank::Account;
let mut acct = Account::new("Ada");
acct.deposit(50).unwrap();
println!("{} {}", acct.owner, acct.balance());   // Ada 50
// acct.balance = 1_000_000;                // E0616: field `balance` is private
// let a = Account { owner: "x".into(), balance: 5 };   // E0451: private field
let k = bank::Kind::Savings;
println!("{}", bank::internal_rate());
// bank::audit::log(1);                     // E0603: module `audit` is private
```

CODE EXPLAINED

- → `balance` can only change through `deposit`
- → `pub(crate)` hides `internal_rate` from other crates
- → `pub(super)` lets only `bank` call `audit::log`
- → Private fields block struct literals outside the module

COMING FROM…

- ↔ **Java** `public` / package-private / `private`; `pub(crate)` ≈ package-private
- ↔ **C++** public / private / protected, per class not per module
- ↔ **Go** capitalized = exported; **Python** `_name` by convention only

THE RUST WAY

- ⇄ all fields `pub` on a type with invariants → private fields + methods
- ⇄ `pub` helpers used only inside the crate → `pub(crate)`

LEVELS

MODIFIER	VISIBLE TO
`(none)`	Current module + children
`pub(self)`	Same as none
`pub(super)`	Parent module
`pub(in crate::a)`	Given ancestor
`pub(crate)`	Whole crate
`pub`	Everyone

ERRORS

CODE	MEANING
`E0603`	Item is private
`E0616`	Field is private
`E0451`	Private field in literal
`E0624`	Method is private

⚠ COMMON MISTAKES

- Making every field `pub` and losing control of invariants
- Expecting `pub` items in a private module to be reachable
- Exposing internal types in public function signatures (warning: private type in public API)

✓ TIPS & BEST PRACTICES

- Start private; open up as needed
- Use `pub(crate)` for internal APIs
- Review your public surface with `cargo doc`

QUICK SUMMARY

- private by default
- `pub` / `pub(crate)` / `pub(super)`
- fields private per field
- enum variants follow the enum

Sheet 085

Cargo Dependencies

[dependencies] · semver "1.0" / "=1.2.3" · cargo add / remove / update · Cargo.lock · [dev-dependencies] · git & path deps

USING CRATES

Dependencies are listed in `Cargo.toml` and downloaded from **crates.io**. A version like `"1.4"` is a caret requirement: any compatible 1.x release ≥ 1.4. `Cargo.lock` pins the exact versions chosen. `[dev-dependencies]` are only for tests, examples and benchmarks; `[build-dependencies]` are for `build.rs`.

KEY POINTS

- `"1.4"` = `^1.4` = ≥1.4.0, <2.0.0
- For 0.x, `"0.4"` means ≥0.4.0, <0.5.0
- `cargo update` refreshes the lock file within requirements
- Features enable optional parts of a crate

CARGO.TOML DEPENDENCIES

```toml
[package]
name = "webtool"
version = "0.3.1"
edition = "2024"
rust-version = "1.85"           # minimum supported Rust

[dependencies]
serde = { version = "1", features = ["derive"] }
tokio = { version = "1.40", features = ["rt-multi-thread", "macros"] }
reqwest = { version = "0.12", default-features = false,
            features = ["json", "rustls-tls"] }
regex = "1.10"                  # ^1.10
uuid = "=1.8.0"                 # exactly this version
log = ">=0.4.20, <0.5"          # explicit range
anyhow = "1"

# local and git sources
shared = { path = "../shared" }
fancy = { git = "https://github.com/org/fancy", tag = "v2.1.0" }

# renamed dependency
http1 = { package = "http", version = "0.2" }

[dev-dependencies]
pretty_assertions = "1"
tempfile = "3"

[build-dependencies]
cc = "1"

[target.'cfg(windows)'.dependencies]
winapi = { version = "0.3", features = ["winuser"] }

# Terminal:
# cargo add serde -F derive      cargo remove regex
# cargo update                   cargo update -p tokio
# cargo tree -d                  # duplicate versions
# cargo outdated                 # (cargo-outdated plugin)
```

CODE EXPLAINED

- → Caret requirements accept compatible updates
- → `default-features = false` trims a crate to what you enable
- → `path` and `git` deps work for local or unreleased code
- → Target-specific tables add deps per platform

COMING FROM…

- ↔ **JavaScript** package.json `^1.4.0` and package-lock.json (same model)
- ↔ **Python** pyproject.toml `>=1.4,<2` + uv.lock / poetry.lock
- ↔ **Java** Maven/Gradle coordinates with exact versions by default

THE RUST WAY

- ⇄ `"*"` versions → a real requirement like `"1"`
- ⇄ pinning with `=` everywhere → caret + committed Cargo.lock

VERSION SYNTAX

REQUIREMENT	ALLOWS
`"1.4"`	≥1.4.0 <2.0.0
`"0.4"`	≥0.4.0 <0.5.0
`"~1.4"`	≥1.4.0 <1.5.0
`"=1.4.2"`	Exactly 1.4.2
`">=1.2, <1.5"`	Range
`"*"`	Anything (avoid)

SECTIONS

TABLE	USED FOR
`[dependencies]`	Normal code
`[dev-dependencies]`	Tests, benches, examples
`[build-dependencies]`	build.rs
`[target.'cfg(…)'.dependencies]`	Per platform
`[patch.crates-io]`	Override a crate

⚠ COMMON MISTAKES

- Not committing Cargo.lock for binaries
- Turning on `full` features and bloating compile times
- Two versions of the same crate (check `cargo tree -d`)

✓ TIPS & BEST PRACTICES

- Check crates on crates.io, docs.rs and lib.rs before adding
- Use `cargo audit` / `cargo deny` for security and licenses
- Set `rust-version` so users get a clear MSRV error

QUICK SUMMARY

- `[dependencies]` from crates.io
- `"1.4"` = compatible updates
- Cargo.lock pins versions
- dev / build / target deps

Sheet 086

Workspaces, Features & Profiles

[workspace] · members · shared lock & target/ · [features] · optional deps · --features / --all-features · [profile.release]

MULTI-CRATE PROJECTS AND BUILD OPTIONS

A **workspace** groups several crates that share one `Cargo.lock` and `target/` folder, so they build together and agree on dependency versions. **Features** are named, additive compile-time switches (`#[cfg(feature = "…")]`). **Profiles** tune optimization, debug info, LTO and panic strategy.

KEY POINTS

- Run commands for one member with `-p name`
- `[workspace.dependencies]` shares versions across members
- Features must be additive: enabling one must not break another
- `opt-level`, `lto`, `codegen-units`, `strip` live in profiles

WORKSPACE, FEATURES, PROFILES

```
# Cargo.toml (workspace root)
[workspace]
members = ["app", "core", "cli"]
resolver = "3"                          # default for edition 2024

[workspace.package]
edition = "2024"
version = "0.5.0"

[workspace.dependencies]
serde = { version = "1", features = ["derive"] }

[profile.release]
opt-level = 3
lto = "fat"                             # whole-program optimization
codegen-units = 1                       # slower build, faster binary
strip = true                            # smaller binary
panic = "abort"

# core/Cargo.toml
[package]
name = "core"
edition.workspace = true
version.workspace = true

[dependencies]
serde = { workspace = true }
chrono = { version = "0.4", optional = true }   # becomes a feature

[features]
default = ["json"]
json = []
time = ["dep:chrono"]                   # enable chrono only with "time"
full = ["json", "time"]

# In code:  #[cfg(feature = "time")] pub mod timestamps;
```

CODE EXPLAINED

- → `members` lists crates in the workspace
- → `workspace = true` inherits shared versions and settings
- → `dep:chrono` ties an optional dependency to a feature
- → Release profile settings trade build time for speed and size

COMING FROM…

- ↔ **JavaScript** npm/pnpm workspaces with a shared lock file
- ↔ **Java** Gradle multi-project builds / Maven modules
- ↔ **Go** go.work workspaces; build tags ≈ features

THE RUST WAY

- ⇄ copying versions into each member → `[workspace.dependencies]`
- ⇄ features that remove functionality → additive features only

PROFILE KEYS

KEY	EFFECT
`opt-level 0-3, "s", "z"`	Speed / size
`debug = true/false`	Debug info
`lto = "thin"/"fat"`	Link-time opt
`codegen-units = 1`	Better opt, slower build
`strip = true`	Remove symbols
`panic = "abort"`	No unwinding
`incremental`	Faster rebuilds

COMMANDS

COMMAND	DOES
`cargo build -p x`	One member
`cargo test --workspace`	All members
`--features a,b`	Enable features
`--all-features`	Everything
`--no-default-features`	Disable defaults
`cargo build --profile p`	Custom profile

⚠ COMMON MISTAKES

- Non-additive features that conflict when both are enabled
- Profiles in member Cargo.toml files (only the root counts)
- Testing only default features in CI

✓ TIPS & BEST PRACTICES

- Use `cargo hack --feature-powerset` to test feature combos
- Add a `[profile.profiling]` that inherits release with debug info
- Keep binaries thin; put logic in a library member

QUICK SUMMARY

- workspace — shared lock + target
- `[workspace.dependencies]`
- features — additive switches
- profiles — opt, lto, strip

Sheet 087

Publishing & Documentation

`cargo publish` · crates.io & docs.rs · `cargo doc` · `#![warn(missing_docs)]` · package metadata · `cargo semver-checks` · yanking

SHARING A CRATE

Publishing uploads a crate to **crates.io**, and **docs.rs** builds its documentation automatically. A published version is permanent; it can be **yanked** to stop new projects depending on it, but never deleted. Follow semver: breaking changes need a new major version (or minor for 0.x).

KEY POINTS

- Log in once with `cargo login <token>`
- Required metadata: description, license (or license-file)
- `cargo package --list` shows what will be uploaded
- Use `exclude` / `include` to trim the package

METADATA AND PUBLISHING

```toml
[package]
name = "tiny-geo"
version = "0.2.0"
edition = "2024"
description = "Small geometry helpers for 2-D shapes"
license = "MIT OR Apache-2.0"
repository = "https://github.com/ada/tiny-geo"
documentation = "https://docs.rs/tiny-geo"
readme = "README.md"
keywords = ["geometry", "math", "2d"]          # up to 5
categories = ["mathematics"]
rust-version = "1.85"
exclude = ["/benches/data", "*.png"]

[package.metadata.docs.rs]
all-features = true                            # document every feature

# Terminal:
# cargo doc --open --no-deps        preview docs
# cargo test --doc                  run doc examples
# cargo package --list              files that will ship
# cargo publish --dry-run           build the package without uploading
# cargo publish                     upload (permanent)
# cargo yank --version 0.2.0        block new users of a bad release
# cargo yank --version 0.2.0 --undo
# cargo semver-checks               detect accidental breaking changes

# src/lib.rs
# #![warn(missing_docs)]
# #![doc = include_str!("../README.md")]   // README as crate docs
# //! Small geometry helpers.
# #[doc(hidden)] pub mod internal;
# #[deprecated(since = "0.2.0", note = "use `area` instead")]
```

CODE EXPLAINED

→ `description` and `license` are required by crates.io

→ docs.rs reads `[package.metadata.docs.rs]`

→ `--dry-run` checks that the package builds

→ Yanking hides a version from new lock files only

COMING FROM…

↔ **JavaScript** `npm publish` / `npm deprecate` (npm allows unpublish briefly)

↔ **Python** build + `twine upload` to PyPI; Read the Docs for docs

↔ **Java** Maven Central with signed artifacts; Javadoc jars

THE RUST WAY

⇄ breaking changes in a patch release → bump major (or minor for 0.x)

⇄ separate hand-written docs → doc comments + README as crate docs

SEMVER FOR CRATES

CHANGE	BUMP (1.X)	BUMP (0.X)
Bug fix	Patch	Patch
New public item	Minor	Patch
Remove/rename public item	Major	Minor
New required trait method	Major	Minor
Raise MSRV	Minor (common practice)	Minor

⚠ COMMON MISTAKES

- Publishing secrets or large test data (check `package --list`)
- Expecting to delete a published version
- Breaking changes without a version bump

✓ TIPS & BEST PRACTICES

- Dual-license MIT OR Apache-2.0 like the Rust ecosystem
- Run `cargo semver-checks` in CI before releases
- Use `release-plz` or `cargo-release` to automate

QUICK SUMMARY

- `cargo publish` — permanent
- docs.rs builds docs
- yank, never delete
- semver discipline

Standard Library Tour

`time::{Instant, Duration}` · `SystemTime` · `chrono` / `jiff` dates · `rand` crate · `process::Command` · `std::env`

EVERYDAY UTILITIES

The standard library is small but solid. `Instant` and `Duration` time code, `SystemTime` gives wall-clock time, and `process::Command` runs other programs. Dates and random numbers live in crates: **chrono** or **jiff** for calendars and time zones, **rand** for randomness.

KEY POINTS

- `Instant` is monotonic: use it for measuring
- `SystemTime` can go backwards (clock changes)
- std has no date formatting or time zones
- `Command` does not use a shell unless you call one

TIME, PROCESSES, CRATES

```rust
use std::time::{Duration, Instant, SystemTime, UNIX_EPOCH};
use std::process::Command;
use std::thread;

let start = Instant::now();
thread::sleep(Duration::from_millis(50));
println!("{:?}", start.elapsed());          // 50.1ms
println!("{}", start.elapsed().as_millis() >= 50);   // true

let d = Duration::from_secs(90) + Duration::from_millis(500);
println!("{} {:?}", d.as_secs_f64(), d);    // 90.5 90.5s

let unix = SystemTime::now().duration_since(UNIX_EPOCH).unwrap().as_secs();
println!("{unix}");                         // e.g. 1794500000

let out = Command::new("git").args(["rev-parse", "--short", "HEAD"]).output()?;
if out.status.success() {
    println!("commit {}", String::from_utf8_lossy(&out.stdout).trim());
}
let status = Command::new("ls").arg("-l").current_dir("/tmp").status()?;
println!("exit {:?}", status.code());       // Some(0)

// chrono (cargo add chrono)
use chrono::{Local, NaiveDate, Duration as CDuration};
let today = Local::now().date_naive();
let due = NaiveDate::from_ymd_opt(2026, 12, 31).unwrap();
println!("{} days left", (due - today).num_days());
println!("{}", Local::now().format("%Y-%m-%d %H:%M"));
let next_week = today + CDuration::days(7);

// rand 0.9 (cargo add rand)
use rand::Rng;
let mut rng = rand::rng();
let roll: u8 = rng.random_range(1..=6);
let coin: bool = rng.random();
```

CODE EXPLAINED

- `Instant::elapsed` measures durations reliably
- Durations add and print in readable form
- `Command::output` captures stdout/stderr and the exit status
- chrono handles calendar dates and formatting

COMING FROM…

- **Python** `time.perf_counter`, `datetime`, `subprocess.run`, `random`
- **Java** `System.nanoTime`, `java.time`, `ProcessBuilder`, `Random`
- **Go** `time.Since`, `time.Format`, `os/exec`, `math/rand`

THE RUST WAY

- measuring with `SystemTime` → `Instant`
- `Command::new("sh").arg("-c")` with user input → pass args directly

STD UTILITIES

ITEM	USE
`Instant`	Measure elapsed time
`Duration`	Time spans
`SystemTime`	Wall clock, timestamps
`thread::sleep`	Pause
`process::Command`	Run programs
`env::var / args`	Environment
`std::hint::black_box`	Benchmark guard

COMMON CRATES

CRATE	FOR
`chrono`	Dates, formatting
`jiff`	Dates + time zones
`rand`	Random numbers
`uuid`	UUIDs
`regex`	Regular expressions
`itertools`	Extra iterator tools

⚠ COMMON MISTAKES

- Using SystemTime for benchmarks
- Passing user input through a shell (`sh -c`)
- Expecting date formatting in std

✓ TIPS & BEST PRACTICES

- Use `humantime` to parse/print durations like "5m"
- Check `status.success()` after running a command
- Seed rand with a fixed `StdRng` for reproducible tests

QUICK SUMMARY

- `Instant` / `Duration`
- `SystemTime` — wall clock
- `process::Command`
- chrono / jiff / rand crates

Unit Tests

BUILT-IN TESTING

Unit tests live next to the code in a `#[cfg(test)] mod tests` module, compiled only for `cargo test`. Each `#[test]` function passes if it returns without panicking. Because the module is a child, tests can call **private** functions. Tests run in parallel threads by default.

KEY POINTS

- `use super::*;` imports the code under test
- Output is captured unless a test fails (or `--nocapture`)
- `cargo test parse` runs tests whose names contain "parse"
- `#[ignore]` skips slow tests unless `--ignored`

UNIT TESTS

```rust
pub fn add(a: i32, b: i32) -> i32 { a + b }
fn parse_pair(s: &str) -> Result<(i32, i32), String> {     // private
    let (a, b) = s.split_once(',').ok_or("missing comma")?;
    let a = a.trim().parse().map_err(|_| "bad a")?;
    Ok((a, b.trim().parse().map_err(|_| "bad b")?))
}
pub fn divide(a: i32, b: i32) -> i32 {
    if b == 0 { panic!("divide by zero"); }
    a / b
}
#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn adds_numbers() {
        assert_eq!(add(2, 3), 5);
        assert_ne!(add(2, 2), 5, "math is broken");
    }
    #[test]
    fn parses_private_fn() {                // private items are visible
        assert_eq!(parse_pair("1, 2"), Ok((1, 2)));
        assert!(parse_pair("1 2").is_err());
    }
    #[test]
    #[should_panic(expected = "divide by zero")]
    fn panics_on_zero() { divide(1, 0); }
    #[test]
    fn returns_result() -> Result<(), String> {
        let (a, b) = parse_pair("4,5")?;    // ? instead of unwrap
        assert_eq!(a + b, 9);
        Ok(())
    }
}
```

CODE EXPLAINED

→ `#[cfg(test)]` keeps tests out of normal builds

→ `assert_eq!` prints both values when they differ

→ `should_panic(expected = …)` checks the panic message

→ Tests returning Result can use `?`

COMING FROM…

↔ **Python** pytest discovers `test_*` functions; asserts are plain `assert`

↔ **Java** JUnit 5 `@Test`, `assertEquals`, `assertThrows`

↔ **Go** `func TestX(t *testing.T)` in `_test.go` files

THE RUST WAY

⇄ `assert!(a == b)` → `assert_eq!(a, b)` (better failure output)

⇄ `.unwrap()` chains in tests → return `Result` and use `?`

MACROS & ATTRIBUTES

ITEM	USE
`#[test]`	Mark a test
`assert!(cond, msg…)`	Boolean check
`assert_eq! / assert_ne!`	Equality
`#[should_panic(expected)]`	Expect panic
`#[ignore]`	Skip by default
`#[cfg(test)]`	Test-only code

CARGO TEST FLAGS

FLAG	EFFECT
`name`	Filter by substring
`-- --exact`	Exact name
`-- --nocapture`	Show output
`-- --test-threads=1`	Serial
`-- --ignored`	Run ignored
`--release`	Optimized

⚠ COMMON MISTAKES

- Tests sharing files or globals while running in parallel
- Forgetting `use super::*;`
- `should_panic` without `expected` (any panic passes)

✓ TIPS & BEST PRACTICES

- Name tests as behaviors: `rejects_empty_name`
- Use `pretty_assertions` for readable diffs
- Use `cargo-nextest` for faster, isolated runs (sheet 091)

QUICK SUMMARY

- `#[cfg(test)] mod tests`
- `#[test]` + `assert_eq!`
- `should_panic` / Result tests
- `cargo test filter`

Integration & Doc Tests

TESTING THE PUBLIC API

Integration tests in the `tests/` folder are separate crates that use your library exactly as a user would: only `pub` items are reachable. **Doc tests** are the code blocks in `///` comments; they compile and run with `cargo test`, keeping examples correct. Binary-only crates need a `lib.rs` to be integration-tested.

KEY POINTS

- Each file in `tests/` is its own test crate
- Shared helpers go in `tests/common/mod.rs` (not `tests/common.rs`)
- Doc test lines starting with `#` are hidden but run
- Code block attributes: `no_run`, `ignore`, `should_panic`, `compile_fail`

INTEGRATION AND DOC TESTS

```rust
// src/lib.rs
/// Returns the slug form of a title.
///
/// ```
/// use blogtool::slugify;
/// assert_eq!(slugify("Hello World"), "hello-world");
/// ```
///
/// Hidden setup lines:
/// ```
/// # use blogtool::slugify;
/// let s = slugify("A  B");
/// assert_eq!(s, "a-b");
/// ```
/// ```should_panic
/// blogtool::slugify("");   // panics on empty input
/// ```
/// ```no_run
/// blogtool::publish_to("https://example.com");   // compiled, not run
/// ```
pub fn slugify(s: &str) -> String {
    assert!(!s.is_empty(), "empty title");
    s.split_whitespace().map(str::to_lowercase).collect::<Vec<_>>().join("-")
}
pub fn publish_to(_url: &str) {}
// tests/common/mod.rs
// pub fn sample_titles() -> Vec<&'static str> { vec!["One Two", "Three"] }
// tests/slug_tests.rs
// mod common;
// use blogtool::slugify;
// #[test]
// fn slugs_sample_titles() {
//     for t in common::sample_titles() {
//         assert!(!slugify(t).contains(' '));
//     }
// }
// cargo test                    unit + integration + doc tests
// cargo test --doc              doc tests only
```

CODE EXPLAINED

- → Doc examples run as tests, so docs stay accurate
- → `#` lines run but do not appear in rendered docs
- → `should_panic` and `no_run` change how a block is tested
- → Integration tests import the crate by name, like users

COMING FROM…

- ↔ **Python** doctest + a `tests/` folder for pytest
- ↔ **Java** separate `src/test` tree; no runnable Javadoc examples
- ↔ **Go** `Example` functions in `_test.go` run and appear in docs

THE RUST WAY

- ⇄ examples in README that drift → doc tests (`include_str!` the README)
- ⇄ testing internals from `tests/` → unit tests in the module

TEST KINDS

KIND	LOCATION	SEES
Unit	src/*.rs mod tests	Private items
Integration	tests/*.rs	Public API
Doc	/// code blocks	Public API
Example	examples/*.rs	Public API (built by test)

DOC BLOCK ATTRIBUTES

ATTRIBUTE	EFFECT
(none)	Compile + run
no_run	Compile only
ignore	Skip
should_panic	Must panic
compile_fail	Must not compile
text	Not Rust

⚠ COMMON MISTAKES

- Putting helpers in `tests/common.rs` (treated as a test crate)
- Integration-testing a binary crate with no lib target
- Doc examples that use `unwrap` on fallible network calls without `no_run`

✓ TIPS & BEST PRACTICES

- Keep `main.rs` thin and test the library
- Use `?` in doc tests by ending with `# Ok::<(), Box<dyn std::error::Error>>(())`
- Run `cargo test --doc` in CI

QUICK SUMMARY

- `tests/` — public API tests
- `tests/common/mod.rs` helpers
- doc tests run with cargo test
- `no_run` / `should_panic`

Benchmarks & Debugging

MEASURING AND INVESTIGATING

Use **criterion** for statistically sound benchmarks with `cargo bench`. Debug with `dbg!` for quick looks or `rust-lldb` / `rust-gdb` (or an IDE) for breakpoints. **cargo expand** shows macro output, **Miri** detects undefined behavior in unsafe code, and **cargo-nextest** runs tests faster with per-test isolation.

KEY POINTS

- Benchmark only `--release` builds (cargo bench does this)
- Use `black_box` so the optimizer does not delete benchmarked work
- Miri runs on nightly: `cargo +nightly miri test`
- Profilers: `cargo flamegraph`, `samply`, `perf`

CRITERION AND TOOLS

```
// Cargo.toml
// [dev-dependencies]
// criterion = "0.5"
// [[bench]]
// name = "fib"
// harness = false

// benches/fib.rs
use criterion::{criterion_group, criterion_main, Criterion};
use std::hint::black_box;

fn fib(n: u64) -> u64 { if n < 2 { n } else { fib(n - 1) + fib(n - 2) } }
fn fib_iter(n: u64) -> u64 {
    let (mut a, mut b) = (0, 1);
    for _ in 0..n { (a, b) = (b, a + b); }
    a
}

fn bench(c: &mut Criterion) {
    c.bench_function("fib 20 recursive", |b| b.iter(|| fib(black_box(20))));
    c.bench_function("fib 20 iter", |b| b.iter(|| fib_iter(black_box(20))));
}
criterion_group!(benches, bench);
criterion_main!(benches);

// $ cargo bench
// fib 20 recursive   time: [15.2 µs 15.3 µs 15.4 µs]
// fib 20 iter        time: [6.1 ns 6.2 ns 6.2 ns]

// Tools
// $ cargo install cargo-expand && cargo expand module::path
// $ cargo +nightly miri test       UB detector for unsafe code
// $ cargo install cargo-nextest && cargo nextest run
// $ cargo install flamegraph && cargo flamegraph --bin app
```

CODE EXPLAINED

→ `harness = false` lets criterion provide `main`

→ `black_box` prevents the compiler from precomputing `fib(20)`

→ criterion reports a confidence interval for each benchmark

→ Command-line debuggers wrap lldb/gdb with Rust pretty-printers

COMING FROM…

↔ **Go** built-in `go test -bench` and `pprof`

↔ **Java** JMH for benchmarks; IDE debuggers

↔ **Python** `timeit`, `pytest-benchmark`, `pdb`

THE RUST WAY

⇄ timing with `Instant` in a loop → criterion (or divan)

⇄ `println!` debugging spread everywhere → `dbg!` or a debugger

TOOLS

TOOL	USE
`criterion / divan`	Benchmarks
`dbg!`	Quick inspection
`rust-lldb / rust-gdb`	Breakpoints
`cargo expand`	Macro output
`Miri`	Undefined behavior
`cargo-nextest`	Fast test runner
`cargo flamegraph / samply`	Profiling
`cargo llvm-cov`	Coverage

⚠ COMMON MISTAKES

- Benchmarking debug builds
- Benchmarks the optimizer folds away (no `black_box`)
- Debugging release builds without debug info

✓ TIPS & BEST PRACTICES

- Add `debug = true` to a profiling profile for symbols
- Compare against a baseline with criterion's saved runs
- Run Miri on any crate with unsafe code

QUICK SUMMARY

- criterion + `cargo bench`
- `black_box`
- dbg! / lldb / gdb
- expand · Miri · nextest

Threads

`thread::spawn` · `JoinHandle::join` · `move` closures · `thread::scope` · `thread::sleep` · `available_parallelism`

OS THREADS

`thread::spawn` runs a closure on a new OS thread and returns a `JoinHandle`; `join()` waits and returns the closure's result, or `Err` if it panicked. Spawned closures must own their data (`move`) because the thread may outlive the caller. **Scoped threads** (`thread::scope`) can borrow local data safely because they are joined before the scope ends.

KEY POINTS

- When `main` returns, remaining threads are killed
- A panicking thread does not crash others; `join` reports it
- `thread::Builder` sets name and stack size
- Use rayon for data parallelism over collections

SPAWNING THREADS

```rust
use std::thread;
use std::time::Duration;

let handle = thread::spawn(|| {
    for i in 1..=3 {
        println!("worker {i}");
        thread::sleep(Duration::from_millis(10));
    }
    42                                      // thread's return value
});
println!("main continues");
let result = handle.join().unwrap();        // wait: 42

let data = vec![1, 2, 3];
let h = thread::spawn(move || data.iter().sum::<i32>());   // data moved in
println!("{}", h.join().unwrap());          // 6

let handles: Vec<_> = (0..4)
    .map(|id| thread::spawn(move || id * id))
    .collect();
let squares: Vec<i32> =
    handles.into_iter().map(|h| h.join().unwrap()).collect();
println!("{squares:?}");                    // [0, 1, 4, 9]

let mut counts = [0u32; 2];
let words = ["a", "bb", "a", "ccc"];
thread::scope(|s| {                         // borrow locals safely
    let (left, right) = counts.split_at_mut(1);
    s.spawn(|| left[0] = words.iter().filter(|w| w.len() == 1).count() as u32);
    s.spawn(|| right[0] = words.len() as u32);
});                                         // both joined here
println!("{counts:?}");                     // [2, 4]

let r = thread::spawn(|| panic!("boom")).join();
println!("{}", r.is_err());                 // true
println!("{}", thread::available_parallelism()?.get());   // e.g. 8
```

CODE EXPLAINED

- → `join()` returns the closure's value
- → `move` gives the thread ownership of `data`
- → Scoped threads borrow `words` and parts of `counts`
- → A panic in a thread shows up as `Err` from `join`

COMING FROM…

- ↔ **C++** `std::thread` / `std::jthread`; capture rules are unchecked
- ↔ **Java** `Thread` / virtual threads; shared objects need manual locking
- ↔ **Go** goroutines are lighter (M:N); Rust threads are 1:1 OS threads

THE RUST WAY

- ⇄ `Arc` just to share read-only locals with short threads → `thread::scope`
- ⇄ manual chunking for parallel loops → rayon `par_iter()`

API

CALL	DOES
`thread::spawn(f)`	Start thread
`h.join()`	Wait, get result
`thread::scope(|s| …)`	Borrowing threads
`s.spawn(f)`	Scoped spawn
`thread::sleep(d)`	Pause
`thread::current()`	Handle to self
`thread::Builder`	Name, stack size
`available_parallelism()`	CPU count hint

⚠ COMMON MISTAKES

- Borrowing locals in `thread::spawn` (E0373: may outlive borrowed value)
- Forgetting to `join` and losing work when main exits
- Spawning one thread per tiny task (use a pool or rayon)

✓ TIPS & BEST PRACTICES

- Prefer `thread::scope` when threads finish inside a function
- Use rayon for CPU-bound loops
- Name threads to make debugging and panics clearer

QUICK SUMMARY

- `thread::spawn` + `join`
- `move` closures
- `thread::scope` borrows
- panics surface via join

Sheet 093

Channels

mpsc::channel · Sender / Receiver · send / recv / try_recv · cloning senders · sync_channel · iterating a receiver

MESSAGE PASSING

Channels move values between threads: "do not communicate by sharing memory; share memory by communicating." `mpsc::channel()` returns a `Sender` and a `Receiver` (multiple producers, single consumer). Sending **moves** the value to the receiving thread. When all senders are dropped, the receiver's loop ends.

KEY POINTS

- `channel()` is unbounded; `sync_channel(n)` blocks when full
- `recv()` blocks; `try_recv()` and `recv_timeout()` do not
- `send` fails only if the receiver is gone
- crossbeam-channel and flume add multi-consumer and `select!`

MPSC CHANNELS

```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

let (tx, rx) = mpsc::channel();
thread::spawn(move || {
    for i in 1..=3 {
        tx.send(format!("msg {i}")).unwrap();   // ownership moves
        thread::sleep(Duration::from_millis(5));
    }
});                                             // tx dropped here
for msg in rx {                                 // ends when all senders drop
    println!("{msg}");
}

let (tx, rx) = mpsc::channel::<(usize, u64)>();
for id in 0..3 {
    let tx = tx.clone();                        // one sender per producer
    thread::spawn(move || {
        let sum = (1..=1000u64).map(|n| n * id as u64).sum();
        tx.send((id, sum)).unwrap();
    });
}
drop(tx);                                       // drop the original
let mut results: Vec<_> = rx.iter().collect();
results.sort();
println!("{results:?}");            // [(0, 0), (1, 500500), (2, 1001000)]

let (tx, rx) = mpsc::sync_channel(2);       // bounded: backpressure
thread::spawn(move || for i in 0..5 { tx.send(i).unwrap(); });
thread::sleep(Duration::from_millis(10));
println!("{:?}", rx.try_recv());            // Ok(0)
println!("{:?}", rx.recv_timeout(Duration::from_secs(1)));   // Ok(1)

let (tx, rx) = mpsc::channel::<i32>();
drop(tx);
println!("{:?}", rx.recv());                // Err(RecvError): no senders
```

CODE EXPLAINED

- → Each `send` moves a String into the channel
- → `for msg in rx` stops once the sender is dropped
- → Clone the Sender for each producer, then drop the original
- → `sync_channel(2)` makes senders wait when 2 items are queued

COMING FROM…

- ↔ **Go** `ch := make(chan T, n)`, `ch <- v`, `range ch` (the model Rust copies)
- ↔ **Java** `BlockingQueue` (`LinkedBlockingQueue`, `ArrayBlockingQueue`)
- ↔ **Python** `queue.Queue` between threads

THE RUST WAY

- ⇄ `Arc<Mutex<Vec<T>>>` as a work queue → a channel
- ⇄ busy-looping on `try_recv` → `recv()` or `recv_timeout`

API

CALL	BEHAVIOR
`mpsc::channel()`	Unbounded
`mpsc::sync_channel(n)`	Bounded (0 = rendezvous)
`tx.send(v)`	Err if receiver gone
`tx.clone()`	Another producer
`rx.recv()`	Block
`rx.try_recv()`	Never block
`rx.recv_timeout(d)`	Block up to d
`for v in rx` / `rx.iter()`	Until senders drop

⚠ COMMON MISTAKES

- Keeping the original `tx` alive so the receive loop never ends
- Unbounded channels growing without limit under load
- Using std mpsc when multiple consumers are needed

✓ TIPS & BEST PRACTICES

- Drop the original sender after cloning for workers
- Use bounded channels for backpressure
- Use `tokio::sync::mpsc` inside async code

QUICK SUMMARY

- `mpsc::channel()` — tx / rx
- `send` moves the value
- clone tx for producers
- loop ends when senders drop

Sheet 094

Shared State: Arc, Mutex & Atomics

Arc<T> · Mutex<T> / lock() · RwLock · poisoning · AtomicUsize & Ordering · avoiding deadlocks

SHARING DATA BETWEEN THREADS

Arc<T> is a thread-safe reference-counted pointer that lets threads share ownership. Pair it with Mutex<T> to mutate: lock() returns a guard that unlocks when dropped, and the data is only reachable through the guard. RwLock allows many readers or one writer, and atomics handle simple counters without locks.

KEY POINTS

- lock() returns Result: Err means a thread panicked while holding it (poisoned)
- Keep critical sections short; never hold a lock across .await
- Lock in a consistent order to avoid deadlocks
- parking_lot offers faster, non-poisoning locks

UNDER THE HOOD

- Arc clone/drop use atomic increments; Rc uses plain ones
- std Mutex on Linux is a futex: uncontended lock/unlock costs a few atomic ops
- The guard's Drop calls unlock, so early returns and ? cannot leak a lock
- Atomic Ordering::Relaxed is enough for counters; use Acquire / Release to publish data

ARC + MUTEX + ATOMICS

```rust
use std::sync::{Arc, Mutex, RwLock};
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

let counter = Arc::new(Mutex::new(0));
let mut handles = vec![];
for _ in 0..8 {
    let c = Arc::clone(&counter);
    handles.push(thread::spawn(move || {
        for _ in 0..1000 { *c.lock().unwrap() += 1; }   // unlock at each ;
    }));
}
for h in handles { h.join().unwrap(); }
println!("{}", *counter.lock().unwrap());   // 8000

let hits = Arc::new(AtomicUsize::new(0));   // lock-free counter
thread::scope(|s| {
    for _ in 0..4 {
        s.spawn(|| { hits.fetch_add(1, Ordering::Relaxed); });
    }
});
println!("{}", hits.load(Ordering::Relaxed));   // 4

let config = RwLock::new(String::from("v1"));
println!("{}", *config.read().unwrap());    // many readers at once
config.write().unwrap().push_str("-patched");   // one writer

let poisoned = Arc::new(Mutex::new(0));
let p2 = Arc::clone(&poisoned);
let _ = thread::spawn(move || { let _g = p2.lock().unwrap(); panic!() }).join();
println!("{}", poisoned.is_poisoned());     // true
let value = poisoned.lock().unwrap_or_else(|e| e.into_inner());   // recover
```

CODE EXPLAINED

- → Each thread gets its own Arc clone of the same Mutex
- → The guard unlocks at the end of each statement
- → Atomics count without a lock
- → A panic while locked poisons the Mutex; into_inner recovers the data

COMING FROM…

- ↔ **C++** std::shared_ptr + std::mutex (data not tied to the lock)
- ↔ **Java** synchronized / ReentrantLock; AtomicInteger
- ↔ **Go** sync.Mutex beside the data (by convention only)

THE RUST WAY

- ⇄ Mutex<i32> counters → AtomicI32 / AtomicUsize
- ⇄ locking inside a hot loop → accumulate locally, lock once

TYPES

TYPE	USE
Arc<T>	Shared ownership across threads
Mutex<T>	Exclusive access
RwLock<T>	Many readers / one writer
AtomicBool / AtomicUsize …	Lock-free primitives
OnceLock<T>	Init once
Condvar	Wait for a condition
Barrier	Sync N threads

ATOMIC ORDERINGS

ORDERING	USE
Relaxed	Counters, stats
Acquire / Release	Publish data to another thread
AcqRel	Read-modify-write both
SeqCst	Simplest, strongest

⚠ COMMON MISTAKES

- Holding a guard while calling code that locks the same Mutex (deadlock)
- Holding a std Mutex guard across .await
- Rc<RefCell<T>> across threads (E0277: not Send)

✓ TIPS & BEST PRACTICES

- Wrap shared state in a struct with methods that lock internally
- Prefer message passing when ownership can move
- Use parking_lot::Mutex or dashmap for heavy contention

QUICK SUMMARY

- Arc — shared ownership
- Mutex / RwLock guards
- atomics for counters
- short critical sections

Send & Sync

Send · Sync · why Rc isn't Send · auto traits · opting out · compile-time data-race freedom

THREAD-SAFETY MARKER TRAITS

Send means a value can be **moved** to another thread; Sync means &T can be **shared** between threads (T is Sync if &T is Send). The compiler implements both automatically for types made of Send/Sync parts. `thread::spawn` requires `Send + 'static`, which is how Rust rejects data races at compile time.

KEY POINTS

- Most types are Send + Sync: ints, String, Vec, Arc<Mutex<T>>
- `Rc<T>`: neither (non-atomic count)
- `Cell` / `RefCell`: Send but not Sync
- Raw pointers are neither; `unsafe impl` opts back in

SEND AND SYNC IN PRACTICE

```
use std::rc::Rc;
use std::sync::{Arc, Mutex};
use std::cell::RefCell;
use std::thread;

fn assert_send<T: Send>() {}
fn assert_sync<T: Sync>() {}

assert_send::<String>();
assert_sync::<Vec<u8>>();
assert_send::<Arc<Mutex<i32>>>();
assert_sync::<Arc<Mutex<i32>>>();
assert_send::<RefCell<i32>>();              // can move to another thread…
// assert_sync::<RefCell<i32>>();           // …but not be shared: error
// assert_send::<Rc<i32>>();                // E0277: Rc cannot be sent

let rc = Rc::new(5);
// thread::spawn(move || println!("{rc}"));
// error[E0277]: `Rc<i32>` cannot be sent between threads safely
let arc = Arc::new(5);                      // fix: atomic counting
thread::spawn(move || println!("{arc}")).join().unwrap();

fn run_in_background<F>(f: F) -> thread::JoinHandle<()>
where
    F: FnOnce() + Send + 'static,           // what spawn requires
{
    thread::spawn(f)
}
run_in_background(|| println!("ok"));

struct Handle(*mut u8);                     // raw pointer: not Send
unsafe impl Send for Handle {}              // SAFETY: we promise exclusive use

use std::marker::PhantomData;
struct NotSend { _p: PhantomData<*const ()> }   // opt out on stable
```

CODE EXPLAINED

- → The `assert_*` helpers check traits at compile time
- → `RefCell` is Send but not Sync
- → Replacing `Rc` with `Arc` makes the value Send
- → `unsafe impl Send` is a promise the compiler cannot check

COMING FROM…

- ↔ **C++ / Java** thread safety is documentation and discipline
- ↔ **Go** the race detector finds races at runtime, in tests
- ↔ **Swift** `Sendable` (a direct analog, checked at compile time)

THE RUST WAY

- ⇄ wrapping everything in `Arc<Mutex>` "to be safe" → only what threads share
- ⇄ `unsafe impl Send` to silence an error → fix the type (Arc, Mutex)

COMMON TYPES

TYPE	SEND	SYNC
`i32, String, Vec<T>`	Yes	Yes
`Rc<T>`	No	No
`Arc<T> (T: Send+Sync)`	Yes	Yes
`Cell / RefCell`	Yes	No
`Mutex<T> (T: Send)`	Yes	Yes
`MutexGuard`	No	Yes
`*const T / *mut T`	No	No
`&T`	If T: Sync	If T: Sync

⚠ COMMON MISTAKES

- Trying to send `Rc` or `RefCell`-shared data to threads
- Forgetting `+ Send` on boxed trait objects used in threads
- Unsound `unsafe impl Sync` on types with interior mutability

✓ TIPS & BEST PRACTICES

- Read E0277 notes: they show which field is not Send
- Add `+ Send + Sync` to `Box<dyn Error>` in threaded apps
- Async runtimes need futures to be Send for multi-threaded executors

QUICK SUMMARY

- `Send` — move across threads
- `Sync` — share `&T` across threads
- auto-implemented
- data races fail to compile

Sheet 096

async / await & Futures

async fn · .await · Future trait · futures are lazy · async blocks · executors · async closures · async fn in traits

ASYNCHRONOUS RUST

An `async fn` returns a **Future**, a value representing work that has not run yet. Futures are **lazy**: nothing happens until something `.await`s them or an executor polls them. Rust has no built-in runtime; crates like Tokio provide the executor. Async suits many concurrent I/O tasks; threads suit CPU work.

KEY POINTS

- `.await` only works inside async code
- Async closures `async || {}` are stable since Rust 1.85
- `async fn` in traits is stable since 1.75
- Blocking calls inside async code stall the executor

UNDER THE HOOD

- The compiler turns each `async fn` into a **state machine** enum: one variant per `.await` point
- Locals that live across `.await` are stored in that enum (its size can grow large)
- `Future::poll` returns `Ready(v)` or `Pending`; a `Waker` tells the executor when to poll again
- Futures may be self-referential, so polling requires `Pin<&mut Self>`

ASYNC BASICS

```rust
use std::time::Duration;

async fn fetch_user(id: u32) -> String {
    tokio::time::sleep(Duration::from_millis(50)).await;   // yields, no block
    format!("user{id}")
}

#[tokio::main]
async fn main() {
    let fut = fetch_user(1);                // nothing runs yet (lazy)
    println!("created");
    let name = fut.await;                   // runs now
    println!("{name}");                     // user1

    let (a, b) = tokio::join!(fetch_user(2), fetch_user(3));   // concurrent
    println!("{a} {b}");                    // ~50ms total, not 100ms

    let block = async { fetch_user(4).await.len() };   // async block
    println!("{}", block.await);            // 5

    let greet = async |n: &str| format!("hi {n}");   // async closure (1.85)
    println!("{}", greet("Ada").await);
}

trait Store {
    async fn get(&self, key: &str) -> Option<String>;   // async fn in traits
}

// The Future trait (simplified):
// trait Future {
//     type Output;
//     fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)
//         -> Poll<Self::Output>;
// }
```

CODE EXPLAINED

- → Calling `fetch_user(1)` only builds a future
- → `.await` drives the future to completion
- → `join!` runs futures concurrently on one task
- → Async blocks and closures create futures inline

COMING FROM…

- ↔ **JavaScript** `async` / `await` with Promises, but Promises start eagerly
- ↔ **Python** `asyncio` coroutines are lazy like Rust futures
- ↔ **C#** `async Task<T>`; **Go** uses goroutines instead of async

THE RUST WAY

- ⇄ `std::thread::sleep` in async code → `tokio::time::sleep(...).await`
- ⇄ awaiting independent futures one by one → `join!` / `try_join!`

PIECES

ITEM	MEANING
`async fn f() -> T`	Returns impl Future<Output = T>
`f().await`	Run and wait
`async { … }`	Future block
`async move { … }`	Owns captures
`async |x| …`	Async closure
`Future::poll`	Executor API
`Pin`	No moving after poll

THREADS VS ASYNC

ITEM	THREADS	ASYNC
Best for	CPU work	Many I/O waits
Cost per task	MBs of stack	Bytes (state machine)
Scheduling	OS	Runtime (cooperative)
Blocking calls	Fine	Stall others

⚠ COMMON MISTAKES

- Calling an async fn and forgetting `.await` (warning: unused future)
- Blocking I/O or heavy CPU loops inside async tasks
- Holding a std `MutexGuard` across `.await` (future not Send)

✓ TIPS & BEST PRACTICES

- Use `spawn_blocking` for blocking work (sheet 097)
- Keep large locals out of futures that live across awaits
- Pick one runtime (usually Tokio) per application

QUICK SUMMARY

- `async fn` returns a Future
- futures are lazy
- `.await` drives them
- runtime (Tokio) executes

Tokio

THE MOST COMMON ASYNC RUNTIME

Tokio provides the executor, timers, async I/O, channels and sync primitives. `#[tokio::main]` starts a multi-threaded runtime. `tokio::spawn` runs a future as an independent **task** (like a lightweight thread). `join!` waits for several futures, `select!` takes the first to finish, and `spawn_blocking` moves blocking work off the async threads.

KEY POINTS

- `cargo add tokio -F full` (or pick features)
- Spawned tasks need `Send + 'static` futures
- `JoinHandle::await` returns `Result<T, JoinError>`
- Use `tokio::sync::Mutex` only when holding the lock across `.await`

TOKIO TASKS, TIMERS, CHANNELS

```rust
use std::time::Duration;
use tokio::sync::mpsc;
use tokio::time::{sleep, timeout};

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let h = tokio::spawn(async { 7 });      // independent task
    println!("{}", h.await?);               // 7

    let (a, b) = tokio::join!(work("a", 30), work("b", 10)); // both at once
    println!("{a} {b}");
    tokio::select! {                        // first one wins, other is dropped
        v = work("fast", 5) => println!("got {v}"),
        _ = sleep(Duration::from_secs(1)) => println!("timed out"),
    }
    let (tx, mut rx) = mpsc::channel::<String>(16);   // bounded async channel
    for id in 0..3 {
        let tx = tx.clone();
        tokio::spawn(async move { tx.send(format!("job {id}")).await.ok(); });
    }
    drop(tx);
    while let Some(msg) = rx.recv().await { println!("{msg}"); }

    let hash = tokio::task::spawn_blocking(|| expensive_cpu_hash()).await?;
    println!("{hash}");
    Ok(())
}

async fn work(name: &str, ms: u64) -> String {
    sleep(Duration::from_millis(ms)).await;
    name.to_uppercase()
}
fn expensive_cpu_hash() -> u64 { (0..1_000_000u64).fold(0, |h, x| h ^ x) }
```

CODE EXPLAINED

- → `tokio::spawn` returns a handle you can `.await`
- → `join!` runs both futures and waits for both
- → `select!` cancels the losing branch by dropping it
- → `spawn_blocking` keeps CPU work off the async worker threads

COMING FROM…

- ↔ **JavaScript** `Promise.all` ≈ `join!`, `Promise.race` ≈ `select!`
- ↔ **Python** `asyncio.gather`, `asyncio.wait_for`, `asyncio.Queue`
- ↔ **Go** goroutines + channels + `select` (built into the language)

THE RUST WAY

- ⇄ `std::sync::mpsc` in async code → `tokio::sync::mpsc`
- ⇄ CPU-heavy loops in tasks → `spawn_blocking` or rayon

CORE API

ITEM	DOES
`#[tokio::main]`	Start runtime
`tokio::spawn(fut)`	New task
`join! / try_join!`	Wait for all
`select!`	First to finish
`time::sleep / timeout`	Timers
`sync::mpsc / oneshot / broadcast`	Channels
`sync::Mutex / RwLock`	Async locks
`task::spawn_blocking`	Blocking work
`fs / net / process`	Async I/O

RUNTIME FLAVORS

ATTRIBUTE	RUNTIME
`#[tokio::main]`	Multi-thread
`#[tokio::main(flavor = "current_thread")]`	Single thread
`#[tokio::test]`	Test runtime
`Runtime::new()?.block_on(f)`	Manual

⚠ COMMON MISTAKES

- Calling blocking std functions inside tasks
- Ignoring `JoinHandle` results and missing panics
- Assuming `select!` lets the losing future finish (it is dropped)

✓ TIPS & BEST PRACTICES

- Use `JoinSet` to manage a dynamic group of tasks
- Use `CancellationToken` (tokio-util) for graceful shutdown
- Enable only the Tokio features you use

QUICK SUMMARY

- `#[tokio::main]`
- `tokio::spawn` tasks
- `join!` / `select!` / `timeout`
- `spawn_blocking` for CPU

Sheet 098

HTTP Clients with reqwest

reqwest::get · Client · .json() / .text() · POST .json(&body) · headers & timeouts · reqwest::blocking

CALLING HTTP APIS

reqwest is the standard HTTP client, built on hyper and async by default. Create one `Client` and reuse it: it pools connections. Requests are built with method calls, sent with `.send().await`, and the response body is read as text, bytes or deserialized JSON via serde. A `blocking` API exists for simple scripts.

KEY POINTS

- `cargo add reqwest -F json` (+ tokio)
- Non-2xx statuses are **not** errors until `error_for_status()`
- Set timeouts on the Client builder
- The `blocking` feature must not be used inside an async runtime

REQWEST

```rust
use serde::{Deserialize, Serialize};
use std::time::Duration;

#[derive(Debug, Deserialize)]
struct Repo { name: String, stargazers_count: u32 }
#[derive(Serialize)]
struct NewTodo<'a> { title: &'a str, done: bool }
#[tokio::main]
async fn main() -> Result<(), reqwest::Error> {
    let body = reqwest::get("https://httpbin.org/ip").await?.text().await?;
    println!("{body}");

    let client = reqwest::Client::builder()
        .user_agent("cheatsheet-demo/1.0")
        .timeout(Duration::from_secs(10))
        .build()?;

    let repo: Repo = client
        .get("https://api.github.com/repos/rust-lang/rust")
        .header("Accept", "application/vnd.github+json")
        .send().await?
        .error_for_status()?                // 4xx/5xx → Err
        .json().await?;                     // deserialize
    println!("{} ⭐ {}", repo.name, repo.stargazers_count);

    let resp = client
        .post("https://httpbin.org/post")
        .bearer_auth("TOKEN")
        .query(&[("page", "1")])
        .json(&NewTodo { title: "write sheet", done: false })
        .send().await?;
    println!("{}", resp.status());          // 200 OK

    Ok(())
}
```

CODE EXPLAINED

- → `reqwest::get` is a one-off shortcut
- → The builder sets a user agent and timeout once
- → `error_for_status` turns HTTP errors into `Err`
- → `.json(&body)` serializes the request; `.json()` parses the response

COMING FROM…

- ↔ **Python** `requests` / `httpx` (`httpx.AsyncClient` ≈ reqwest Client)
- ↔ **JavaScript** `fetch()` with `await res.json()`
- ↔ **Java** `java.net.http.HttpClient`; **Go** `net/http` Client

THE RUST WAY

- ⇄ `reqwest::get` in a loop → one shared `Client`
- ⇄ checking only `?` on send → also `error_for_status()`

REQUEST BUILDER

METHOD	DOES
`.get/.post/.put/.delete(url)`	Start request
`.header(k, v)`	Header
`.bearer_auth(t) / .basic_auth`	Auth
`.query(&[…])`	URL params
`.json(&body)`	JSON body
`.form(&[…])`	Form body
`.timeout(d)`	Per request
`.send().await`	Execute

RESPONSE

METHOD	RETURNS
`.status()`	StatusCode
`.headers()`	HeaderMap
`.error_for_status()`	Err on 4xx/5xx
`.text().await`	String
`.json::<T>().await`	T
`.bytes().await`	Bytes

⚠ COMMON MISTAKES

- Creating a new Client per request (no connection reuse)
- Assuming a 404 returns `Err` from `send()`
- Calling `reqwest::blocking` from inside Tokio (panics)

✓ TIPS & BEST PRACTICES

- Store the Client in app state and clone it (cheap Arc)
- Use `rustls-tls` to avoid OpenSSL system deps
- Add retries with `reqwest-middleware` / `reqwest-retry`

QUICK SUMMARY

- one shared `Client`
- `.send().await?`
- `error_for_status()`
- `.json()` both ways

Web Servers with Axum

HTTP SERVICES

Axum (from the Tokio team) builds web services from async **handler** functions. Handler arguments are **extractors** that parse the request (path params, query strings, JSON bodies, shared state), and return values implement `IntoResponse`. Middleware comes from the tower ecosystem, such as `tower-http` for tracing, CORS and compression.

KEY POINTS

- `cargo add axum tokio -F tokio/full serde -F serde/derive`
- Axum 0.8 path syntax: `/users/{id}`
- The body extractor (`Json`) must be the last argument
- Share state with `with_state` and `State<T>` (T: Clone)

AN AXUM SERVICE

```rust
use axum::{extract::{Path, Query, State}, http::StatusCode,
           response::IntoResponse, routing::get, Json, Router};
use serde::{Deserialize, Serialize};
use std::sync::{Arc, Mutex};

#[derive(Clone, Default)]
struct AppState { todos: Arc<Mutex<Vec<Todo>>> }
#[derive(Clone, Serialize, Deserialize)]
struct Todo { id: u32, title: String }
#[derive(Deserialize)]
struct Paging { limit: Option<usize> }
async fn list(State(s): State<AppState>, Query(p): Query<Paging>)
    -> Json<Vec<Todo>> {
    let todos = s.todos.lock().unwrap();
    Json(todos.iter().take(p.limit.unwrap_or(50)).cloned().collect())
}
async fn get_one(State(s): State<AppState>, Path(id): Path<u32>)
    -> impl IntoResponse {
    match s.todos.lock().unwrap().iter().find(|t| t.id == id) {
        Some(t) => Ok(Json(t.clone())),
        None => Err((StatusCode::NOT_FOUND, "no such todo")),
    }
}
async fn create(State(s): State<AppState>, Json(t): Json<Todo>)
    -> impl IntoResponse {
    s.todos.lock().unwrap().push(t.clone());
    (StatusCode::CREATED, Json(t))
}

#[tokio::main]
async fn main() {
    let app = Router::new()
        .route("/todos", get(list).post(create))
        .route("/todos/{id}", get(get_one))
        .with_state(AppState::default());
    let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
    axum::serve(listener, app).await.unwrap();
}
```

CODE EXPLAINED

- → Routes map paths and methods to handler functions
- → Extractors like `Path(id)` destructure in the parameter list
- → Tuples of (StatusCode, body) are valid responses
- → `with_state` makes `AppState` available to every handler

COMING FROM…

- ↔ **Python** FastAPI (typed params ≈ extractors)
- ↔ **JavaScript** Express / Hono route handlers
- ↔ **Java** Spring `@GetMapping` + `@PathVariable` ; **Go** net/http + chi

THE RUST WAY

- ⇄ parsing `req.uri()` by hand → `Path` / `Query` extractors
- ⇄ `std::sync::Mutex` held across `.await` → lock, copy, drop, then await

EXTRACTORS

EXTRACTOR	GETS
`Path<T>`	URL params /{id}
`Query<T>`	?key=value
`Json<T>`	JSON body (last)
`Form<T>`	Form body
`State<T>`	Shared state
`HeaderMap`	Headers
`Extension<T>`	Middleware data

RESPONSES

RETURN	BECOMES
`&str / String`	text/plain
`Json(v)`	application/json
`StatusCode`	Empty body
`(StatusCode, body)`	Status + body
`Result<A, B>`	Either
`Html(s)`	text/html

⚠ COMMON MISTAKES

- Putting `Json<T>` before other extractors (compile error)
- Old `/:id` path syntax with Axum 0.8 (now `/{id}`)
- Long-running blocking work inside handlers

✓ TIPS & BEST PRACTICES

- Add `tower_http::trace::TraceLayer` for request logs
- Return `Result<T, AppError>` with `impl IntoResponse for AppError`
- Test routers with `tower::ServiceExt::oneshot`

QUICK SUMMARY

- `Router` + `.route()`
- extractors in parameters
- `IntoResponse` returns
- state via `with_state`

Sheet 100

Logging with log & tracing

log facade · env_logger · tracing · #[instrument] · spans & events · tracing-subscriber · RUST_LOG

STRUCTURED DIAGNOSTICS

The **log** crate is a facade: libraries call `info!` / `warn!`, and the application picks a backend such as `env_logger`. **tracing** adds **spans** (timed scopes with fields) and structured key-value events, which matters for async code where many tasks interleave. Both are filtered with the `RUST_LOG` environment variable.

KEY POINTS

- Libraries depend only on `log` or `tracing`, never a backend
- Levels: error, warn, info, debug, trace
- `RUST_LOG=info,my_crate=debug` filters per module
- tracing can also capture `log` records via `tracing-log`

LOG AND TRACING

```rust
// ---- log + env_logger ----
use log::{debug, error, info, warn};

fn init_simple() {
    env_logger::init();                    // reads RUST_LOG
    info!("starting v{}", env!("CARGO_PKG_VERSION"));
    warn!("disk at {}%", 91);
    debug!("only with RUST_LOG=debug");
    error!(target: "db", "connection lost");
}

// ---- tracing ----
use tracing::{info_span, instrument, Level};
use tracing_subscriber::EnvFilter;

#[instrument(skip(password), fields(user = %name))]
async fn login(name: &str, password: &str) -> bool {
    tracing::info!("checking credentials");  // inside the login span
    tracing::debug!(attempt = 1, "verifying");
    !password.is_empty()
}

#[tokio::main]
async fn main() {
    tracing_subscriber::fmt()
        .with_env_filter(EnvFilter::from_default_env())   // RUST_LOG
        .with_target(false)
        .init();

    let span = info_span!("request", id = 42, path = "/login");
    {
        let _enter = span.enter();      // sync code only: never across .await
        tracing::info!(status = 200, ms = 12.5, "handled"); // structured fields
    }
    let ok = login("ada", "secret").await;  // async: #[instrument] instead
    tracing::event!(Level::WARN, ok, "login finished");
}
```

CODE EXPLAINED

- `env_logger::init()` sets the log backend once
- `#[instrument]` wraps a function in a span with its arguments
- `skip(password)` keeps secrets out of logs
- Fields like `status = 200` are recorded as structured data

COMING FROM…

- **Java** SLF4J facade + Logback (same split as log + env_logger)
- **Python** `logging` module; structlog for structured events
- **Go** `log/slog` structured logging; OpenTelemetry for spans

THE RUST WAY

- `println!` for diagnostics → `tracing::info!` / `log::info!`
- formatting values into the message → structured fields

CRATES

CRATE	ROLE
log	Facade (macros)
env_logger	Simple backend
tracing	Spans + events
tracing-subscriber	Formatting, filters
tracing-appender	File output
tracing-opentelemetry	Export traces

RUST_LOG EXAMPLES

VALUE	SHOWS
info	info and above
debug	debug and above
warn,myapp=trace	Per crate
myapp::db=debug	Per module
off	Nothing

⚠ COMMON MISTAKES

- Initializing a logger in a library
- Logging secrets (use `skip` / redact)
- Forgetting to initialize any backend, so nothing prints

✓ TIPS & BEST PRACTICES

- Use tracing for async services, log for small tools
- Use `%` for Display and `?` for Debug in fields: `user = %name`
- Output JSON in production with `.json()` on the fmt subscriber

QUICK SUMMARY

- `log` facade + backend
- `tracing` spans & events
- `#[instrument]`
- filter with `RUST_LOG`

Databases with sqlx

SQL FROM RUST

sqlx is an async, pure-Rust SQL toolkit for PostgreSQL, MySQL and SQLite. Its `query!` macros check SQL against a real database **at compile time**, catching typos and type mismatches before running. Connection pools are cheap to clone. Alternatives: **rusqlite** (sync SQLite), **Diesel** (typed query builder) and **SeaORM** (async ORM).

KEY POINTS

- `cargo add sqlx -F runtime-tokio,postgres,macros`
- Compile-time checks need `DATABASE_URL` or `cargo sqlx prepare`
- Always bind parameters (`$1` , `?`); never format SQL strings
- Migrations live in `migrations/` and run with `sqlx migrate run`

SQLX WITH POSTGRESQL

```rust
use sqlx::postgres::PgPoolOptions;
use sqlx::FromRow;

#[derive(Debug, FromRow)]
struct User { id: i64, name: String, email: Option<String> }

#[tokio::main]
async fn main() -> Result<(), sqlx::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let pool = PgPoolOptions::new().max_connections(5).connect(&url).await?;

    sqlx::migrate!("./migrations").run(&pool).await?;   // embedded migrations

    let id: i64 = sqlx::query_scalar(
        "INSERT INTO users (name, email) VALUES ($1, $2) RETURNING id")
        .bind("Ada").bind("ada@example.com")
        .fetch_one(&pool).await?;

    let user = sqlx::query_as!(User,        // checked at compile time
        "SELECT id, name, email FROM users WHERE id = $1", id)
        .fetch_one(&pool).await?;
    println!("{user:?}");

    let all: Vec<User> = sqlx::query_as("SELECT id, name, email FROM users")
        .fetch_all(&pool).await?;           // runtime-checked variant

    let mut tx = pool.begin().await?;       // transaction
    sqlx::query!("UPDATE users SET name = $1 WHERE id = $2", "Ada L.", id)
        .execute(&mut *tx).await?;
    tx.commit().await?;                     // dropped without commit → rollback
    println!("{} users", all.len());
    Ok(())
}
// $ sqlx migrate add create_users && sqlx migrate run     (sqlx-cli)
```

CODE EXPLAINED

- → A pool manages and reuses connections
- → `query_as!` verifies columns and types against the database
- → `.bind()` passes parameters safely
- → A transaction rolls back automatically if not committed

COMING FROM…

- ↔ **Python** SQLAlchemy / asyncpg; psycopg for Postgres
- ↔ **Java** JDBC / jOOQ (typed SQL) / JPA-Hibernate (ORM)
- ↔ **Go** database/sql + sqlc (compile-time generated, similar goal)

THE RUST WAY

- ⇄ `format!("… WHERE id = {id}")` → bind parameters
- ⇄ a new connection per request → a shared pool

SQLX API

CALL	RETURNS
`query("…").bind(x)`	Runtime-checked
`query!("…", x)`	Compile-time checked
`query_as!(T, …)`	Rows → struct
`query_scalar`	Single value
`.fetch_one / _optional / _all`	Row(s)
`.execute`	Rows affected
`pool.begin()`	Transaction

CHOOSING A CRATE

CRATE	STYLE
`sqlx`	Async, raw SQL, checked
`rusqlite`	Sync SQLite
`Diesel`	Typed query builder
`SeaORM`	Async ORM
`tokio-postgres`	Low-level Postgres

⚠ COMMON MISTAKES

- String-formatting SQL (injection risk)
- CI builds failing because no database is reachable (use `cargo sqlx prepare`)
- Forgetting `tx.commit()` and silently rolling back

✓ TIPS & BEST PRACTICES

- Commit the `.sqlx/` offline data for reproducible builds
- Use `Option<T>` for nullable columns
- Keep SQL in the code where reviewers can read it

QUICK SUMMARY

- sqlx — async SQL
- `query!` checked at compile time
- pools + transactions
- bind, never format

Declarative Macros

MACROS BY EXAMPLE

`macro_rules!` defines a macro by pattern matching on Rust syntax: each rule matches input tokens and expands to code. **Fragment specifiers** like `$x:expr` capture pieces, and `$( … ),*` handles repetition. Macros can take variable numbers of arguments and generate items, which functions cannot. Prefer functions when they suffice.

KEY POINTS

- Invoke with `!` : `my_vec![1, 2]`, `m!(…)`, `m!{…}`
- Rules are tried top to bottom
- Macros must be defined before use in a file (textual order)
- `#[macro_export]` makes a macro available at the crate root

UNDER THE HOOD

- Expansion happens after parsing, before type checking
- **Hygiene**: local variables created inside a macro do not clash with the caller's names
- Expanded code is then type-checked like hand-written code
- `cargo expand` shows the exact expansion

MACRO_RULES!

```
macro_rules! square {
    ($x:expr) => { $x * $x };
}
println!("{}", square!(2 + 3));              // 25: $x is one expression

macro_rules! my_vec {
    () => { Vec::new() };
    ($($item:expr),+ $(,)?) => {{        // one or more, optional trailing comma
        let mut v = Vec::new();
        $( v.push($item); )+
        v
    }};
}
let v: Vec<i32> = my_vec![1, 2, 3,];         // [1, 2, 3]

#[macro_export]
macro_rules! hashmap {
    ($($k:expr => $v:expr),* $(,)?) => {
        std::collections::HashMap::from([$(($k, $v)),*])
    };
}
let m = hashmap! { "a" => 1, "b" => 2 };

macro_rules! getter {                        // generate methods
    ($name:ident : $t:ty) => {
        pub fn $name(&self) -> &$t { &self.$name }
    };
}
struct User { name: String, age: u32 }
impl User { getter!(name: String); getter!(age: u32); }
```

CODE EXPLAINED

- → `square!(2 + 3)` treats `2 + 3` as one expression, so the result is 25
- → `$( … ),+` repeats for each comma-separated item
- → `ident` and `ty` fragments let macros generate methods
- → `#[macro_export]` makes `hashmap!` usable from other crates

COMING FROM…

- ↔ **C / C++** `#define` text substitution: no hygiene, no syntax awareness
- ↔ **Python** no macros; decorators and metaclasses cover some uses
- ↔ **Java** no macros; annotation processors / Lombok generate code

THE RUST WAY

- ⇄ a macro that could be a generic function → the function
- ⇄ C-style `#define` constants → `const` items

FRAGMENT SPECIFIERS

SPEC	MATCHES
`expr`	Expression
`ident`	Identifier
`ty`	Type
`pat`	Pattern
`stmt / block`	Statement / block
`item`	fn, struct, …
`literal`	"x", 5
`tt`	Any single token tree
`path / lifetime / vis / meta`	Others

REPETITION

SYNTAX	MEANS
`$( … ),*`	Zero or more, comma-separated
`$( … ),+`	One or more
`$( … )?`	Optional
`$(,)?`	Optional trailing comma
`$( … );*`	Semicolon-separated

⚠ COMMON MISTAKES

- Expecting C-like text substitution (macros work on tokens)
- Using a macro before its definition in the file
- Unreadable recursive macros where a function would do

✓ TIPS & BEST PRACTICES

- Add a `$(,)?` rule for trailing commas
- Use `cargo expand` to debug expansions
- Use `$crate::path` inside exported macros

QUICK SUMMARY

- `macro_rules!` pattern → code
- `$x:expr`, `$i:ident`, `$t:ty`
- `$( … ),*` repetition
- hygienic, token-based

Sheet 103

Procedural Macros

derive macros · attribute macros · function-like macros · `proc-macro` crate · `syn` / `quote` · reading `cargo expand` output

MACROS AS COMPILER PLUGINS

Procedural macros are Rust functions that run at compile time, take a `TokenStream` and return a new one. **Derive** macros (`#[derive(Serialize)]`) add impls, **attribute** macros (`#[tokio::main]`) transform an item, and **function-like** macros (`sql!(…)`) look like calls. They live in a separate crate with `proc-macro = true` and almost always use `syn` and `quote`.

KEY POINTS

- A proc-macro crate can only export macros
- `syn` parses tokens into a syntax tree
- `quote!` builds tokens with `#var` interpolation
- Errors: `syn::Error::new_spanned(...).to_compile_error()`

A DERIVE MACRO

```
// hello_derive/Cargo.toml: [lib] proc-macro = true · deps: syn, quote

// hello_derive/src/lib.rs
use proc_macro::TokenStream;
use quote::quote;
use syn::{parse_macro_input, DeriveInput};
#[proc_macro_derive(Hello)]
pub fn derive_hello(input: TokenStream) -> TokenStream {
    let ast = parse_macro_input!(input as DeriveInput);
    let name = &ast.ident;                    // the type's name
    let expanded = quote! {
        impl #name {
            pub fn hello() -> String {
                format!("Hello from {}!", stringify!(#name))
            }
        }
    };
    expanded.into()
}

#[proc_macro_attribute]
pub fn log_call(_attr: TokenStream, item: TokenStream) -> TokenStream {
    let f = parse_macro_input!(item as syn::ItemFn);
    let sig = &f.sig;
    let body = &f.block;
    let fname = sig.ident.to_string();
    quote! { #sig { println!("calling {}", #fname); #body } }.into()
}

// app/src/main.rs
// use hello_derive::{Hello, log_call};
// #[derive(Hello)]
// struct Pancakes;
// #[log_call]
// fn work() { println!("working"); }
// fn main() { println!("{}", Pancakes::hello()); work(); }
// → Hello from Pancakes!  calling work  working
```

CODE EXPLAINED

- → `parse_macro_input!` turns tokens into a `DeriveInput`
- → `#name` inside `quote!` inserts the type name
- → The attribute macro wraps the function body with a log line
- → Users import macros like any other item

COMING FROM…

- ↔ **Java** annotation processors (generate new files, cannot modify)
- ↔ **Python** decorators run at runtime, not compile time
- ↔ **C#** source generators; **Kotlin** KSP

THE RUST WAY

- ⇄ complex `macro_rules!` parsing → a proc macro with `syn`
- ⇄ a proc macro for something a trait + generics can do → the trait

KINDS

KIND	ATTRIBUTE	USED AS
Derive	`#[proc_macro_derive(N)]`	`#[derive(N)]`
Attribute	`#[proc_macro_attribute]`	`#[name(args)]`
Function-like	`#[proc_macro]`	`name!(…)`

TOOLING

CRATE / TOOL	USE
`syn`	Parse Rust syntax
`quote`	Generate tokens
`proc-macro2`	Testable token types
`darling`	Parse attribute args
`cargo expand`	View output
`trybuild`	Test compile errors

⚠ COMMON MISTAKES

- Trying to export normal functions from a proc-macro crate
- Panicking instead of emitting `compile_error!` with a span
- Heavy syn features slowing every build

✓ TIPS & BEST PRACTICES

- Read real expansions with `cargo expand` before writing your own
- Use spans so errors point at user code
- Keep proc macros thin; put logic in a normal library

QUICK SUMMARY

- TokenStream in → out
- derive · attribute · function-like
- `syn` parse · `quote!` generate
- separate proc-macro crate

Unsafe Rust

unsafe blocks & fns · the five unsafe superpowers · *const T / *mut T · unsafe impl · unsafe extern (2024) · // SAFETY: comments

OPTING OUT OF SOME CHECKS

unsafe unlocks five extra abilities that the compiler cannot verify, and makes **you** responsible for upholding Rust's rules. It does not turn off the borrow checker. The goal is to build small, well-reviewed unsafe cores wrapped in **safe APIs**, as Vec, String and Mutex do internally.

KEY POINTS

- Undefined behavior anywhere can break code everywhere
- Every `unsafe` block deserves a `// SAFETY:` comment
- Edition 2024: `unsafe extern` blocks, `unsafe_op_in_unsafe_fn` warns
- Test unsafe code with Miri

UNSAFE IN PRACTICE

```
let mut x = 10;
let r = &x as *const i32;                    // creating raw pointers is safe
println!("{}", unsafe { *r });               // 10 (SAFETY: x is alive)
let p = &raw mut x;                          // raw borrow syntax (1.82)
unsafe {
    // SAFETY: p points to x, which is alive and not otherwise borrowed.
    *p += 1;
    println!("{}", *p);                      // 11
}

/// # Safety
/// `idx` must be less than `v.len()`.
unsafe fn get_fast(v: &[u32], idx: usize) -> u32 {
    // SAFETY: the caller guarantees idx is in bounds.
    unsafe { *v.get_unchecked(idx) }
}
let v = [1, 2, 3];
let n = unsafe { get_fast(&v, 2) };          // 3

fn split_halves(s: &mut [i32]) -> (&mut [i32], &mut [i32]) {   // safe wrapper
    let mid = s.len() / 2;
    let ptr = s.as_mut_ptr();
    // SAFETY: the two ranges don't overlap and both lie within `s`.
    unsafe {
        (std::slice::from_raw_parts_mut(ptr, mid),
         std::slice::from_raw_parts_mut(ptr.add(mid), s.len() - mid))
    }
}

static mut COUNTER: u32 = 0;
unsafe { COUNTER += 1; }    // SAFETY: single-threaded, no other access

unsafe extern "C" {                          // 2024 edition syntax
    safe fn abs(x: i32) -> i32;              // declared safe to call
    fn strlen(s: *const std::ffi::c_char) -> usize;
}
println!("{}", abs(-5));                     // 5, no unsafe block needed
```

CODE EXPLAINED

→ Raw pointers can be created safely; dereferencing needs unsafe

→ `unsafe fn` documents its contract in a `# Safety` section

→ `split_halves` wraps unsafe code in a safe, sound API

→ `safe fn` inside `unsafe extern` marks foreign functions safe to call

COMING FROM…

↔ **C / C++** all code is effectively unsafe; UB is everywhere possible

↔ **Go** `unsafe` package; **C#** `unsafe` blocks with pointers

↔ **Java** `sun.misc.Unsafe` / FFM API; **Python** ctypes

THE RUST WAY

⇄ `unsafe` to get around the borrow checker → restructure, or use a safe std API

⇄ unsafe spread across a codebase → one small module with a safe API

THE FIVE SUPERPOWERS

ABILITY	EXAMPLE
Deref raw pointer	*ptr
Call unsafe fn	get_unchecked(i)
Access static mut	COUNTER += 1
Implement unsafe trait	unsafe impl Send
Access union fields	u.as_int

COMMON UB

BUG	EXAMPLE
Dangling pointer	Use after free
Aliasing &mut	Two &mut to one value
Out of bounds	get_unchecked(len)
Invalid value	bool from 2
Data race	static mut from threads
Uninit memory	Reading MaybeUninit early

⚠ COMMON MISTAKES

- Thinking `unsafe` disables the borrow checker (it does not)
- Missing `// SAFETY:` justifications
- `static mut` in multi-threaded code (use atomics or Mutex)

✓ TIPS & BEST PRACTICES

- Run `cargo +nightly miri test` on unsafe code
- Enable `#![deny(unsafe_op_in_unsafe_fn)]`
- Search crates.io first: someone may have a safe abstraction

QUICK SUMMARY

- five superpowers, same borrow rules
- you uphold the invariants
- `// SAFETY:` every block
- wrap in safe APIs

FFI & Calling C

TALKING TO C

Rust calls C functions declared in `extern "C"` blocks and exposes Rust functions to C with `extern "C" fn` plus `#[unsafe(no_mangle)]`. Strings cross the boundary as `CString` (owned, NUL-terminated) and `CStr` (borrowed). **bindgen** generates Rust declarations from C headers, **cbindgen** generates C headers from Rust, and `build.rs` with the `cc` crate compiles C sources.

KEY POINTS

- Use `#[repr(C)]` for structs shared with C
- Types: `c_int`, `c_char`, `c_void` in `std::ffi`
- `#[link(name = "z")]` or `build.rs` links native libraries
- Never let a Rust panic unwind into C (use `catch_unwind` or `extern "C-unwind"`)

FFI BOTH WAYS

```
use std::ffi::{c_char, c_int, CStr, CString};

// ---- calling C from Rust ----
#[link(name = "m")]
unsafe extern "C" {
    fn cos(x: f64) -> f64;
    fn strlen(s: *const c_char) -> usize;
}

let c = unsafe { cos(0.0) };                // 1.0
let s = CString::new("hello").unwrap();     // adds the NUL byte
let n = unsafe { strlen(s.as_ptr()) };      // 5

// ---- exposing Rust to C ----
#[repr(C)]
pub struct Point { pub x: f64, pub y: f64 }

#[unsafe(no_mangle)]                        // 2024 edition spelling
pub extern "C" fn point_len(p: Point) -> f64 { (p.x * p.x + p.y * p.y).sqrt() }

#[unsafe(no_mangle)]
pub extern "C" fn greet(name: *const c_char) -> c_int {
    if name.is_null() { return -1; }
    // SAFETY: caller passes a valid NUL-terminated string.
    let name = unsafe { CStr::from_ptr(name) }.to_string_lossy();
    println!("hello, {name}");
    0
}

// Cargo.toml for a C-callable library: [lib] crate-type = ["cdylib"]

// build.rs: fn main() { cc::Build::new().file("src/fast.c").compile("fast"); }

// Generate bindings:
// $ bindgen wrapper.h -o src/bindings.rs        (C → Rust)
// $ cbindgen --lang c -o include/mylib.h        (Rust → C)
```

CODE EXPLAINED

- → Calls into C are `unsafe` because Rust cannot check them
- → `CString::new` adds the NUL terminator C expects
- → `#[repr(C)]` gives `Point` C-compatible layout
- → `cdylib` / `staticlib` build a library C programs can link

COMING FROM…

- ↔ **C++** `extern "C"` linkage works the same way
- ↔ **Python** ctypes / cffi; PyO3 for Rust extensions
- ↔ **Java** JNI / Panama FFM API; **Go** cgo

THE RUST WAY

- ⇄ passing `&str` to C → `CString::new(s)?.as_ptr()`
- ⇄ hand-written C declarations for big headers → bindgen

TYPE MAPPING

C	RUST
`int`	c_int (i32)
`char*`	*const c_char
`void*`	*mut c_void
`size_t`	usize
`double`	f64
`struct (repr C)`	#[repr(C)] struct
`NULL`	std::ptr::null()

TOOLS

TOOL	DOES
`bindgen`	C headers → Rust
`cbindgen`	Rust → C header
`cc`	Compile C in build.rs
`pkg-config`	Find system libs
`PyO3 / napi-rs`	Python / Node bindings

⚠ COMMON MISTAKES

- Dropping a `CString` while C still holds its pointer
- Struct layout mismatch without `#[repr(C)]`
- Panics unwinding across `extern "C"` (aborts in recent Rust)

✓ TIPS & BEST PRACTICES

- Wrap raw FFI in a `-sys` crate plus a safe wrapper crate
- Use `CStr::to_str()` when you need validated UTF-8
- Use PyO3 / napi-rs instead of raw FFI for Python / Node

QUICK SUMMARY

- `extern "C"` both ways
- `#[repr(C)]` layouts
- `CString` / `CStr`
- bindgen / cbindgen / cc

Idiomatic Patterns

PATTERNS RUST CODE USES EVERYWHERE

Rust's type system makes some patterns especially effective: **newtypes** prevent mixing values, **builders** replace default arguments, **typestate** encodes valid state transitions in types, **RAII guards** pair setup with automatic cleanup, `From` impls make conversions and ? smooth, and returning `impl Iterator` keeps APIs lazy.

KEY POINTS

- Newtypes are free at runtime (same layout)
- Builders often consume and return `self`
- Typestate turns runtime state errors into compile errors
- Guards implement `Drop`

SIX PATTERNS

```
// Newtype
#[derive(Debug, Clone, Copy, PartialEq)]
struct UserId(u64);                          // not mixable with OrderId(u64)
// Builder
#[derive(Debug, Default)]
struct Request { url: String, timeout: u64, retries: u8 }
struct RequestBuilder(Request);
impl RequestBuilder {
    fn new(url: &str) -> Self {
        Self(Request { url: url.into(), timeout: 30, ..Default::default() })
    }
    fn timeout(mut self, s: u64) -> Self { self.0.timeout = s; self }
    fn build(self) -> Request { self.0 }
}
let req = RequestBuilder::new("https://x.io").timeout(5).build();
// Typestate
struct Draft; struct Published;
struct Post<S> { body: String, _s: std::marker::PhantomData<S> }
impl Post<Draft> {
    fn publish(self) -> Post<Published> {
        Post { body: self.body, _s: Default::default() }
    }
}
impl Post<Published> { fn url(&self) -> String { format!("/p/{}", self.body) } }
// RAII guard
struct Timer(&'static str, std::time::Instant);
impl Drop for Timer {
    fn drop(&mut self) { println!("{} took {:?}", self.0, self.1.elapsed()); }
}
{ let _t = Timer("load", std::time::Instant::now()); /* work */ }
// From conversions power `?`
impl From<u64> for UserId { fn from(v: u64) -> Self { UserId(v) } }
let id: UserId = 7.into();
// Iterator-returning API
struct Library { books: Vec<(String, u16)> }
impl Library {
    fn published_after(&self, y: u16) -> impl Iterator<Item = &str> + '_ {
        self.books.iter().filter(move |b| b.1 > y).map(|b| b.0.as_str())
    }
}
```

CODE EXPLAINED

- → `UserId` and `OrderId` are both u64 inside but cannot be mixed
- → The builder chains settings and fills the rest with defaults
- → Only `Post<Published>` has a `url` method
- → Returning `impl Iterator` lets callers chain without allocating

COMING FROM…

- ↔ **Java** builders and value objects are common; typestate is rare
- ↔ **Python** keyword arguments replace builders; NewType is only a hint
- ↔ **C++** strong typedefs, RAII and builders all apply

THE RUST WAY

- ⇄ raw `u64` IDs everywhere → newtypes
- ⇄ returning `Vec<T>` just to iterate → `impl Iterator<Item = T>`

PATTERNS

PATTERN	SOLVES
`Newtype`	Mixing similar values
`Builder`	Many optional settings
`Typestate`	Invalid transitions
`RAII guard`	Forgotten cleanup

MORE PATTERNS

PATTERN	SOLVES
`From / TryFrom`	Conversions, ? errors
`impl Iterator`	Lazy results
`Extension trait`	Methods on foreign types
`Enum state machine`	Runtime states

⚠ COMMON MISTAKES

- Builders that allow `build()` with required fields missing (use typestate or Result)
- Newtypes without the traits users need (derive them)
- Overusing typestate for simple runtime states

✓ TIPS & BEST PRACTICES

- Derive or forward traits on newtypes with `derive_more`
- Use `bon` / `typed-builder` to generate builders
- Name guards `_guard` so they live to the end of scope

QUICK SUMMARY

- newtype · builder
- typestate · RAII guard
- `From` conversions
- return `impl Iterator`

Sheet 107

Quick Reference — Syntax, Std & Compiler Errors

syntax · ownership · Option / Result combinators · iterator methods · collections · cargo commands · E0382 · E0499 · E0502 · E0505 · E0106 · E0597 · E0277 · E0308

SYNTAX

SYNTAX	DOES	SHEET #
`let / let mut x = 1;`	Bind / mutable	009
`const N: usize = 4;`	Constant	010
`fn f(a: i32) -> i32 { a }`	Function	046
`|x| x + 1`	Closure	048
`if c { a } else { b }`	If expression	027
`match v { P => e, _ => d }`	Match	031
`if let / let … else`	One pattern	033
`for x in &v { }`	Loop	030
`'a: loop { break 'a v; }`	Labeled loop	028

OWNERSHIP

SYNTAX	DOES	SHEET #
`let b = a;`	Move (non-Copy)	034
`a.clone()`	Deep copy	035
`&x / &mut x`	Borrow	036
`*r`	Deref	036
`&v[1..3]`	Slice	038
`fn f<'a>(x: &'a T) -> &'a T`	Lifetime	039
`mem::take / swap`	Move out	037
`Rc / Arc / RefCell`	Shared ownership	073

OPTION / RESULT

METHOD	DOES	SHEET #
`?`	Propagate None / Err	043
`unwrap_or(d) / _else(f)`	Default	042
`map / and_then`	Transform / chain	042
`ok_or(e)`	Option → Result	042
`map_err(f)`	Convert error	043
`ok() / err()`	Result → Option	043
`expect("why")`	Panic with reason	041
`.context("…")`	anyhow message	045

ITERATORS

METHOD	DOES	SHEET #
`iter / iter_mut / into_iter`	&T / &mut T / T	059
`map / filter / filter_map`	Transform	060
`enumerate / zip / chain`	Combine	060
`take / skip / step_by`	Limit	060
`flat_map / flatten`	Un-nest	060
`sum / count / fold`	Reduce	060
`any / all / find / position`	Search	061
`collect::<Vec<_>>()`	Gather	060

COLLECTIONS

TYPE	KEY METHODS	SHEET #
`Vec<T>`	push pop get retain	053
`[T; N] / &[T]`	len iter chunks	052
`String / &str`	push_str split trim	017
`HashMap<K, V>`	insert get entry	055
`BTreeMap<K, V>`	range first_key_value	056
`HashSet<T>`	insert contains union	057
`VecDeque<T>`	push_front pop_back	058
`BinaryHeap<T>`	push pop peek	058

CARGO

COMMAND	DOES	SHEET #
`cargo new / init`	Create	003
`cargo check`	Fast check	003
`cargo run --release`	Optimized run	003
`cargo test`	Tests	089
`cargo add x -F f`	Add dependency	085
`cargo fmt / clippy`	Format / lint	005
`cargo doc --open`	Docs	008
`cargo publish`	Release crate	087

BORROW CHECKER SURVIVAL GUIDE

ERROR CODE	MESSAGE	CAUSE	FIX	SHEET #
E0382	borrow of moved value	Value used after it moved (assignment, call, loop)	Borrow with &, clone deliberately, or restructure	034
E0499	cannot borrow as mutable more than once	Two `&mut` to the same value alive at once	Shorten the first borrow; split with `split_at_mut`	037
E0502	cannot borrow as mutable because it is also borrowed as immutable	Reading reference still alive during a mutation	Finish using the & first, or copy the value out	037
E0505	cannot move out because it is borrowed	Moving a value while a reference to it exists	End the borrow before the move, or clone	036
E0106	missing lifetime specifier	Returned reference with unclear source	Add `<'a>` tying output to an input, or return owned data	039
E0597	does not live long enough	Reference outlives the value it points to	Move the value to an outer scope or return it owned	039
E0277	the trait bound is not satisfied	Type lacks a required trait (Display, Send, Ord…)	Implement or derive the trait, or change the type	069
E0308	mismatched types	Wrong type, often a stray `;` or missing conversion	Remove the `;`, add `.into()` / `as` / `?`	004

MORE FREQUENT ERRORS

CODE	MEANING	USUAL FIX
E0425	Cannot find value	Typo or missing `use`
E0433	Unresolved path	Add the crate or `use`
E0599	No method found	Import the trait; check bounds
E0384	Assign twice to immutable	Add `mut`
E0596	&mut of non-mut binding	Declare `let mut`
E0373	Closure may outlive borrow	Add `move`
E0507	Move out of borrowed content	`.clone()`, `take()`, or borrow
E0515	Return reference to local	Return the owned value

READING AN ERROR

PART	TELLS YOU
`error[E0382]`	Code: `rustc --explain E0382`
`--> src/main.rs:9:5`	File, line, column
`value moved here`	Where it started
`value borrowed here after move`	Where it failed
`help: consider cloning`	Suggested fix
`note:`	Extra context

STRATEGIES

SITUATION	TRY
Fighting borrows in a loop	Collect indices / changes, apply after
Struct method borrow clash	Borrow fields separately
Need shared mutation	Rc<RefCell> / Arc<Mutex>
Lifetimes spreading	Own the data (String, Vec)
Option field move-out	`.take()` / `as_ref()`
Still stuck	Clone first, optimize later